# Masked Circuit Compiler in the Cardinal Random Probing Composability Framework

Sonia Belaïd[1], Victor Normand[1,2], and Matthieu Rivain[1]

[1] CryptoExperts, Paris, France
{firstname.lastname}@cryptoexperts.com,
[2] DIENS, Ecole normale supérieure, PSL University, CNRS, Inria, Paris, France

**Abstract.** Designing practically secure masked circuits remains a central problem in the field of cryptographic implementation. While most masking schemes have been proven secure in the classical probing model, this model fails to capture more advanced side-channel attacks such as horizontal attacks. In recent years, the community has shifted toward the more realistic random probing model, which offers stronger guarantees. Yet, balancing strong security with practical efficiency continues to be a significant challenge.

In this work, we introduce new tools and constructions that significantly improve the design and analysis of random probing secure circuits. First, we formalize new security notions that combine the benefits of *cardinal* and *general* Random Probing Composability (RPC), two recently introduced notions enabling more flexible and efficient composition of secure gadgets. We then show how uniformly random permutations can be applied to transform any cardinal or general RPC gadget into a so-called *uniformly* cardinal RPC gadget, thereby enhancing security at low cost.

Using these techniques, we propose the first non-linear multiplication gadget, inspired by the recursive construction from CHES 2016, that achieves concrete cardinal RPC security. We provide a detailed comparison with state-of-the-art multiplication gadgets in terms of both random probing advantage and implementation complexity. Building upon this gadget, we design a tighter random probing compiler that strategically uses permutations to improve security bounds while preserving efficiency. Finally, we apply our compiler to the AES and demonstrate improved performance and security compared to existing methods.

**Keywords:** Masking · Random probing model · Secure composition

## 1 Introduction

Many cryptographic schemes have been proven secure against black-box attacks by the research community. However, once implemented on physical devices, these schemes becomes vulnerable to *side-channel attacks*. These attacks exploit the physical characteristics of the device such as execution time, temperature, power consumption, or electromagnetic emissions during algorithm execution to infer sensitive values. *Masking* is one of the most widely deployed countermeasures against *side-channel attacks*. For a sensitive value $x$, it consists of splitting $x$ into $n$ shares such that the knowledge of any $(n-1)$ shares is insufficient to deduce $x$. As a result, an adversary must determine the values of all $n$ shares to recover $x$, significantly increasing the difficulty of the attack.

To assess the security of circuits protected by masking countermeasures, several *leakage models* have been formalized. These models characterize the capabilities of an adversary attempting to recover sensitive information, ultimately providing security guarantees for the implementation. The *d-probing model*, introduced in [18], allows an adversary to access $d$ internal values from a circuit execution, known as *probes*, and determines whether this limited information suffices to deduce the secret. A circuit is said to be *d-probing secure* if no set of $d$ probed values reveals any sensitive information about the secret. The $d$-probing model has been extensively studied due to its convenience for formal security proofs. However, it remains far from capturing the realities of physical leakages. For example, it does not model powerful horizontal attacks (see, e.g., [4]). In contrast, the *noisy leakage model* [20] assumes that every wire in the circuit leaks a noisy function

of its value, offering a more realistic representation of side-channel attacks. However, this model is much less convenient for formal proofs, which motivated a follow-up work that established a security reduction from the probing model to the noisy leakage model [15]. This reduction introduced an intermediate model, the *random probing model*, previously considered in [18,1]. In this model, each variable leaks its value independently with a fixed probability $p$, known as the *leakage rate*. Compared to the standard $d$-probing model, the random probing model enjoys a much tighter reduction to the noisy leakage model.

There are two main approaches to evaluating the random probing security of a circuit. The first consists in using automated tools that exhaustively analyze all possible probing tuples [6,7], or a representative subset using probabilistic methods [12]. The second approach involves deriving a theoretical, generic formula that upper bounds the security level for any number of shares. However, both techniques suffer from scalability limitations and become impractical for large circuits. To overcome this limitation, the more scalable strategy of *composition* consists in proving stronger local security properties for small gadgets. These properties are specifically designed to support secure composition, ensuring that the global circuit inherits a guaranteed level of random probing security. The notion of *random probing composability* (RPC), originally introduced in [6] and referred to as *threshold RPC* in [8] and in this work, enjoys trivial composability. A variant aimed at enabling tighter composition, introduced in [12] and termed *general RPC* in [8] and here, allows for more precise composition bounds but quickly becomes intractable when composing even a few gadgets. More recently, [8] formalized a new notion called *cardinal RPC*, which strikes a balance between compositional tightness and computational efficiency. An alternative approach to composability, based on *leakage diagrams*, was also proposed in [16,11], offering a different framework for reasoning about leakage propagation across composed gadgets.

An increasing number of works have focused on proposing masked circuit compilers aimed at achieving random probing security [1,3,2,17,6,9,10,11,19,8]. To the best of our knowledge, only a few of these works provide concrete security guarantees for actual circuits. This was first achieved by the line of research on *random probing expansion* [6,9,10], which builds upon the expansion strategy from [2] and attains arbitrary security levels for constant leakage probability $p$ up to $2^{-7.5}$. More recently, [11] revisited the ISW scheme [18] within the leakage diagram framework, deriving clean analytical expressions for random probing security as a function of the leakage probability $p$ and the number of shares $n$ in the underlying gadgets. The work of [19] introduced an algebraic approach to random probing security and succeeded in building a secure compiler for leakage rates up to $p = 2^{-5.6}$, leveraging a variant of the multiplication gadget from [4]. Finally, the recent work of [8] provides a concrete random-probing secure implementation of Raccoon [14], relying only on linear masking gadgets and leveraging the cardinal RPC framework.

*Our contributions.* In this work, we revisit and extend the recent cardinal RPC framework introduced in [8]. Our contributions are as follows:

- We formalize the notion of *random-wiring circuits* and their associated random probing security and composition properties, which underlie the construction from [8] but had not been previously formalized.
- We revisit the notion of *uniformly cardinal RPC*, and show that it achieves security guarantees as tight as general RPC, while significantly reducing the complexity of composition.
- We demonstrate how any gadget satisfying cardinal or general RPC can be transformed into a uniformly cardinal RPC gadget by applying random permutations to the input and output shares. This transformation offers improved security with minimal overhead.
- While [8] focuses solely on linear gadgets, which are enough for masking the Raccoon scheme, we complete the framework by designing a non-linear multiplication gadget inspired by [4]. We prove its cardinal RPC security and provide a detailed comparison with existing multiplication gadgets in terms of random probing security advantage and implementation complexity.
- We present our complete compiler along with its composition strategy, and apply it to the AES circuit, demonstrating that our approach outperforms state-of-the-art methods in terms of the achievable security-performance trade-off.

## 2 Preliminaries

### 2.1 Notations

Along the paper, $\mathbb{K}$ shall denote a finite field or a finite ring. For any $n \in \mathbb{N}$, we shall denote $[n]$ the integer set $[n] = [1, n] \cap \mathbb{Z}$. We shall further denote $\mathcal{P}([n]) = \{S \subseteq [n]\}$ the power set of $[n]$ and $\mathcal{S}_n$ the symmetric group on $n$ elements, i.e. the set of all possible permutations of the set $\{1, 2, \ldots, n\}$. Each element of $\mathcal{S}_n$ is a permutation, which is a bijective function that maps the set $\{1, 2, \ldots, n\}$ to itself.

For some tuple $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{K}^n$ and for some set $I \subseteq [n]$, the tuple $(x_i)_{i \in I}$ is denoted $\mathbf{x}|_I$. We use the general notation of $[\![x]\!] = (x_1, \ldots, x_n) \in \mathbb{K}^n$ for a sharing of a secret value $x$.

*Distributions.* Two probability distributions $D_1$ and $D_2$ are *$\varepsilon$-close*, denoted as $D_1 \approx_\varepsilon D_2$, if their statistical distance is upper bounded by $\varepsilon$, namely

$$\mathsf{SD}(D_1; D_2) := \frac{1}{2} \sum_x |p_{D_1}(x) - p_{D_2}(x)| \leq \varepsilon$$

where $p_{D_1}(\cdot)$ and $p_{D_2}(\cdot)$ represent the probability mass functions of $D_1$ and $D_2$.

*Distribution Envelopes.* Since exact probabilities are often difficult to compute directly, we follow the approach of [8] and instead rely on upper bounds expressed through the notion of a *probability envelope*. Specifically, for a discrete random variable $X \sim \mathcal{D}$, we define $\mathcal{E}$ as an upper envelope of the distribution, meaning that for any value $x$, the probability $\mathbb{P}(X = x)$ is upper bounded by $\mathcal{E}(x)$, which we denote by $X \lesssim \mathcal{E}$.

When dealing with probability distributions conditioned on disjoint events, we use collections of probability envelopes indexed by potentially complex structures. For example, we may consider collections $\boldsymbol{\mathcal{E}} = (\mathcal{E}_{(t_1^{\mathtt{out}}, \ldots, t_m^{\mathtt{out}})})$ indexed by tuples $(t_1^{\mathtt{out}}, \ldots, t_m^{\mathtt{out}}) \in [0, n]^m$ as well as collections $\boldsymbol{\mathcal{E}} = (\mathcal{E}_{(J_1, \ldots, J_m)})$ indexed by sets $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$. In this case, each tuple corresponds to a distinct distribution and its associated envelope.

### 2.2 Linear Sharing

In the following, the *$n$-linear decoding mapping*, denoted by $\mathsf{LinDec}$, refers to the function $\mathbb{K}^n \to \mathbb{K}$ defined, for any $n \in \mathbb{N}$ and $(x_1, \ldots, x_n) \in \mathbb{K}^n$, as

$$\mathsf{LinDec} : (x_1, \ldots, x_n) \mapsto x_1 + \ldots + x_n$$

We shall further consider that, for every $n, \ell \in \mathbb{N}$, on input $([\![x_1]\!], \ldots, [\![x_n]\!]) \in (\mathbb{K}^n)^\ell$ the $n$-linear decoding mapping acts as

$$\mathsf{LinDec} : ([\![x_1]\!], \ldots, [\![x_\ell]\!]) \mapsto (\mathsf{LinDec}([\![x_1]\!]), \ldots, \mathsf{LinDec}([\![x_\ell]\!]))$$

**Definition 1 (Linear Sharing from [6]).** *Let $n, \ell \in \mathbb{N}$. For any $x \in \mathbb{K}$, an $n$-sharing of $x$ is a random vector $[\![x]\!] \in \mathbb{K}^n$ such that $\mathsf{LinDec}([\![x]\!]) = x$. It is uniform if for any set $I \subseteq [n]$ with $|I| < n$ the tuple $[\![x]\!]|_I$ is uniformly distributed over $\mathbb{K}^{|I|}$. An $n$-linear encoding is a probabilistic algorithm $\mathsf{LinEnc}$ which on input a tuple $\mathbf{x} = (x_1, \ldots, x_\ell) \in \mathbb{K}^\ell$ outputs a tuple $[\![\mathbf{x}]\!] = ([\![x_1]\!], \ldots [\![x_\ell]\!]) \in (\mathbb{K}^n)^\ell$ such that $[\![x_i]\!]$ is a uniform $n$-sharing of $x_i$ for every $i \in [\ell]$.*

### 2.3 Circuits and Gadgets

An *arithmetic circuit* on a finite field (or finite ring) $\mathbb{K}$ is a labeled directed acyclic graph whose edges are *wires* and vertices are *arithmetic gates* processing operations on $\mathbb{K}$. The circuit is built from a base set of gates $\mathbb{B} = \{g : \mathbb{K}^\ell \to \mathbb{K}^m\}$, such as addition gates $(x_1, x_2) \mapsto x_1 + x_2$, multiplication gates $(x_1, x_2) \mapsto x_1 \cdot x_2$, and copy gates $x \mapsto (x, x)$. We consider *probabilistic* (or *randomized*) arithmetic circuit that further includes random gates. Each random gate outputs a fresh uniform random value from $\mathbb{K}$.

In the following, we shall call an *(n-share, ℓ-to-m) gadget*, a randomized arithmetic circuit that maps an input $[\![\mathbf{x}]\!] \in (\mathbb{K}^n)^\ell$ to an output $[\![\mathbf{y}]\!] \in (\mathbb{K}^n)^m$ such that $\mathbf{x} = \mathsf{LinDec}(\mathbf{x}) \in \mathbb{K}^\ell$ and $\mathbf{y} = \mathsf{LinDec}(\mathbf{y}) \in \mathbb{K}^m$ satisfy $\mathbf{y} = g(\mathbf{x})$ for some function $g$.

## 3 Random Probing Security for Randomized-Wiring Circuits

In this section, we define the randomized-wiring arithmetic circuits that our constructions will rely on. We also recall the simulation-based notions used to formalize random probing security and adapt them to randomized-wiring arithmetic circuits.

### 3.1 Randomized-Wiring Circuits

A *randomized-wiring arithmetic circuit* is a (probabilistic) arithmetic circuit, as defined in Section 2.3, in which the wiring, i.e. the connections between gates, is determined randomly. Specifically, a randomized-wiring arithmetic circuit $C$ consists of a fixed set of gates, while the wires connecting these gates are sampled according to a predefined probability distribution. Formally, $C$ is characterized by: (1) a set of gates, and (2) a distribution over possible wire configurations. The randomness of this distribution is parameterized by a *random tape* $\rho$. We denote by $C^\rho$ the deterministic arithmetic circuit obtained by instantiating $C$ with a particular realization of $\rho$.

This model is particularly useful for capturing masked circuits and gadgets in which operations are performed on shares whose indices are randomized. An example is the cardinal RPC refresh gadget, $\mathsf{RPRefresh}$, introduced in [8]. This shall also capture the gadgets proposed in this paper that apply random permutations to their input or output shares (see Section 4).

It is important to emphasize that in our model, the random tape $\rho$ is *fully leaked to the adversary.* That is, the simulator in the random probing model receives $C^\rho$ for a randomly chosen $\rho$. Consequently, the randomness in wiring is fundamentally different from the randomness from random gates. First, they operate over different domains: random wiring typically involves selecting indices from $[n]$, whereas random gates produce values in the base field (or ring) $\mathbb{K}$. Second, and more critically, while random gates must be implemented with mechanisms that limit leakage (quantified by the leakage probability $p$ in the random probing model), randomized wiring does *not* require such protection. Since the wiring randomness is entirely leaked to the adversary, its generation might rely on much lighter mechanisms in practice.

In the following, we extend existing notions of random probing security and composition to encompass randomized-wiring arithmetic circuits and gadgets.

### 3.2 Random Probing Security

Let $p \in [0, 1]$ be some constant leakage probability parameter, a.k.a. the *leakage rate*. In the $p$-random probing model, an evaluation of an arithmetic circuit $C^\rho$, for a randomly chosen $\rho$, leaks the value carried by each wire with a probability $p$, all the wire leakage events being mutually independent. We also consider that the adversary knows the circuit instance $C^\rho$ and its underlying wiring which is captured by allowing a full leakage of the random tape $\rho$.

As in [6], we formally define the random-probing leakage of an arithmetic circuit from the two following probabilistic algorithms:

– The *leaking-wires sampler* takes as input an arithmetic circuit $C^\rho$, for a randomly chosen $\rho$, and a probability $p \in [0, 1]$, and outputs a set $\mathcal{W}$, denoted as

$$\mathcal{W} \leftarrow \mathsf{LeakingWires}(C^\rho, p) \ ,$$

where $\mathcal{W}$ is constructed by including each wire label from the circuit with probability $p$ to $\mathcal{W}$ (where all the probabilities are mutually independent).

– The *assign-wires sampler* takes as input an arithmetic circuit $C^\rho$, for a randomly chosen $\rho$, a set of wire labels $\mathcal{W}$ (subset of the wire labels of $C^\rho$), and an input $\mathbf{x}$, and it outputs a $|\mathcal{W}|$-tuple $\mathbf{w} \in \mathbb{K}^{|\mathcal{W}|} \cup \{\perp\}$ ($\{\perp\}$ if $|\mathcal{W}| = 0$), denoted as

$$\mathbf{w} \leftarrow \mathsf{AssignWires}(C^\rho, \mathcal{W}, \mathbf{x}) \ ,$$

where $\mathbf{w}$ corresponds to the assignments of the wires of $C^\rho$ with label in $\mathcal{W}$ for an evaluation on input $\mathbf{x}$.

We now extend the definitions of *random probing leakage* and *random probing security* from [6] to the setting of randomized-wiring circuits. We remark that in the special case where the wiring distribution is a singleton (i.e. the circuit has a fixed wiring), these definitions coincide with the original ones for standard arithmetic circuits.

**Definition 2 (Random Probing Leakage).** *The $p$-random probing leakage of a randomized-wiring arithmetic circuit $C$ on input $\mathbf{x}$ is the distribution $\mathcal{L}_p(C, \mathbf{x})$ obtained by picking a wiring random tape $\rho$ and composing the leaking-wires and assign-wires samplers applies to $C^\rho$:*

$$\mathcal{L}_p(C, \mathbf{x}) \stackrel{id}{=} \left(\mathsf{AssignWires}(C^\rho, \mathsf{LeakingWires}(C^\rho, p), \mathbf{x}) \mid \rho \leftarrow \$\right)$$

**Definition 3 (Random Probing Security).** *A randomized-wiring arithmetic circuit $C$ with $\ell \cdot n \in \mathbb{N}$ input gates is $(p, \varepsilon)$-random probing secure with respect to encoding $\mathsf{Enc}$ if there exists a simulator $\mathsf{Sim}$ such that for every $\mathbf{x} \in \mathbb{K}^\ell$:*

$$\mathsf{Sim}(C^\rho) \approx_\varepsilon \mathcal{L}_p(C, \mathsf{Enc}(x))$$

### 3.3 Random Probing Composability

Proving the random probing security directly is generally feasible only for small gadgets and does not scale for larger circuits. To address this, we rely on composability notions: by proving stronger local security properties for individual gadgets, we can demonstrate that their composition guarantees global random probing security for the entire circuit.

Several composability notions for random probing security have been introduced in the literature, each offering different trade-offs between precision and verification efficiency. The *(threshold) random probing composability* (RPC) notion, introduced in [6], requires that the random probing leakage of a gadget's internal wires, along with any set of up to $t$ output shares, can be perfectly simulated using only up to $t$ input shares, except with a simulation failure probability $\varepsilon$. A key advantage of this notion is its *trivial composability*: any circuit composed of threshold RPC gadgets remains threshold RPC, with the overall failure probability being the sum of the individual failure probabilities. While powerful, the threshold RPC notion lacks tightness, as requiring more than $t$ input shares to simulate a single gadget does not necessarily imply failure in a composed setting.

To address this limitation, the notion of *general random probing composability* (general RPC) was later introduced [12,8]. In this framework, the simulation failure probability is specified for each combination of output shares (to be simulated along with internal random probing leakage) and input shares (used in the simulation). This leads to the concept of the *probe distribution table* (PDT) introduced in [12], also referred to as the general RPC envelope in [8] and in our work. However, computing such tables or envelopes is exponential in the number of input and output shares, quickly becoming intractable when composing even a small number of gadgets into a larger cluster. Specifically, a cluster with $m$ input sharings and $\ell$ output

sharings has a general RPC envelope that contains (bounds on) $2^{(m+\ell)\cdot n}$ distinct failure probabilities, where $n$ is the number of shares per sharing.

To mitigate this, the notion of *cardinal random probing composability* (cardinal RPC) was introduced in [8] as a compromise between tightness and efficiency. This approach characterizes the simulation failure probability for every number of output shares (to be simulated) and input shares (used in the simulation), considering only the cardinality of these sets rather than their specific elements. Thus, a cardinal RPC envelope for a cluster with $m$ input sharings and $\ell$ output sharings consists of (bounds on) $(n+1)^{(m+\ell)}$ distinct failure probabilities. This is polynomial in the number of shares $n$, rather than exponential (as general RPC), though it remains exponential in the number of input and output sharings. This abstraction hence significantly reduces the complexity of composition while still preserving a meaningful degree of tightness.

We adapt these three composability notions to the setting of randomized-wiring circuits. As before, in the special case where the wiring distribution is a singleton, these adapted definitions coincide with their original counterparts.

**Definition 4 (Threshold Random Probing Composability).** *Let $n, \ell, m, t \in \mathbb{N}$. An $n$-share randomized-wiring gadget $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ is $(p, t, \varepsilon)$-*threshold random probing composable (threshold RPC for short) *for some $p \in [0,1]$ if there exists a PPT stateful two-stage simulator $(\mathsf{Sim}_1, \mathsf{Sim}_2)$ such that for every shared input $[\![x]\!] \in (\mathbb{K}^n)^\ell$ and for every set collection $(J_1, \ldots, J_m)$ such that $|J_1| \le t, \ldots, |J_m| \le t$, the outputs of the following experiment:*

$$\begin{aligned}
&\rho \leftarrow \$\\
&\mathcal{W} \leftarrow \mathsf{LeakingWires}(G^\rho, p)\\
&I_1, \ldots, I_\ell \leftarrow \mathsf{Sim}_1(G^\rho, \mathcal{W}, J_1, \ldots, J_m)\\
&out_1, out_2 \leftarrow \mathsf{Sim}_2\big([\![x_1]\!]|_{I_1}, \ldots, [\![x_\ell]\!]|_{I_\ell}\big)\\
&\textit{return } (I_1, \ldots, I_\ell, out_1, out_2)
\end{aligned}$$

*are such that*

1. $\mathbb{P}(|I_1| \le t, \ldots, |I_\ell| \le t) \le \varepsilon$,
2. $$(out_1, out_2) \overset{id}{=} \left( \underbrace{\mathcal{L}_{\mathcal{W}}\left(G^\rho, [\![\boldsymbol{x}]\!], [\![\boldsymbol{y}]\!]\right)}_{\texttt{Induced Random probing leakage}} \quad , \quad \underbrace{\left([\![y_1]\!]|_{J_1}, \ldots, [\![y_m]\!]|_{J_m}\right)}_{\texttt{Output leakage}} \right),$$

   *where $\mathcal{W} \leftarrow \mathsf{LeakingWires}(C, p)$ and $[\![\boldsymbol{y}]\!] \leftarrow G^\rho([\![\boldsymbol{x}]\!])$.*

**Definition 5 (General Random Probing Composability).** *Let $n, \ell, m \in \mathbb{N}$. Let $\boldsymbol{\mathcal{E}}^g$ be a collection of probability envelopes indexed by $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ and defined over $\mathcal{P}([n])^\ell$. An $n$-share randomized-wiring gadget $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ is $(p, \boldsymbol{\mathcal{E}}^g)$-*general random probing composable (general RPC for short) *for some $p \in [0,1]$ if there exists a PPT stateful two-stage simulator $(\mathsf{Sim}_1, \mathsf{Sim}_2)$ such that for every shared input $[\![x]\!] \in (\mathbb{K}^n)^\ell$ and for every set collection $(J_1, \ldots, J_m)$, the outputs of the following experiment:*

$$\begin{aligned}
&\rho \leftarrow \$\\
&\mathcal{W} \leftarrow \mathsf{LeakingWires}(G^\rho, p)\\
&I_1, \ldots, I_\ell \leftarrow \mathsf{Sim}_1(G^\rho, \mathcal{W}, J_1, \ldots, J_m)\\
&out_1, out_2 \leftarrow \mathsf{Sim}_2\big([\![x_1]\!]|_{I_1}, \ldots, [\![x_\ell]\!]|_{I_\ell}\big)\\
&\textit{return } (I_1, \ldots, I_\ell, out_1, out_2)
\end{aligned}$$

*are such that*

1. $(I_1, \ldots, I_\ell) \lesssim \mathcal{E}^g_{J_1, \ldots, J_m}$,

2.

$$(out_1, out_2) \overset{id}{=} \left( \underbrace{\mathcal{L}_{\mathcal{W}}\left(G^{\rho}, [\![\boldsymbol{x}]\!], [\![\boldsymbol{y}]\!]\right)}_{\texttt{Induced Random probing leakage}} \quad , \quad \underbrace{\left([\![y_1]\!]|_{J_1}, \ldots, [\![y_m]\!]|_{J_m}\right)}_{\texttt{Output leakage}} \right),$$

*where* $\mathcal{W} \leftarrow \mathsf{LeakingWires}(C, p)$ *and* $[\![\boldsymbol{y}]\!] \leftarrow G^{\rho}([\![\boldsymbol{x}]\!])$.

**Definition 6 (Uniformly Cardinal RPC - Formalization from [8]).** *Let* $n, \ell, m \in \mathbb{N}$. *Let* $\boldsymbol{\mathcal{E}}^u$ *be a collection of probability envelopes indexed by* $(t_1^o, \ldots, t_m^o) \subseteq [n]^m$ *and defined over* $[n]^{\ell}$. *An* $n$*-share gadget* $G : (\mathbb{K}^n)^{\ell} \to (\mathbb{K}^n)^m$ *is* uniformly $(p, \boldsymbol{\mathcal{E}}^u)$-cardinal random probing composable (uniformly cardinal RPC for short) *for some* $p \in [0, 1]$ *if it is* $(p, \boldsymbol{\mathcal{E}}^u)$*-cardinal RPC and if it is* $(p, \boldsymbol{\mathcal{E}}^g)$*-general RPC such that the collection of general RPC envelopes* $\boldsymbol{\mathcal{E}}^g$ *satisfies the two following conditions:*

1. *for all* $(J_1, \ldots, J_m)$, $(J'_1, \ldots, J'_m) \in \mathcal{P}([n])^m$ *such that* $(|J_1|, \ldots, |J_m|) = (|J'_1|, \ldots, |J'_m|)$, *and for all* $(I_1, \ldots, I_{\ell})$, $(I'_1, \ldots, I'_{\ell}) \in \mathcal{P}([n])^{\ell}$ *such that* $(|I_1|, \ldots, |I_{\ell}|) = (|I'_1|, \ldots, |I'_{\ell}|)$, *we have*

$$\mathcal{E}^g_{J_1, \ldots, J_m}(I_1, \ldots, I_l) = \mathcal{E}^g_{J'_1, \cdots, J'_m}(I'_1, \ldots, I'_l).$$

2. *for all* $(t_1^i, \ldots, t_{\ell}^i) \in [n]^{\ell}$, $(t_1^o, \ldots, t_m^o) \in [n]^m$, $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ *such that* $|J_1| = t_1^o, \ldots, |J_m| = t_m^o$ *and* $(I_1, \cdots, I_{\ell}) \in \mathcal{P}([n])^{\ell}$ *such that* $|I_1| = t_1^i, \ldots, |I_{\ell}| = t_{\ell}^i$, *we have*

$$\mathcal{E}^g_{J_1, \ldots, J_m}(I_1, \ldots, I_l) \prod_{\alpha=1}^{\ell} \binom{n}{t_{\alpha}^i} = \mathcal{E}^u_{t_1^o, \ldots, t_m^o}(t_1^i, \ldots, t_{\ell}^i).$$

**Definition 7 (Cardinal Random Probing Composability).** *Let* $n, \ell, m \in \mathbb{N}$. *Let* $\boldsymbol{\mathcal{E}}^c$ *be a collection of probability envelopes indexed by* $(t_1^{\texttt{out}}, \cdots t_m^{\texttt{out}}) \in [0, n]^m$ *and defined over* $[0, n]^{\ell}$. *An* $n$*-share randomized-wiring gadget* $G : (\mathbb{K}^n)^{\ell} \to (\mathbb{K}^n)^m$ *is* $(p, \boldsymbol{\mathcal{E}}^c)$-cardinal random probing composable ($(p, \boldsymbol{\mathcal{E}}^c)$-cardinal RPC) *for some* $p \in [0, 1]$ *if there exists a PPT stateful two-stage simulator* $(\mathsf{Sim}_1, \mathsf{Sim}_2)$ *such that for all shared input* $[\![\boldsymbol{x}]\!] \in (\mathbb{K}^n)^{\ell}$ *and for every collection* $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$, *the outputs of the Experiment 1:*

*Experiment 1: RPC*

| | |
|---|---|
| $\rho \leftarrow \$$ | *Sampling the random tape for the wires.* |
| $\mathcal{W} \leftarrow \mathsf{LeakingWires}(G^{\rho}, p)$ | *Sampling the leaking wires.* |
| $I_1, \cdots I_{\ell} \leftarrow \mathsf{Sim}_1(G^{\rho}, \mathcal{W}, J_1, \cdots J_m)$ | *From the indexes of the leaking outputs, $\mathsf{Sim}_1$ determines the indexes of the required inputs.* |
| $out_1, out_2 \leftarrow \mathsf{Sim}_2\left([\![x_1]\!]|_{I_1}, \cdots, [\![x_{\ell}]\!]|_{I_{\ell}}\right)$ | *From the selected input shares, $\mathsf{Sim}_2$ returns simulated values of the internal leaking wires $out_1$ and the output leaking wires $out_2$* |
| *return* $(I_1, \cdots I_{\ell}, out_1, out_2)$ | |

*are such that*

1. $(|I_1|, \cdots, |I_{\ell}|) \lesssim \mathcal{E}^c_{t_1^{\texttt{out}}, \cdots t_m^{\texttt{out}}}$, *where* $t_1^{\texttt{out}} = |J_1|, \ldots, t_m^{\texttt{out}} = |J_m|$, *and*
2.

$$(out_1, out_2) \overset{id}{=} \left( \underbrace{\mathsf{AssignWires}\left(G^{\rho}, R, \mathcal{W}, [\![\boldsymbol{x}]\!]\right)}_{\texttt{Induced Random probing leakage}}, \quad \underbrace{\left([\![y_1]\!]|_{J_1}, \cdots, [\![y_m]\!]|_{J_m}\right)}_{\texttt{Output leakage}} \right),$$

*where* $\mathcal{W} \leftarrow \mathsf{LeakingWires}(G^{\rho}, p)$ *and* $[\![\boldsymbol{y}]\!] \leftarrow G^{\rho}([\![\boldsymbol{x}]\!])$.

We now explicitly derive the envelope for the cardinal RPC property of a gadget from its envelope for the general RPC property in Lemma 1. This follows directly from Definitions 5 and 7, using the same simulators for both definitions.

**Lemma 1.** *Let $n, \ell, m \in \mathbb{N}$. Let $\boldsymbol{\mathcal{E}}^g$ be a collection of probability envelopes indexed by $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ and defined over $\mathcal{P}([n])^\ell$. Let $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ be an $n$-share randomized-wiring gadget achieving $(p, \boldsymbol{\mathcal{E}}^g)$-general RPC for some $p \in [0, 1]$. Then, $G$ is also $(p, \boldsymbol{\mathcal{E}}^c)$-cardinal RPC with $\boldsymbol{\mathcal{E}}^c$ defined for all $(t_1^{\boldsymbol{out}}, \cdots t_m^{\boldsymbol{out}}) \in [0, n]^m$ and for all $(t_1^{\boldsymbol{in}}, \cdots t_\ell^{\boldsymbol{in}}) \in [0, n]^\ell$, as follows:*

$$\mathcal{E}^c_{t_1^{\boldsymbol{out}}, \cdots t_m^{\boldsymbol{out}}}(t_1^{\boldsymbol{in}}, \cdots, t_\ell^{\boldsymbol{in}}) = \max_{\substack{J_1, \ldots, J_m \\ |J_1| = t_1^{\boldsymbol{out}}, \ldots, |J_m| = t_m^{\boldsymbol{out}}}} \sum_{\substack{I_1, \ldots, I_\ell \\ |I_1| = t_1^{\boldsymbol{in}}, \ldots, |I_\ell| = t_\ell^{\boldsymbol{in}}}} \mathcal{E}^g_{J_1, \cdots, J_m}(I_1, \cdots, I_\ell)$$

## 4 Uniformly Cardinal RPC Property from Permutations

In this section, we revisit and refine the notion of uniformly cardinal RPC introduced in [8], and demonstrate how to enforce it effectively. Our goal is to improve the trade-offs offered by earlier simulation-based composition notions (namely threshold, cardinal, and general RPC) particularly with respect to the balance between tight security bounds and scalability.

As discussed previously, general RPC offers the tightest bounds on the adversarial advantage but incurs exponential composition complexity. With threshold RPC, the verification of the composition is highly efficient, yet it yields looser bounds. Cardinal RPC strikes a middle ground, trading some tightness for improved compositional efficiency by restricting leakage set structures. Uniformly cardinal RPC, briefly introduced in [8], emerges as a better compromise: when a gadget is uniformly cardinal RPC, then the general RPC and cardinal RPC envelopes are equivalent, so one retains the tightness of general RPC while inheriting the lower verification complexity of cardinal RPC. In particular, [8, Corollary 1] shows that leveraging this property when composing two refresh gadgets inside a global copy gadget improves the overall bounds compared to using cardinal RPC alone. That said, [8] does not provide concrete gadgets satisfying the uniformly cardinal RPC property, nor design principles to obtain them, leaving construction strategies as an open problem.

In the following, we decompose the uniformly cardinal RPC notion into two jointly equivalent sub-properties: *UI cardinal RPC* and *UO cardinal RPC*, which isolate the effects of uniformity on the input and output shares, respectively. We show that UI cardinal RPC provides clear advantages in certain composition scenarios, while several existing gadgets already satisfy UO cardinal RPC for tight envelopes. Moreover, we demonstrate that applying uniform random permutations to the input (resp. output) shares of a gadget is sufficient to ensure UI (resp. UO) cardinal RPC, with a small computational overhead.

We conclude the section with a concrete example, a butterfly network, that illustrates the strengths and limitations of each composability notion across various settings. The proofs of the different lemmas introduced in this section are provided in Appendix B.

### 4.1 Uniformly Cardinal RPC Property

Informally, a gadget is said to be uniformly cardinal RPC if it is cardinal RPC, and also general RPC for an envelope collection in which, for any set of input shares of a given cardinality, the envelopes defined over all sets of output share of the same cardinality are identical. This notion is formalized in Definition 6. If a gadget is uniformly cardinal RPC, then its general RPC and cardinal RPC envelopes are equivalent. Consequently, we retain the tight security guarantees of general RPC (for the corresponding envelope collection) while retaining the efficiency of cardinal RPC for verifying its composition in a larger circuit. Indeed, the uniformity condition ensures that no information is missing beyond what is already captured by the cardinal RPC envelope.

Building on this property, we introduce two intermediate notions: UI cardinal RPC (Definition 8) and UO cardinal RPC (Definition 9). Informally, UI cardinal RPC captures the idea of uniformity with respect

to the input shares, while UO cardinal RPC captures uniformity with respect to the output shares. These two notions are jointly equivalent to uniformly cardinal RPC, as stated in Lemma 2.

This decomposition is motivated by the observation that a gadget may satisfy neither, one, or both of these sub-properties, each case having distinct implications for composability. By isolating the role of uniformity on input and output shares separately, we gain more flexibility to efficiently enforce uniformity in gadget construction or composition.

**Definition 8 (UI Cardinal RPC).** *Let $n, \ell, m \in \mathbb{N}$. Let $\boldsymbol{\mathcal{E}}^{ui}$ be a collection of probability envelopes indexed by $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \in [0,n]^m$ over $[0,n]^\ell$. An $n$-share gadget $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ is $(p, \boldsymbol{\mathcal{E}}^{ui})$-*UI cardinal random probing composable *for some $p \in [0,1]$ if it is $(p, \boldsymbol{\mathcal{E}}^{ui})$-cardinal RPC and if it is $(p, \boldsymbol{\mathcal{E}}^{g})$-general RPC such that the collection of general RPC envelopes $\boldsymbol{\mathcal{E}}^{g}$ satisfies the two following conditions:*

1. *for all $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$, $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$, $(I'_1, \ldots, I'_\ell) \in \mathcal{P}([n])^\ell$ such that $(|I_1|, \ldots, |I_\ell|) = (|I'_1|, \ldots, |I'_\ell|)$, we have*
$$\boldsymbol{\mathcal{E}}^{g}_{J_1,\ldots,J_m}(I_1, \ldots, I_l) = \boldsymbol{\mathcal{E}}^{g}_{J_1,\ldots,J_m}(I'_1, \ldots, I'_l),$$
2. *for all $(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) \in [0,n]^\ell$, $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \in [0,n]^m$, $(I_1, \cdots, I_\ell) \in \mathcal{P}([n])^\ell$ such that $|I_1| = t_1^{\boldsymbol{in}}, \ldots, |I_\ell| = t_\ell^{\boldsymbol{in}}$, we have*
$$\max_{\substack{J_1,\ldots,J_m \\ |J_1|=t_1^{\boldsymbol{out}},\ldots,|J_m|=t_m^{\boldsymbol{out}}}} \boldsymbol{\mathcal{E}}^{g}_{J_1,\ldots,J_m}(I_1, \ldots, I_l) \prod_{\alpha=1}^{\ell} \binom{n}{t_\alpha^{\boldsymbol{in}}} = \boldsymbol{\mathcal{E}}^{ui}_{t_1^{\boldsymbol{out}},\ldots,t_m^{\boldsymbol{out}}}(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}).$$

**Definition 9 (UO Cardinal RPC).** *Let $n, \ell, m \in \mathbb{N}$. Let $\boldsymbol{\mathcal{E}}^{uo}$ be a collection of probability envelopes indexed by $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \in [0,n]^m$ over $[0,n]^\ell$. An $n$-share gadget $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ is $(p, \boldsymbol{\mathcal{E}}^{uo})$-*UO cardinal random probing composable *for some $p \in [0,1]$ if it is $(p, \boldsymbol{\mathcal{E}}^{uo})$-cardinal RPC and if it is $(p, \boldsymbol{\mathcal{E}}^{g})$-general RPC such that the collection of general RPC envelopes $\boldsymbol{\mathcal{E}}^{g}$ satisfies the two following conditions:*

1. *for all $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$, $(J'_1, \ldots, J'_m) \in \mathcal{P}([n])^m$ such that $(|J_1|, \ldots, |J_m|) = (|J'_1|, \ldots, |J'_m|)$, for all $(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) \in [0,n]^\ell$, we have*
$$\sum_{\substack{I_1,\cdots,I_\ell \\ |I_1|=t_1^{\boldsymbol{in}},\ldots,|I_\ell|=t_\ell^{\boldsymbol{in}}}} \boldsymbol{\mathcal{E}}^{g}_{J_1,\ldots,J_m}(I_1, \ldots, I_l) = \sum_{\substack{I_1,\cdots,I_\ell \\ |I_1|=t_1^{\boldsymbol{in}},\ldots,|I_\ell|=t_\ell^{\boldsymbol{in}}}} \boldsymbol{\mathcal{E}}^{g}_{J'_1,\cdots,J'_m}(I_1, \ldots, I_l).$$
2. *for all $(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) \in [0,n]^\ell$, $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \in [0,n]^m$, $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ such that $|J_1| = t_1^{\boldsymbol{out}}, \ldots, |J_m| = t_m^{\boldsymbol{out}}$,*
$$\sum_{\substack{I_1,\cdots,I_\ell \\ |I_1|=t_1^{\boldsymbol{in}},\ldots,|I_\ell|=t_\ell^{\boldsymbol{in}}}} \boldsymbol{\mathcal{E}}^{g}_{J_1,\ldots,J_m}(I_1, \ldots, I_l) = \boldsymbol{\mathcal{E}}^{uo}_{t_1^{\boldsymbol{out}},\ldots,t_m^{\boldsymbol{out}}}(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}).$$

**Lemma 2.** *Let $n, \ell, m \in \mathbb{N}$. Let $\boldsymbol{\mathcal{E}}^{u}, \boldsymbol{\mathcal{E}}^{ui}, \boldsymbol{\mathcal{E}}^{uo}$ be collections of probability envelopes indexed by $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \in [0,n]^m$ over $[0,n]^\ell$. An $n$-share gadget $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ is uniformly-$(p, \boldsymbol{\mathcal{E}}^{u})$-cardinal RPC for some $p \in [0,1]$ if and only if $G$ is $(p, \boldsymbol{\mathcal{E}}^{ui})$-UI cardinal RPC and $(p, \boldsymbol{\mathcal{E}}^{uo})$-UO cardinal RPC with $\boldsymbol{\mathcal{E}}^{u} = \boldsymbol{\mathcal{E}}^{ui} = \boldsymbol{\mathcal{E}}^{uo}$.*

### 4.2 Improved Properties of UI/UO Cardinal RPC Gadgets

As discussed earlier, for a $(p, \boldsymbol{\mathcal{E}}^{G})$-general RPC gadget $G$ with a tight envelope collection, if $G$ also satisfies the uniform cardinal RPC property with equality constraints satisfied on the envelope collection $\boldsymbol{\mathcal{E}}^{G}$, then the corresponding $\boldsymbol{\mathcal{E}}^{u}$ uniformly cardinal RPC envelope collection carries equivalent information in terms of security, while making it possible to compute the advantage of the composition much more efficiently.

More specifically, we show that the UI cardinal RPC property alone is already sufficient to achieve tighter security bounds in certain composition settings. In [8, Corollary 1], it was demonstrated that composing two uniformly cardinal RPC refresh gadgets into a copy gadget improves the resulting security bound. Here, we extend this result in three important directions:

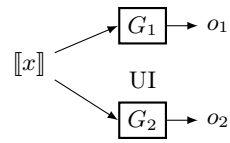



Fig. 1: Tree-UI gadget built from a cardinal RPC gadget $G_1$ and a UI cardinal RPC gadget $G_2$.

(i) the result holds for arbitrary gadgets, not just refresh gadgets;
(ii) the UI cardinal RPC property alone is sufficient (UO cardinal RPC is not required);
(iii) it suffices for only one of the two gadgets in the copy composition to satisfy UI cardinal RPC.

Concretely, we consider a gadget constructed as the parallel composition of two gadgets, $G_1$ and $G_2$, which share a common input sharing $[\![x]\!]$ (as depicted in Figure 1). We refer to this type of construction as a tree-UI gadget. To simulate the overall leakage, shares of $[\![x]\!]$ are needed both for $G_1$ and $G_2$. The key observation is that if at least one of these gadgets, say $G_2$, satisfies the UI cardinal RPC property, then the number of shares required to simulate the combined leakage can be tightly bounded, as formalized in Lemma 3. In contrast, if neither gadget satisfies UI cardinal RPC, the estimation becomes looser: in the worst case, the number of required shares must be treated as the sum of those needed for each individual gadget, potentially up to the full number of shares $n$.

**Lemma 3 (Cardinal RPC of Tree-UI).** *Let $[\![\mathbf{x}]\!]$ a $n$-sharing of a secret value $\mathbf{x} \in \mathbb{K}$. Let $G_1 : \mathbb{K}^n \to \mathbb{K}^n$ and $G_2 : \mathbb{K}^n \to \mathbb{K}^n$ two $n$-share gadgets. Let $p \in [0,1]$. If $G_1$ is $(p, \boldsymbol{\mathcal{E}}^1)$-cardinal RPC and $G_2$ is $(p, \boldsymbol{\mathcal{E}}^{ui2})$-UI cardinal RPC, then the Tree-UI circuit described in Figure 1 is $(p, \boldsymbol{\mathcal{E}}^{TUI})$-cardinal RPC such that for all $t^{in} \in [0,n]$ and all $(t_1^{out}, t_2^{out}) \in [0,n]^2$ we have*

$$\boldsymbol{\mathcal{E}}^{TUI}_{t_1^{out},t_2^{out}}(t^{in}) = \sum_{i_1=0}^{t^{in}} \sum_{i_2=t^{in}-i_1}^{t^{in}} \frac{\binom{i_1}{i_1+i_2-t^{in}} \cdot \binom{n-i_1}{t^{in}-i_1}}{\binom{n}{i_2}} \cdot \mathcal{E}^1_{t_1^{out}}(i_1) \cdot \mathcal{E}^{ui2}_{t_2^{out}}(i_2)$$

Following the approach of [8], we consider copy gates whenever a sharing is duplicated to serve as input to two distinct gadgets. In this setting, the envelope of Tree-UI, corresponding to Lemma 3 and denoted by $\mathcal{E}^{\mathtt{TUI}}$, takes the form

$$\boldsymbol{\mathcal{E}}_{t_1^{\mathtt{out}},t_2^{\mathtt{out}}}(t^{\mathtt{in}}) = \sum_{i=0}^{t^{\mathtt{in}}} \mathcal{E}^{\mathtt{TUI}}_{t_1^{\mathtt{out}},t_2^{\mathtt{out}}}(i) \cdot \binom{n-i}{t^{\mathtt{in}}-i} \cdot p^{t^{\mathtt{in}}-i} \cdot (1-p)^{n-t^{\mathtt{in}}}.$$

If copy gates are not explicitly taken into account, i.e. if duplications are modeled only by gadgets with fan-out 2, then we can rely directly on Lemma 3, or interpret the above expression, as well as our subsequent empirical results, as upper bounds.

### 4.3 Transforming Gadgets to UI/UO or Uniformly Cardinal RPC

Any $(p, \boldsymbol{\mathcal{E}}^G)$-general RPC gadget $G$ for a given envelope collection $\boldsymbol{\mathcal{E}}^G$ is trivially a $(p, \boldsymbol{\mathcal{E}}^u)$-uniformly RPC gadget when considering the following:

– A less tight general RPC envelope collection $\boldsymbol{\mathcal{E}}^{G'}$, where for all $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ and $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$, the envelopes satisfy

$$\mathcal{E}^{G'}_{J_1,\ldots,J_m}(I_1, \ldots, I_\ell) = \max_{\substack{(J'_1,\ldots,J'_m)\in\mathcal{P}([n])^m \\ |J_1|=|J'_1|,\ldots,|J_m|=|J'_m| \\ (I'_1,\ldots,I'_\ell)\in\mathcal{P}([n])^\ell \\ |I_1|=|I'_1|,\ldots,|I_\ell|=|I'_\ell|}} \mathcal{E}^G_{J'_1,\cdots,J'_m}(I'_1, \ldots, I'_\ell)$$

which relaxes the envelope structure while maintaining the essential properties for uniformly cardinal RPC.

– A uniform cardinal RPC envelope collection $\boldsymbol{\mathcal{E}}^u$ for all $(t_1^{\mathtt{in}}, \ldots, t_\ell^{\mathtt{in}}) \in [0,n]^\ell$, for all $(t_1^{\mathtt{out}}, \ldots, t_m^{\mathtt{out}}) \in [0,n]^m$, and $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ such that $|J_1| = t_1^{\mathtt{out}}, \ldots, |J_m| = t_m^{\mathtt{out}}$, and $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ with $|I_1| = t_1^{\mathtt{in}}, \ldots, |I_\ell| = t_\ell^{\mathtt{in}}$, defined as

$$\mathcal{E}^u_{t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}}(t_1^{\mathtt{in}}, \ldots, t_\ell^{\mathtt{in}}) = \mathcal{E}^{G'}{}_{J_1,\ldots,J_m}(I_1, \ldots, I_\ell) \prod_{\alpha=1}^{\ell} \binom{n}{t_\alpha^{\mathtt{in}}}.$$

This ensures that the transformed envelope $\boldsymbol{\mathcal{E}}^u$ retains all the information from $\boldsymbol{\mathcal{E}}^{\boldsymbol{G'}}$.

Similar results can be derived for the UI and UO cardinal RPC notions as well. In this section, we demonstrate that existing gadgets from the literature, specifically the refresh, addition, and scalar multiplication gadgets from [8], are directly UO cardinal RPC, derived from the tight general RPC envelopes we exhibit. We also show how any general RPC gadget can be transformed into a tighter (or, at worst, equivalent) UI or UO cardinal RPC gadget by composing it with uniformly random permutations of the input and/or output shares.

**Direct UO Cardinal RPC of Existing Gadgets.** In [8], the authors demonstrate that their refresh gadget $\mathsf{RPRefresh}^\gamma$ satisfies the cardinal RPC property, using an intermediate security notion named *Random Probing Partitioning* (RPP), that we recall in Appendix A. This gadget is built using a zero encoding gadget, $\mathsf{RPZeroEnc}^\gamma$, as described in Algorithm 4 in Appendix C.1. Informally, at each of the $\gamma$ iterations, two distinct indices $i_1, i_2 \in [n]$ are sampled uniformly and independently, and a fresh random value from $\mathbb{K}$ is added to the shares at these positions. In fact, the security of the gadget improves as $\gamma$ increases, until some threshold.

In Lemma 4, we present what we believe to be a tight and general RPC envelope collection for the refresh gadget. Our construction builds upon the RPP envelope of the intermediate gadget $\mathsf{RPZeroEnc}_+^{n,\gamma}$, as introduced in [8]. This gadget corresponds to the zero-encoding phase of the refresh gadget, augmented with additional leakage on the output shares. The RPP envelope characterizes the distribution of output shares of $\mathsf{RPZeroEnc}_+^{n,\gamma}$ by defining an upper bound to the probability of each possible partition into independent, uniformly random 0-sharings. These bounds serve as a key ingredient to define a tight general RPC envelope collection for $\mathsf{RPRefresh}$. We then establish in Corollary 1 that, based on this general RPC envelopes collection, the gadget also satisfies the UO cardinal RPC property.

**Lemma 4 (General RPC of RPRefresh).** *Let $n, \gamma \in \mathbb{N}$ and $p \in [0,1]$. If $\mathsf{RPZeroEnc}_+^{n,\gamma}$ is $(p, \boldsymbol{\mathcal{E}}^\gamma)$-RPP, then $\mathsf{RPRefresh}$ is $(p, \boldsymbol{\mathcal{E}}^g)$-general RPC such that for all $I \subseteq [n], J \subseteq [n]$, we have*

$$\mathcal{E}^g_J(I) = \sum_{K \subseteq I} p^{|I|-|K|} \cdot (1-p)^{n-|I|} \cdot \left( \sum_{P \in \mathbb{P}(n)} \sum_{(k_1,\ldots,k_\ell) \in \mathcal{A}^P_{K,J}} \mathcal{E}^\gamma(P) \cdot \frac{\prod_{i=1}^{\ell} \binom{c_i}{k_i}}{\binom{|J|}{|K|} \cdot \binom{n}{|J|}} \right)$$

*where $P := (c_1, \ldots, c_\ell)$ is an integer partition of $n$, $\mathbb{P}(n)$ denotes the set of all integer partitions of $n$, and*

$$\mathcal{A}^P_{K,J} := \{(k_1, \ldots, k_\ell) \in \mathbb{N}^\ell | 0 \le k_i \le c_i \wedge (\sum_{i=1}^{\ell} k_i = |J|) \wedge (\sum_{i=1}^{\ell} k_i \mathbf{1}_{(k_i = c_i)} = |K|)\} \ .$$

The UO cardinal RPC property follows from the expression of the general RPC envelope collection of $\mathsf{RPRefresh}$, as stated in Corollary 1.

**Corollary 1 (UO cardinal RPC of RPRefresh from [8]).** *The $n$-share gadget $\mathsf{RPRefresh}^\gamma$ from [8] is $(p, (\mathcal{E}^{uo}_J)_{J \subseteq [n]})$-UO cardinal RPC. The envelopes $\mathcal{E}^{uo}_J$ are derived from the general RPC envelopes $\mathcal{E}^g_J$ as in Definition 9.*

The linear gadgets from [8] are built from the $\mathsf{RPRefresh}^\gamma$ refresh gadget. The addition gadget, $\mathsf{Add}^\gamma$, refreshes both input $n$-sharings before performing the addition, share by share, as outlined in Algorithm 6 in Appendix C.1. Similarly, the scalar multiplication gadget, $\mathsf{cMult}^\gamma$, refreshes its input $n$-sharing and then

performs the scalar multiplication, share by share, as described in Algorithm 7 in Appendix C.1. Building on the results for the refresh gadget, we compute what we expect to be tight general RPC envelope collections for these gadgets in Lemmas 5 and 6 and show that, based on these envelope collections, they also satisfy the UO cardinal RPC property (in Corollaries 2 and 3 whose proofs are given in Appendix C.1).

**Lemma 5.** *Let $n \in \mathbb{N}$. Let $p \in [0,1]$. Let $\boldsymbol{\mathcal{E}}^g$ represent a collection of probability envelopes indexed by $J \subseteq [n]$. Then $\mathsf{Add}^\gamma$ is $(p, \boldsymbol{\mathcal{E}}^g)$-general RPC. For all sets $(I_1, I_2) \subseteq [n]$ and set $J \subseteq [n]$, we have:*

$$\boldsymbol{\mathcal{E}}^g_J(I_1, I_2) = \sum_{I'_1 \subseteq [n]} \sum_{I'_2 \subseteq [n]} \boldsymbol{\mathcal{E}}^{g_1}_{I'_1}(I_1) \cdot \boldsymbol{\mathcal{E}}^{g_1}_{I'_2}(I_2) \cdot \boldsymbol{\mathcal{E}}_J(I'_1, I'_2)$$

*where $\boldsymbol{\mathcal{E}}^{g_1}$ is the general RPC envelope of* $\mathsf{RPRefresh}$ *and for all $(I'_1, I'_2) \subseteq [n]^2$, we have*

$$\boldsymbol{\mathcal{E}}_J(I'_1, I'_2) = \begin{cases} 0 & \text{if } J \not\subset I'_1 \text{ or } J \not\subset I'_2 \\ p^{|I'_1|-|J|} \cdot (1-p)^{n-|I'_1|} \cdot p^{|I'_2|-|J|} \cdot (1-p)^{n-|I'_2|} & \text{otherwise.} \end{cases}$$

**Corollary 2.** $\mathsf{Add}^\gamma$ *is* $(p, \boldsymbol{\mathcal{E}}^{uo})$-UO cardinal RPC. *Where $\boldsymbol{\mathcal{E}}^{uo}$ is defined as in Definition 9, derived from the general RPC envelopes $\boldsymbol{\mathcal{E}}^g$ from Lemma 5.*

**Lemma 6.** *Let $n \in \mathbb{N}$. Let $p \in [0,1]$. Let $\boldsymbol{\mathcal{E}}^g$ represent a collection of probability envelopes indexed by $J \subseteq [n]$. Then $\mathsf{cMult}^\gamma$ is $(p, \boldsymbol{\mathcal{E}}^g)$-general RPC. For all set $I \subseteq [n]$ and set $J \subseteq [n]$, we have:*

$$\boldsymbol{\mathcal{E}}^g_J(I) = \sum_{I_1 \subseteq [n]} \boldsymbol{\mathcal{E}}^{g_1}_{I_1}(I) \cdot \boldsymbol{\mathcal{E}}_J(I_1)$$

*where $\boldsymbol{\mathcal{E}}^{g_1}$ is the general RPC envelope of* $\mathsf{RPRefresh}$ *and*

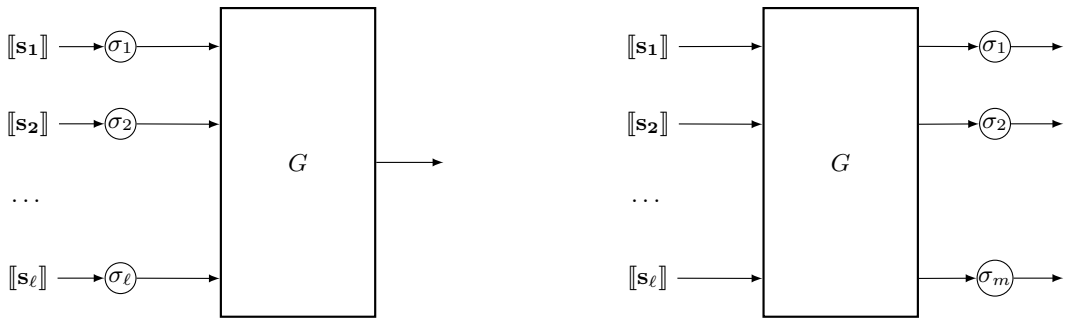
$$\boldsymbol{\mathcal{E}}_J(I_1) = \begin{cases} 0 & \text{if } J \not\subset I_1 \\ p^{|I_1|-|J|} \cdot (1-p)^{n-|I_1|} & \text{otherwise} \end{cases}$$

**Corollary 3.** $\mathsf{cMult}^\gamma$ *is $(p, \boldsymbol{\mathcal{E}}^{uo})$-UO cardinal RPC. Where $\boldsymbol{\mathcal{E}}^{uo}$ is defined as in Definition 9, derived from the general RPC envelopes $\boldsymbol{\mathcal{E}}^g$ from Lemma 6.*

**Tighter UI Cardinal RPC Gadgets using Permutations.** When a gadget is not uniformly cardinal RPC for a given general RPC envelope, we show that composing it with uniformly random permutations on its input and/or output share makes it uniformly cardinal RPC and yields tighter security bounds when composing it. Specifically, in Lemma 7, we show that applying an independent uniformly random permutation to the shares of each input sharing is sufficient to ensure the UI cardinal RPC property. Similarly, Lemma 8 establishes that independently permuting the shares of each output sharing guarantees the UO cardinal RPC property. As a direct corollary, applying independent uniformly random permutations to both the input and output shares of a gadget is sufficient to ensure the uniform cardinal RPC property.

**Lemma 7.** *Let $n, \ell, m \in \mathbb{N}$ and $p \in [0,1]$. Let $\boldsymbol{\mathcal{E}}^G$ represent a collection of probability envelopes indexed by $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$. Let $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ be an $n$-share $(p, \boldsymbol{\mathcal{E}}^G)$-general RPC gadget and $G \circ \sigma : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ denote the composition of $G$ with $\sigma$, the concatenation of $\ell$ independent random permutations $(\sigma_1, \ldots, \sigma_\ell) \in \mathcal{S}_n^\ell$ on the $\ell$ input $n$-sharings, as depicted in Figure 2a. Then $G \circ \sigma$ is $(p, \boldsymbol{\mathcal{E}}^{G\circ\sigma})$-general RPC with, for all $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ and $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$,*

$$\boldsymbol{\mathcal{E}}^{G\circ\sigma}_{J_1,\ldots,J_m}(I_1, \ldots, I_\ell) = \frac{1}{\prod_{i=1}^{\ell} \binom{n}{|I_i|}} \cdot \sum_{\substack{I^*_1,\ldots,I^*_\ell \subseteq [n] \\ |I^*_1|=|I_1|,\ldots,|I^*_\ell|=|I_\ell|}} \boldsymbol{\mathcal{E}}^G_{J_1,\ldots,J_m}(I^*_1, \ldots, I^*_\ell) \tag{1}$$




(a) Construction of Lemma 7.

(b) Construction of Lemma 8.

Fig. 2: Composing gadgets with random permutations.

*In particular, $G \circ \sigma$ is $(p, \boldsymbol{\mathcal{E}}^{ui})$-UI cardinal RPC with the envelopes $\boldsymbol{\mathcal{E}}^{ui}$ defined as follows. For any $(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) \subseteq [0,n]^\ell$ and $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \subseteq [0,n]^m$,*

$$\boldsymbol{\mathcal{E}}^{ui}_{t_1^{\boldsymbol{out}},\ldots,t_m^{\boldsymbol{out}}}(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) = \max_{\substack{J_1,\ldots,J_m \\ |J_1|=t_1^{\boldsymbol{out}},\ldots,|J_m|=t_m^{\boldsymbol{out}}}} \boldsymbol{\mathcal{E}}^{G\circ\sigma}_{J_1,\ldots,J_m}(I_1, \ldots, I_l) \prod_{\alpha=1}^{\ell} \binom{n}{t_\alpha^{\boldsymbol{in}}}$$

*for any $I_1, \cdots, I_\ell$ where $|I_1| = t_1^{\boldsymbol{in}}, \ldots, |I_\ell| = t_\ell^{\boldsymbol{in}}$.*

**Lemma 8.** *Let $n, \ell, m \in \mathbb{N}$ and $p \in [0,1]$. Let $\boldsymbol{\mathcal{E}}^G$ represent a collection of probability envelopes indexed by $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$. Let $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ be an $n$-share $(p, \boldsymbol{\mathcal{E}}^G)$-*general RPC *gadget and $\sigma \circ G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ denote the composition of $G$ with $\sigma$, the concatenation of $m$ independent random permutations $(\sigma_1, \ldots, \sigma_m) \in \mathcal{S}_n^m$ on the $m$ output $n$-sharings, as depicted in Figure 2b. Then $\sigma \circ G$ is $(p, \boldsymbol{\mathcal{E}}^{\sigma\circ G})$-general RPC with, for all $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ and $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$,*

$$\boldsymbol{\mathcal{E}}^{\sigma\circ G}_{J_1,\ldots,J_m}(I_1, \ldots, I_\ell) = \frac{1}{\prod_{i=1}^m \binom{n}{|J_i|}} \cdot \sum_{\substack{J_1^*,\ldots,J_m^* \subseteq [n] \\ |J_1^*|=|J_1|,\ldots,|J_m^*|=|J_m|}} \boldsymbol{\mathcal{E}}^{G}_{J_1^*,\ldots,J_m^*}(I_1, \ldots, I_\ell) \tag{2}$$

*In particular, $\sigma \circ G$ is $(p, \boldsymbol{\mathcal{E}}^{uo})$-*UO cardinal RPC *with the envelopes $\boldsymbol{\mathcal{E}}^{uo}$ defined as follows. For any $(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) \in [0,n]^\ell$ and $(t_1^{\boldsymbol{out}}, \ldots, t_m^{\boldsymbol{out}}) \in [0,n]^m$,*

$$\boldsymbol{\mathcal{E}}^{uo}_{t_1^{\boldsymbol{out}},\ldots,t_m^{\boldsymbol{out}}}(t_1^{\boldsymbol{in}}, \ldots, t_\ell^{\boldsymbol{in}}) = \sum_{\substack{I_1,\cdots,I_\ell \\ |I_1|=t_1^{\boldsymbol{in}},\ldots,|I_\ell|=t_\ell^{\boldsymbol{in}}}} \boldsymbol{\mathcal{E}}^{\sigma\circ G}_{J_1,\ldots,J_m}(I_1, \ldots, I_l).$$

To apply an independent uniformly random permutation to the shares, we use the Knuth-Yates shuffling technique, described in Appendix D. Informally, this process consists of iteratively swapping a randomly chosen *unprocessed* share with the last *unprocessed* share in the list. The chosen share is then marked as processed, reducing the pool of candidates for the next iteration. This continues until all shares have been processed, ensuring a uniform random permutation of the entire set. The whole procedure requires generating $n-1$ random values, each drawn uniformly from a decreasing range $[i]$, for $2 \leq i \leq n$. Additionally, it may involve up to $3(n-1)$ read/write operations.

### 4.4 Comparison of Random Probing Composability Notions

In this subsection, we compare the simulation-based composability notions recalled or introduced so far, emphasizing their respective strengths and limitations in terms of security tightness and verification efficiency.

- *General RPC* offers the tightest upper bound on the adversary's advantage. However, this comes at a steep computational cost. The associated probability envelopes are defined over $\mathcal{P}([n])^\ell \times \mathcal{P}([n])^m$, with size $2^{n(\ell+m)}$, which quickly becomes infeasible to handle for large numbers of shares, inputs, or outputs.
- *Cardinal RPC* provides a looser upper bound than *general RPC*, but significantly improves verification time and memory complexity. Envelopes are defined over $[0,n]^\ell \times [0,n]^m$, yielding a size of $(n+1)^{\ell+m}$, polynomial in the number of shares. In practice, this means that while the bound is looser, larger values of $n$ become tractable for larger circuits.
- *Uniformly cardinal RPC* built with uniformly random permutations is designed to strike a balance between the two extremes. It preserves the tightness of general RPC while benefiting from the lower complexity of cardinal RPC for the verification of compositions.
- *Threshold RPC* provides the loosest upper bound among the four notions but has the advantage of minimal verification overhead. It is particularly suitable in large circuits with high number of shares.

**One-Stage Butterfly Network.** To illustrate the practical implications of these trade-offs, we consider a one-stage butterfly network, a structure common in lattice-based cryptographic algorithms. As shown in Figure 3, this circuit takes 2 masked inputs and produces 2 masked outputs, each with $n$ shares. It is built from three basic gadgets: one scalar multiplication $\mathsf{cMult}^\gamma$, one addition $\mathsf{Add}^\gamma$ and one subtraction (identical to the addition gadget with minus signs).

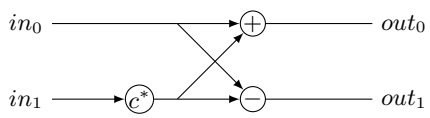



Fig. 3: 1-stage butterfly network.

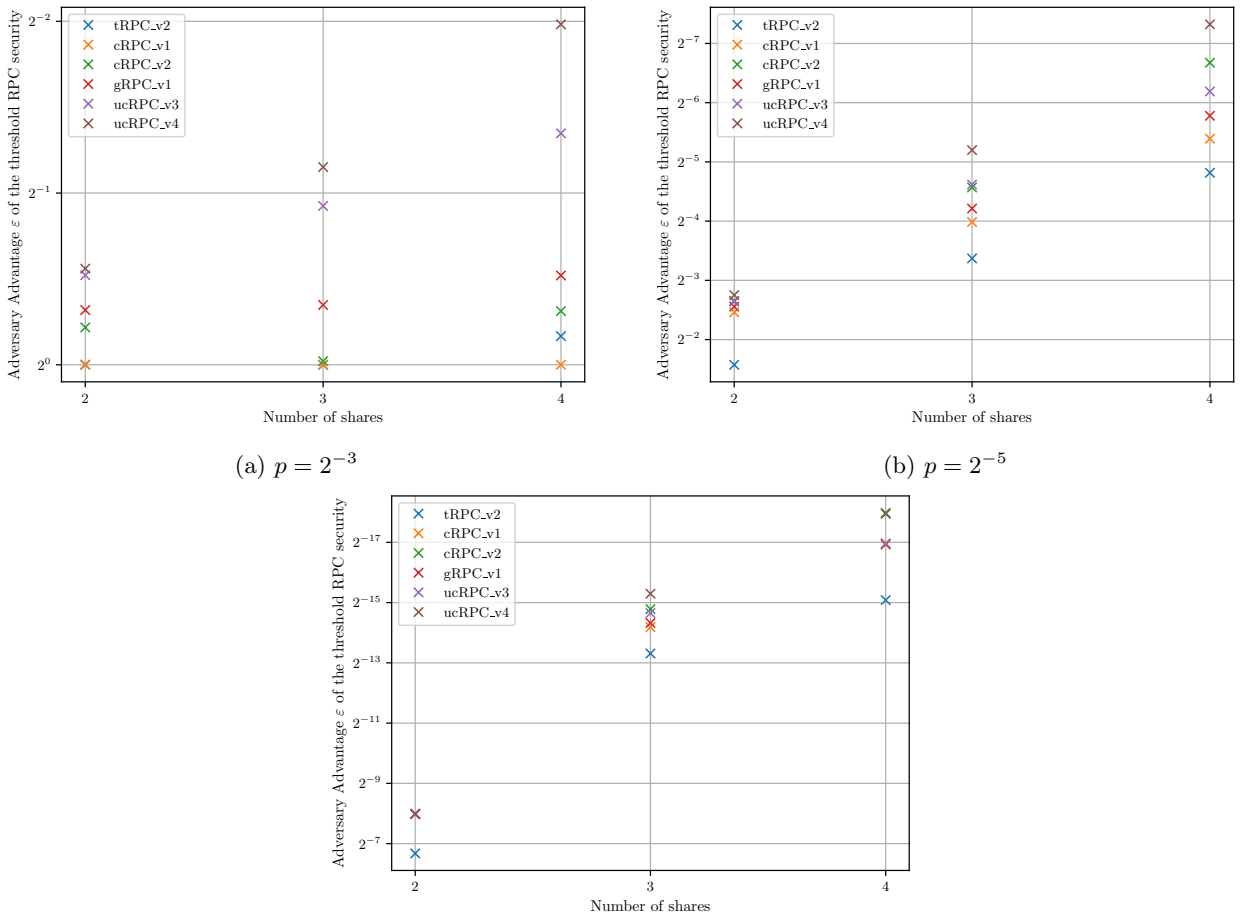
We evaluate the $(t,p)$-threshold RPC security (see Definition 4) of this butterfly network circuit, where $t = \lfloor \frac{n}{2} \rfloor$ and using four different security analyses: (i) the general RPC envelopes; (ii) the UI cardinal RPC envelopes, assuming composition with uniformly random permutations; (iii) the cardinal RPC envelopes; and (iv) the *threshold RPC* bounds derived from the individual gadgets[3]. This evaluation is performed for several leakage rates, namely $p \in \{2^{-3}, 2^{-5}, 2^{-10}\}$ and different number of shares.

Initially, we observe that the circuit provides no security guarantees under the threshold RPC notion. This is because threshold RPC requires dedicated copy gadgets to securely duplicate sharings. Without them, duplicating a sharing would amount to summing two independent sets of $t$ input shares, which exceeds the threshold and breaks the security guarantee. To enable a fair comparison, we therefore consider a modified version of the circuit (v2) in which a copy gadget (simply consisting in refreshing each branch) is inserted wherever a sharing is duplicated to feed multiple gadgets.

Figure 4 compares the threshold RPC advantages of the 1-stage butterfly network, computed under the different composition methods and circuit variants. In each subfigure, **gRPC**, **ucRPC**, **cRPC** correspond to the use of general, uniformly cardinal, and cardinal envelopes, respectively, to evaluate the global threshold RPC security. **tRPC** refers to the threshold RPC advantage, directly computed from the threshold RPC advantages of the individual gadgets. Labels **v1** to **v4** refer to: the original circuit (v1), the version with refresh gadgets at every wire split (v2), the original circuit with permutations at each copy (v3), and the version combining permutations and additional refreshes (v4).

Across all leakage rates $p$, we confirm that the threshold RPC composition yields weaker advantage even when additional refreshes are inserted (see, e.g. `tRPC_v2` vs. `cRPC_v1`). Then, the relative behavior of cardinal RPC, uniformly cardinal RPC and general RPC varies depending on the leakage rate:

[3] For the general RPC, the UI cardinal RPC and the cardinal RPC composition scenarios, we derive the threshold RPC advantage from the final circuit envelope to provide a consistent comparison.




(a) $p = 2^{-3}$

(b) $p = 2^{-5}$

(c) $p = 2^{-10}$

Fig. 4: Threshold RPC security of 1-stage butterfly network with respect to the number of shares $n$ and the leakage rates $p$ for different composition approaches.

- For low leakage rates (e.g., $p = 2^{-10}$), the security levels obtained under cardinal, general and uniformly cardinal RPC are very close, and the difference between the two circuit variants (with or without refresh) grows with the number of shares.
- At a moderate leakage rate ($p = 2^{-5}$), a small but noticeable gap begins to emerge with cardinal RPC, and the difference between the two circuit variants (with or without refresh) is more noticeable.
- When the leakage rate becomes high ($p = 2^{-3}$), the difference becomes significant with cardinal RPC and emerges between general and uniform cardinal RPC (the latter being computed on the variant with permutations). Not only does the uniformly cardinal RPC approach (with permutations) provide substantially better security guarantees than general and cardinal RPC, but the impact of using refresh gadgets (variants v2 and v4) also becomes evident, leading to improved security levels compared to the original circuit (variants v1 and v3).

These results highlight that uniformly cardinal RPC (on a variant using permutations) becomes increasingly advantageous over general and cardinal RPC (without permutations) as the leakage rate increases.

In addition, the verification of compositions based on the gadgets' general RPC envelopes does not scale efficiently to larger circuits[4]. To illustrate this limitation, a two-stage butterfly network is examined in Appendix E: an example where the general RPC composition already becomes intractable, while other methods remain practical.

## 5 Multiplication Gadget with Cardinal RPC Security

To build upon the work of [8], which introduced linear cardinal RPC gadgets, we aim to design a non-linear cardinal RPC multiplication gadget. Our approach builds on the proposal by [4], which is heuristically secure against horizontal attacks and is therefore presumed to offer strong security in the random probing model, despite lacking a formal analysis within that model. We first describe its generic design and demonstrate a general property on its global structure in Lemma 9. In Sections 5.1 and 5.2, we detail how we instantiate this design to facilitate evaluation and enhance its security features. Then, in Section 5.3, we analyze the trade-offs between security and complexity in our different implementations and compare our results with existing random probing multiplication schemes from the literature.

The original multiplication gadget from [4] consists of two steps. The first is the *expansion step* $\mathsf{MatMult}$, which takes as inputs two $n$-sharings $[\![x]\!], [\![y]\!] \in (\mathbb{K}^n)^2$ and outputs a matrix $(M_{i,j})_{1\leq i,j\leq n}$ where each coefficient $M_{i,j}$ represents a cross-product $x_i \cdot y_j$ masked with random values summing to zero, ensuring that $\sum_{i,j=1}^{n} M_{i,j} = \sum_{i,j=1}^{n} x_i \cdot y_j = x \cdot y$. The second step, the *compression step*, converts the resulting $n^2$-sharing of $x \cdot y$ into an $n$-sharing.

**Algorithm 1** Two-step Multiplication Gadget

**Input:** two $n$-sharings $[\![x]\!]$ and $[\![y]\!]$
**Output:** $n$-sharing $[\![z]\!]$ such that $z = x \cdot y$.

$(M_{i,j})_{1\leq i,j\leq n} \leftarrow \mathsf{Expand}([\![x]\!], [\![y]\!])$
$[\![z]\!] \leftarrow \mathsf{Comp}((M_{i,j})_{1\leq i,j\leq n})$
**return** $[\![z]\!]$

Our multiplication gadget follows the same two-step structure. Both the expansion and compression steps are adapted variants of those in [4], while preserving their key properties. The expansion step, referred to as $\mathsf{Expand}$, produces an $n^2$-sharing of $x \cdot y$, and the compression step, referred to as $\mathsf{Comp}$, yields an $n$-sharing of $x \cdot y$. The high-level design is given in Algorithm 1.

Lemma 9 establishes the cardinal RPC security of the high-level multiplication gadget described in Algorithm 1 based on the cardinal RPC security of its two components $\mathsf{Expand}$ and $\mathsf{Comp}$. Its proof directly follows from the composition result in [8, Lemma 4].

**Lemma 9.** *Let $n \in \mathbb{N}$. Let $p \in [0, 1]$. We assume that $\mathsf{Expand}$ is $(p, \mathcal{E}^{\mathsf{Expand}})$-cardinal RPC and $\mathsf{Comp}$ is $(p, \mathcal{E}^{\mathsf{Comp}})$-cardinal RPC. Then the multiplication gadget described in Algorithm 1 is $(p, \mathcal{E}^{\mathsf{mult}})$-cardinal RPC such that for all $t_1^{in}, t_2^{in}, t^{out} \in [0, n]$, we have:*

$$\mathcal{E}^{\mathsf{mult}}_{t^{out}}(t_1^{in}, t_2^{in}) = \sum_{k=0}^{n^2} \mathcal{E}^{\mathsf{Expand}}_{k}(t_1^{in}, t_2^{in}) \cdot \mathcal{E}^{\mathsf{Comp}}_{t^{out}}(k)$$

*Remark 1.* Note that we could partition the output of $\mathsf{Expand}$ (or equivalently, the input of $\mathsf{Comp}$) into multiple sharings to obtain tighter security bounds. Specifically, instead of considering the required number of shares among $n^2$ shares, we would analyze $\alpha$ sub-cardinals among $n^2/\alpha$ shares, leading to a more precise estimation (for $\alpha$ a divisor of $n^2$). However, increasing $\alpha$ not only tightens the bounds but also significantly

[4] Note that this scalability issue only concerns the verification of the composition. In our scripts, the computation of the gadgets' envelope for the different notions among general RPC, (uniformly) cardinal RPC and threshold RPC has almost the same complexity as we derive them all from the general RPC envelope.

**Algorithm 2** MatMult from [4]

**Input:** $n_x$-sharing $[\![x]\!]$ and $n_y$-sharing $[\![y]\!]$
**Output:** $(n_x \cdot n_y)$-sharing $\mathbf{M} = (M_{i,j})_{\substack{1 \le i \le n_x \\ 1 \le j \le n_y}}$ such that $\sum_{\substack{1 \le i \le n_x \\ 1 \le j \le n_y}} M_{i,j} = x \cdot y$

**if** $(n_x, n_y) = (1,1)$ **then return**$(x_1 \cdot y_1)$
**if** $(n_x, n_y) = (1,2)$ **then return** $\begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 \end{pmatrix}$
**if** $(n_x, n_y) = (2,1)$ **then return** $\begin{pmatrix} x_1 \cdot y_1 \\ x_2 \cdot y_1 \end{pmatrix}$
**if** $(n_x, n_y) = (2,2)$ **then return** $\begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 \\ x_2 \cdot y_1 & x_2 \cdot y_2 \end{pmatrix}$
$\mathbf{x}^{(1)} \leftarrow (x_1, \ldots, x_{\lfloor \frac{n_x}{2} \rfloor})$; $\mathbf{x}^{(2)} \leftarrow (x_{\lceil \frac{n_x}{2} \rceil}, \ldots, x_{n_x})$;
$\mathbf{y}^{(1)} \leftarrow (y_1, \ldots, y_{\lfloor \frac{n_y}{2} \rfloor})$; $\mathbf{y}^{(2)} \leftarrow (y_{\lceil \frac{n_y}{2} \rceil}, \ldots, y_{n_y})$
$\mathbf{x}'^{(1)} \leftarrow \mathsf{G}_{\mathsf{ref}}(\mathbf{x}^{(1)})$; $\mathbf{x}'^{(2)} \leftarrow \mathsf{G}_{\mathsf{ref}}(\mathbf{x}^{(2)})$; $\mathbf{y}'^{(1)} \leftarrow \mathsf{G}_{\mathsf{ref}}(\mathbf{y}^{(1)})$; $\mathbf{y}'^{(2)} \leftarrow \mathsf{G}_{\mathsf{ref}}(\mathbf{y}^{(2)})$
**return** $\begin{pmatrix} \mathsf{MatMult}(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}) & \mathsf{MatMult}(\mathbf{x}'^{(1)}, \mathbf{y}^{(2)}) \\ \mathsf{MatMult}(\mathbf{x}^{(2)}, \mathbf{y}'^{(1)}) & \mathsf{MatMult}(\mathbf{x}'^{(2)}, \mathbf{y}'^{(2)}) \end{pmatrix}$

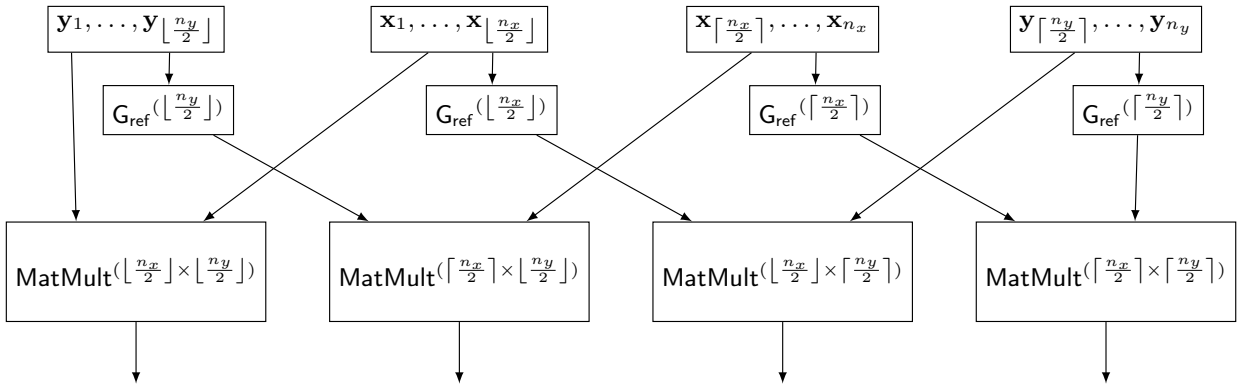



Fig. 5: Illustration of MatMult.

increases the complexity of computing the envelopes, requiring an enumeration over $(\frac{n^2}{\alpha} + 1)^\alpha$ instead of over $n^2 + 1$.

### 5.1 Refining MatMult: New Variants

In [4], the expansion step Expand is instantiated with MatMult which is designed as follows. Each $n$-sharing $[\![x]\!]$ and $[\![y]\!]$ is first split into two parts, resulting in two $\lfloor \frac{n}{2} \rfloor$-sharings and two $\lceil \frac{n}{2} \rceil$-sharings. Each of these sub-sharings is then duplicated into two versions: one remains unchanged (identity copy), while the other undergoes a refresh gadget, denoted $\mathsf{G}_{\mathsf{ref}}$, to introduce additional randomness. MatMult is then applied recursively to each of the four resulting pairs, with one instance made of a sub-sharing of $x$ and one instance made of a sub-sharing of $y$, each copy being used only once. Once $n$ reaches one or two, the algorithm directly computes the corresponding sub-products between the two $1-$ or $2-$ sharings.

We recall the full procedure in Algorithm 2, for any $n \in \mathbb{N}^*$. An illustration is also provided in Figure 5. Note that recursive calls may involve inputs of different lengths, denoted $n_x$ and $n_y$, when $n$ is not a power of two. However, starting from equal input sizes $n_x = n_y = n$, it is easy to show by induction that the invariant $|n_x - n_y| \le 1$ holds throughout the recursion. The authors of [4] instantiate $\mathsf{G}_{\mathsf{ref}}$ using the quasi-linear refresh gadget introduced in the same paper, which optimizes both computational complexity and the number of required random values.

In the following, we present three new variants of MatMult, designed to improve security bounds.

*Variant 1: MatMultRef.* We first apply transformations to MatMult to facilitate the computation of its cardinal RPC envelope. Specifically, we replace the quasi-linear refresh gadget with the one introduced in [8], as its cardinal RPC envelope is established. We refer to this modified version as MatMultRef.

Building upon this initial construction, we explore two alternative trade-offs between security and performance through the following variants:

*Variant 2: MatMultSym* refreshes each sub-sharing before every recursive call, leading to twice as many refresh gadgets as in MatMultRef, as initially suggested in [13,19]. MatMultSym is formally described in Appendix F.

*Variant 3: UnifMatMult* introduces uniformly random permutations at key locations in the circuit to improve the security bounds when duplicating shared values. Specifically, for each parallel composition of two gadgets that share the same input sharing, a random permutation is applied to the input sharing of one of them to ensure it satisfies the UI cardinal RPC property, thereby transforming the composition into a Tree-UI gadget (see Lemma 3). A formal description of UnifMatMult is provided in Appendix F. Note that we do not add random permutations at the output of intermediate gadgets, however, since these already satisfy the UO cardinal RPC property by construction.

*Complexity.* Regarding the complexity of the three variants of MatMult, as the number of random values used, denoted $C_{\text{rand}}(n_x, n_y)$, as well as the number of additions, denoted $C_{\text{add}}(n_x, n_y)$, and the number of multiplications $C_{\text{mult}}(n_x, n_y)$ can be computed by recursively. We note respectively $C^1_*$, $C^2_*$ , $C^3_*$ the complexity of, respectively, variants MatMultRef, MatMultSym and UnifMatMult. We denote by $\gamma(n)$ the number of random values used in RPRefresh when applied to an input sharing of size $n$, allowing this number to depend on $n$. We have

$$\begin{aligned}
C^1_{\text{rand}}(n_x, n_y) &= 2\gamma(n_x) + 2\gamma(n_y) \\
&\quad + C^1_{\text{rand}}(\lfloor \tfrac{n_x}{2} \rfloor, \lfloor \tfrac{n_y}{2} \rfloor) + C^1_{\text{rand}}(\lfloor \tfrac{n_x}{2} \rfloor, \lceil \tfrac{n_y}{2} \rceil) \\
&\quad + C^1_{\text{rand}}(\lceil \tfrac{n_x}{2} \rceil, \lfloor \tfrac{n_y}{2} \rfloor) + C^1_{\text{rand}}(\lceil \tfrac{n_x}{2} \rceil, \lceil \tfrac{n_y}{2} \rceil) \\
C^1_{\text{add}}(n_x, n_y) &= 4\gamma(n_x) + 4\gamma(n_y) + n_x + n_y \\
&\quad + C^1_{\text{add}}(\lfloor \tfrac{n_x}{2} \rfloor, \lfloor \tfrac{n_y}{2} \rfloor) + C^1_{\text{add}}(\lfloor \tfrac{n_x}{2} \rfloor, \lceil \tfrac{n_y}{2} \rceil) \\
&\quad + C^1_{\text{add}}(\lceil \tfrac{n_x}{2} \rceil, \lfloor \tfrac{n_y}{2} \rfloor) + C^1_{\text{add}}(\lceil \tfrac{n_x}{2} \rceil, \lceil \tfrac{n_y}{2} \rceil)
\end{aligned}$$

where we have

$$\begin{aligned}
C^1_{\text{rand}}(1,1) &= C^1_{\text{rand}}(1,2) = C^1_{\text{rand}}(2,1) = C^1_{\text{rand}}(2,2) = 0 \\
C^1_{\text{add}}(1,1) &= C^1_{\text{add}}(1,2) = C^1_{\text{add}}(2,1) = C^1_{\text{add}}(2,2) = 0
\end{aligned}$$

Furthermore, the number of multiplications used in MatMultRef is given by $C^1_{\text{mult}}(n_x, n_y) = n_x \cdot n_y$. For the two other variants, we have

$$\begin{aligned}
C^2_{\text{rand}}(n_x, n_y) &= C^3_{\text{rand}}(n_x, n_y) = 2 \cdot C^1_{\text{rand}}(n_x, n_y) \\
C^2_{\text{add}}(n_x, n_y) &= C^3_{\text{add}}(n_x, n_y) = 2 \cdot C^1_{\text{add}}(n_x, n_y) \\
C^2_{\text{mult}}(n_x, n_y) &= C^3_{\text{mult}}(n_x, n_y) = C^1_{\text{mult}}(n_x, n_y)
\end{aligned}$$

Finally, we compute the number of permutations $\mathsf{NbP}(n_x, n_y, k)$ of size $k > 1$ used in UnifMatMult when called with $n_x = n_y = n$, recursively as follows.

If $k > \max(n_x, n_y)$, then:

$$\mathsf{NbP}(n_x, n_y, k) = 0.$$

Otherwise:

$$\begin{aligned}\mathsf{NbP}(n_x, n_y, k) = \mathsf{NbP}(\left\lfloor \frac{n_x}{2} \right\rfloor, \left\lfloor \frac{n_y}{2} \right\rfloor, k) + \mathsf{NbP}(\left\lceil \frac{n_x}{2} \right\rceil, \left\lfloor \frac{n_y}{2} \right\rfloor, k) \\ + \mathsf{NbP}(\left\lfloor \frac{n_x}{2} \right\rfloor, \left\lceil \frac{n_y}{2} \right\rceil, k) + \mathsf{NbP}(\left\lceil \frac{n_x}{2} \right\rceil, \left\lceil \frac{n_y}{2} \right\rceil, k) \\ + 2 \cdot \left( \mathbf{1}_{k \in \{\lfloor \frac{n_x}{2} \rfloor, \lceil \frac{n_x}{2} \rceil\}} + \mathbf{1}_{k \in \{\lfloor \frac{n_y}{2} \rfloor, \lceil \frac{n_y}{2} \rceil\}} \right).\end{aligned}$$

Table 1 summarizes the complexity in terms of number of operations and number of random values of all the gadgets used in our three multiplication gadget variants: MatMultAsym, MatMultSym, and UnifMatMult, and in TreeAdd, with respect to the number of shares $n$, assumed to be a power of two.

As explained in Section 3.1, it is important to distinguish the two types of randomness involved in our gadgets. Specifically, the randomness required for generating random indexes and random permutations is fundamentally different from that used in random gates. In practice, random gates must be implemented in a *secure* manner to limit information leakage, as characterized by the leakage probability $p$. In contrast, the random bits used for indexes and permutations do not require any protections. Consequently, generating $\log_2(|\mathbb{K}|)$ bits for indexes or permutations is typically much less costly than a gate producing a random element of $\mathbb{K}$. Therefore, we account for these two types of randomness separately.

Regarding the count of random bits, recall that the RPRefresh gadget requires, at each iteration, the selection of two distinct indices: one from $[n]$ and another from $[n-1]$. These selections can be performed efficiently using approximately $\gamma(n) \cdot (\log_2 n + \log_2(n-1))$ random bits per iteration. Similarly, in UnifMatMult, each permutation involves $n-1$ random values, where the $i$-th value is sampled uniformly from $[i]$ for $2 \leq i \leq n$. Since sampling from an interval of size $k$ typically requires around $\log_2(k)$ random bits, the overall cost corresponds to the sum of these logarithmic terms. The total number of such unprotected random bits used in both the RPRefresh gadgets and the input permutations of UnifMatMult is reported in the last column of the table.

The implementation of permutations may also incur additional read/write (R/W) operations, which are not included in the table as they are implementation-dependent. However, as a rough estimate, each permutation of size $k$ may involve up to $3k$ R/W operations. The total number of such permutations is given by $\mathsf{NbP}(n, n, k)$.

Table 1: Complexity of gadgets as a function of $n$.

| **Gadget** | #Randoms (field) | #Additions | #Mult. | #Random (bits) |
|---|---|---|---|---|
| RPRefresh | $\gamma(n)$ | $2\gamma(n) + n$ | 0 | $\gamma(n) \cdot (\log_2(n) + \log_2(n-1))$ |
| Add | $2\gamma(n)$ | $4\gamma(n) + 3n$ | 0 | $2\gamma(n) \cdot (\log_2(n) + \log_2(n-1))$ |
| MatMultAsym | $\sum_{\ell=1}^{\log_2(n)-1} 4^\ell \gamma(\frac{n}{2^\ell})$ | $\sum_{\ell=1}^{\log_2(n)-1} 4^\ell (2\gamma(\frac{n}{2^\ell}) + \frac{n}{2^\ell})$ | $n^2$ | $\sum_{\ell=1}^{\log_2(n)-1} 4^\ell \gamma(\frac{n}{2^\ell})(\log_2(\frac{n}{2^\ell}) + \log_2(\frac{n}{2^\ell} - 1))$ |
| MatMultSym | $\sum_{\ell=1}^{\log_2(n)-1} 8^\ell \gamma(\frac{n}{2^\ell})$ | $\sum_{\ell=1}^{\log_2(n)-1} 8^\ell (2\gamma(\frac{n}{2^\ell}) + \frac{n}{2^\ell})$ | $n^2$ | $\sum_{\ell=1}^{\log_2(n)-1} 8^\ell \gamma(\frac{n}{2^\ell})(\log_2(\frac{n}{2^\ell}) + \log_2(\frac{n}{2^\ell} - 1))$ |
| UnifMatMult | $\sum_{\ell=1}^{\log_2(n)-1} 8^\ell \gamma(\frac{n}{2^\ell})$ | $\sum_{\ell=1}^{\log_2(n)-1} 8^\ell (2\gamma(\frac{n}{2^\ell}) + \frac{n}{2^\ell})$ | $n^2$ | $\sum_{\ell=1}^{\log_2(n)-1} 8^\ell \gamma(\frac{n}{2^\ell})(\log_2(\frac{n}{2^\ell}) + \log_2(\frac{n}{2^\ell} - 1)) + 4^\ell \sum_{k=2}^{n/2^\ell} \log_2(k)$ |
| TreeAdd | $2\gamma(n-1)$ | $(n-1) \cdot (4\gamma(n) + 3n)$ | 0 | 0 |

*Security.* Regarding the cardinal RPC security of the three MatMult variants, the corresponding envelopes can be computed recursively. They are provided, along with a detailed description of the computation method, in Appendix F. All scripts are open-source at

https://github.com/CryptoExperts/AC25-cardinal-rp-compiler.

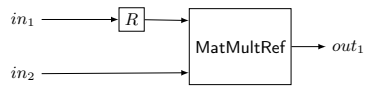



Fig. 6: A recursive call of MatMultRef.

Below, we briefly outline the approach used for MatMultRef; the computations for MatMultSym and UnifMatMult follow a similar structure.

Assuming we are not in the base case, MatMultRef gadget makes four recursive calls to itself, each involving a sub-sharing of $x$ and a sub-sharing of $y$. In each call, one of the sub-sharings is refreshed beforehand, as shown in Figure 6. Given the cardinal RPC envelopes for the refresh gadget and for the recursive call to MatMultRef, we can derive the cardinal RPC envelope of the overall structure in Figure 6 by applying [8, Lemma 4]. This provides us with four recursive cardinal RPC envelopes. To obtain the global cardinal RPC envelope, we must take the union of the required shares across the four recursive calls. However, this step is non-trivial since each sub-sharing appears in two of the recursive calls, meaning that a single share may be required by multiple calls. Consequently, we must account for the probability of all possible cases when constructing our envelopes.

### 5.2 Refining Comp: a New Variant

The original Comp from [4] does not follow a recursive structure, making it difficult to efficiently compute the envelopes of the compression step. To address this issue, we propose a new functionally equivalent Comp gadget, denoted TreeAdd, for which we are able to compute the envelopes efficiently. The choice of a tree of additions then naturally arises for its efficiency and its recursive structure.

Algorithm 3 describes TreeAdd, which implements a tree structure of $(n, 2\text{-to-}1)$ addition gadgets $\mathsf{Add}^\gamma$ (as defined in Algorithm 6) to compress the $n$ $n$-sharings composing $M$ into a single $n$-sharing.

**Algorithm 3** TreeAdd

**Input :** $(M_{i,j})_{\substack{1\le i\le k\\ 1\le j\le n}}$ $k$ $n$-sharings.

**Output :** An $n$-sharing of $\sum_{i=1}^{k}\sum_{j=1}^{n} M_{i,j}$.

```
if k = 1 then return (M_{1,j})_{1≤j≤n}
if k = 2 then return Add^γ((M_{1,j})_{1≤j≤n}, (M_{2,j})_{1≤j≤n})
[[z_1]] ← TreeAdd((M_{i,j})_{1≤i≤⌊k/2⌋, 1≤j≤n}); [[z_2]] ← TreeAdd((M_{i,j})_{⌈k/2⌉≤i≤n, 1≤j≤n})
return Add^γ([[z_1]], [[z_2]])
```

The authors of [8] established the cardinal RPC security of $\mathsf{Add}^\gamma$ based on the cardinal RPC security of the underlying refresh gadget $\mathsf{RPRefresh}^\gamma$. In particular, they demonstrated how to compute $\boldsymbol{\mathcal{E}}^{\mathrm{add}}$ from the envelope of the refresh gadget. Building on these results, the envelopes of the cardinal RPC security $\boldsymbol{\mathcal{E}}^{\mathsf{TA},k}$ of TreeAdd can be computed inductively from $\boldsymbol{\mathcal{E}}^{\mathsf{TA},\lfloor\frac{k}{2}\rfloor}$, $\boldsymbol{\mathcal{E}}^{\mathsf{TA},\lceil\frac{k}{2}\rceil}$ and $\boldsymbol{\mathcal{E}}^{\mathrm{add}}$. This approach is formalized in Lemma 10, whose proof is directly derived from the composition lemma in [8, Lemma 4].

**Lemma 10.** *Let $k, n \in \mathbb{N}^*$, $k \le n$. Let $p \in [0,1]$ the leakage rate. Assume TreeAdd with $\lfloor\frac{k}{2}\rfloor \cdot n$ (resp. $\lceil\frac{k}{2}\rceil \cdot n$) input shares is $(p, \boldsymbol{\mathcal{E}}^{\mathsf{TA},\lfloor\frac{k}{2}\rfloor})$-cardinal RPC (resp. $(p, \boldsymbol{\mathcal{E}}^{\mathsf{TA},\lceil\frac{k}{2}\rceil})$-cardinal RPC) and $\mathsf{Add}^\gamma$ is $(p, \boldsymbol{\mathcal{E}}^{add})$-cardinal RPC. Then TreeAdd with $k \cdot n$ input shares is $(p, \boldsymbol{\mathcal{E}}^{\mathsf{TA},k})$-cardinal RPC where, for all $t^{\boldsymbol{in}} \in [0, k \cdot n]$, all*

$t^{out} \in [0, n]$, *and all* $k > 2$:

$$\boldsymbol{\mathcal{E}}^{\mathsf{TA},k}_{t^{out}}(t^{in}) = \sum_{i_1=\max(0,t^{in}-\lceil \frac{k}{2} \rceil \cdot n)}^{\min(t^{in},\lfloor \frac{k}{2} \rfloor \cdot n)} \sum_{\ell_1=0}^{n} \sum_{\ell_2=0}^{n} \boldsymbol{\mathcal{E}}^{\mathsf{TA},\lfloor \frac{k}{2} \rfloor}_{\ell_1}(i_1) \cdot \boldsymbol{\mathcal{E}}^{\mathsf{TA},\lceil \frac{k}{2} \rceil}_{\ell_2}(t^{in} - i_1) \cdot \boldsymbol{\mathcal{E}}^{add}_{t^{out}}(\ell_1, \ell_2) \quad (3)$$

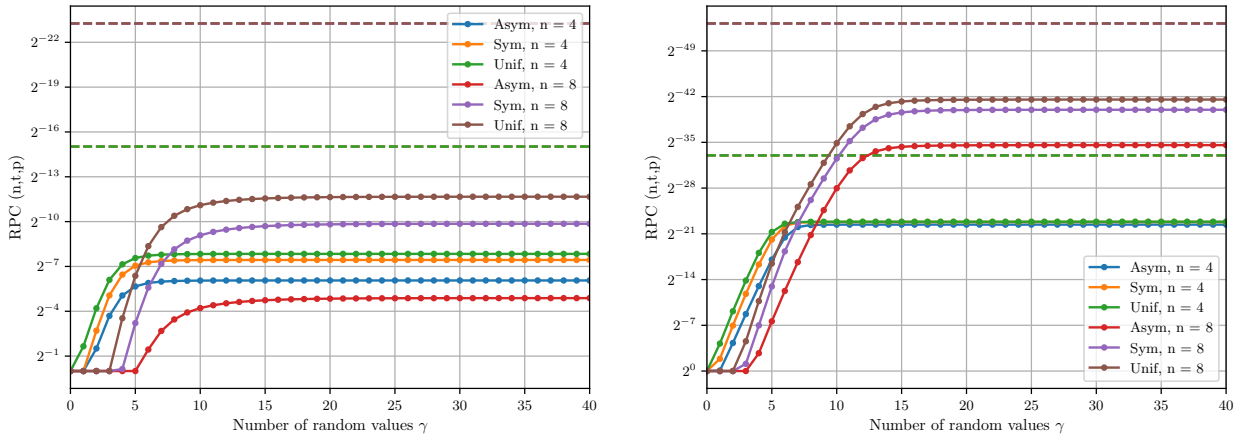
*with*

$$\boldsymbol{\mathcal{E}}^{\mathsf{TA},(2)} = \boldsymbol{\mathcal{E}}^{add} \quad \text{and} \quad \boldsymbol{\mathcal{E}}^{\mathsf{TA},(1)}_{t^{out}}(t^{in}) = \begin{cases} 0 & \text{if } t^{in} \neq t^{out} \\ 1 & \text{otherwise} \end{cases}$$

### 5.3 Results and Comparison

Building on the results from the previous two subsections and Lemma 9, we can derive the cardinal RPC property of our multiplication gadget, denoted CardSecMult, built from Algorithm 1. This construction instantiates Expand with one of the three gadgets MatMultRef, MatMultSym or UnifMatMult, and Comp with TreeAdd. From the cardinal RPC property, we further derive the threshold RPC advantage for CardSecMult using [8, Lemma 1].

Figure 7 illustrates the impact of varying the number $\gamma$ of iterations of the underlying refresh gadget from [8], on the simulation failure probability $\varepsilon$ in the $(p, t, \varepsilon)$-threshold RPC model. We consider several scenarios with $n = 4$ or $n = 8$ shares and a leakage probability $p$ among $2^{-12}$ and $2^{-6}$, always with $t = \frac{n}{2}$. The legend distinguishes between three instantiations of CardSecMult: with MatMultRef ($Asym$), MatMultSym ($Sym$), and UnifMatMult ($Uni$). Dotted lines indicate a lower bound on the simulation failure probability achievable in each case, given by the probability to observe strictly more than $t$ input shares.




(a) $p = 2^{-6}$ (b) $p = 2^{-12}$

Fig. 7: Simulation failure probability $\varepsilon$ of the $(p, t, \varepsilon)$-threshold RPC security for $n \in \{4, 8\}, t = \frac{n}{2}, p \in \{2^{-12}, 2^{-6}\}$.

For each version of CardSecMult, the simulation failure probability decreases as $\gamma$ increases, until it reaches a threshold beyond which further gains become marginal. This threshold value of $\gamma$, which depends on both $n$ and $p$, hence offers a good trade-off between performance and security. Among the three variants, CardSecMult instantiated with UnifMatMult consistently achieves the lowest failure probability, thanks to its

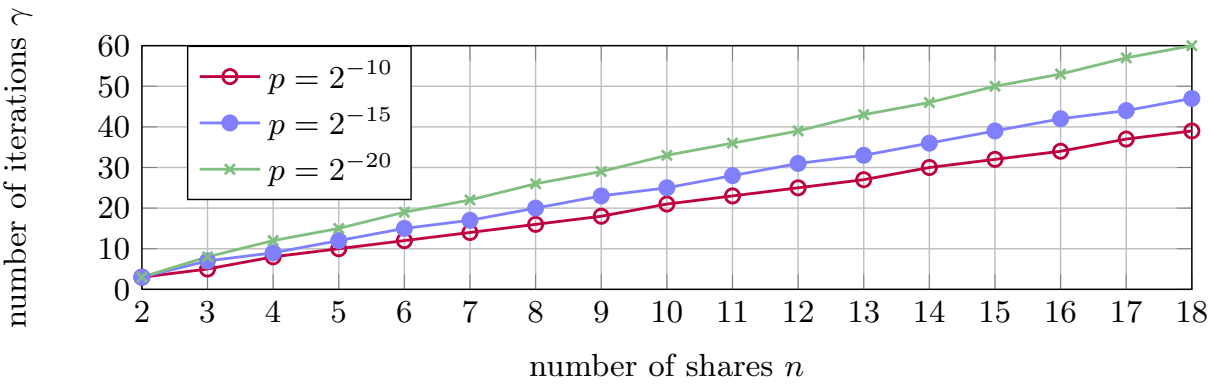



Fig. 8: Number of $\gamma$ values to reach the threshold for a number of shares $n = 2$ to 18 and $p = 2^{-15}$.

additional refresh steps and the use of permutations. This advantage becomes increasingly noticeable as the wire leakage probability $p$ grows.

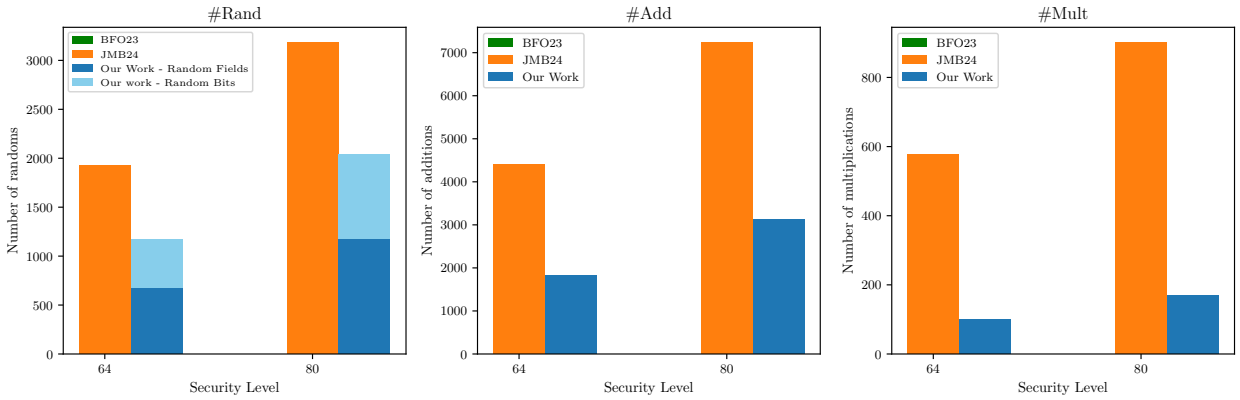
Note that in the MatMultRef (Asym) implementation (based on the original Expand algorithm from[4]), the security level tends to decrease as the number of shares increases, particularly under high leakage probabilities (e.g. $p = 2^{-6}$). This can be attributed to the increased reuse of certain shares when $n$ grows, as some of them are not refreshed before being involved in recursive calls. This observation highlights the importance of refreshing all propagation paths, as implemented in MatMultSym (Sym), and subsequently in UnifMatMult (Uni).

*Remark 2.* Following Remark 1, the cardinal RPC envelopes used to compute the security advantage in Figure 7 split the output of variants of MatMult into four parts and similarly divide the inputs of TreeAdd by four (i.e. $\alpha = 4$). This is feasible when $n$ is a power of two, and thus 4 divides $n^2$. While this optimization is applied in Figure 7, it will not be used in the following sections and benchmarks, as it significantly increases the verification complexity and can not be used with any $n$.

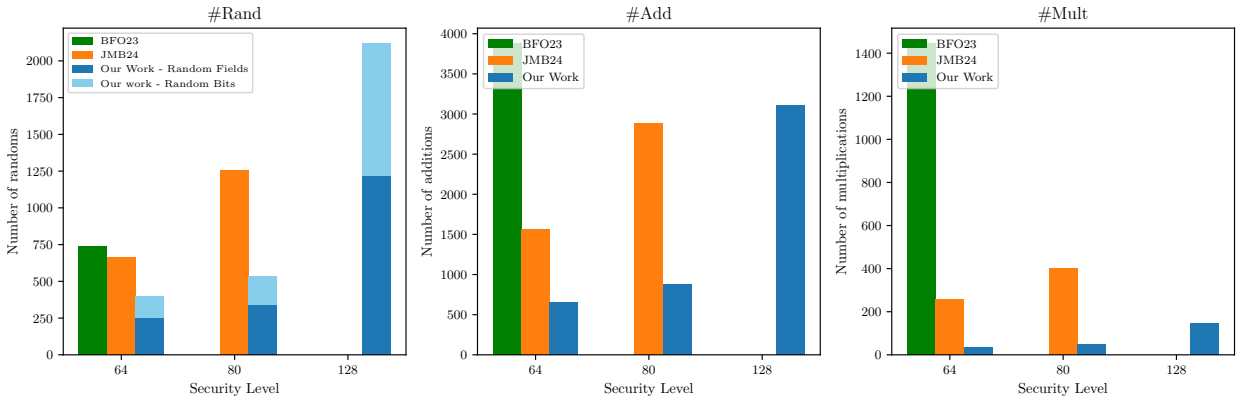
**Fine-tuning of the RPRefresh gadget.** We adopt a strategy to finely tune the value of $\gamma$ (the number of randoms) used in the RPRefresh gadget. Specifically, we adjust $\gamma$ based on the number of shares involved in each recursive invocation of the UnifMatMult algorithm. As discussed earlier, Figure 7 highlights the existence of a threshold $\gamma$ that offers an optimal trade-off between security and computational complexity. Accordingly, for a given number of shares $n$ and leakage rate $p$, our goal is to identify this threshold value of $\gamma$. To do so, we fix a threshold $\delta$ and exhaustively test all values $\gamma$ until the random probing security advantage $\varepsilon_\gamma$ of UnifMatMult is such that

$$|\log_2(\varepsilon_\gamma) - \log_2(\varepsilon_{\gamma+1})| < \delta \ .$$

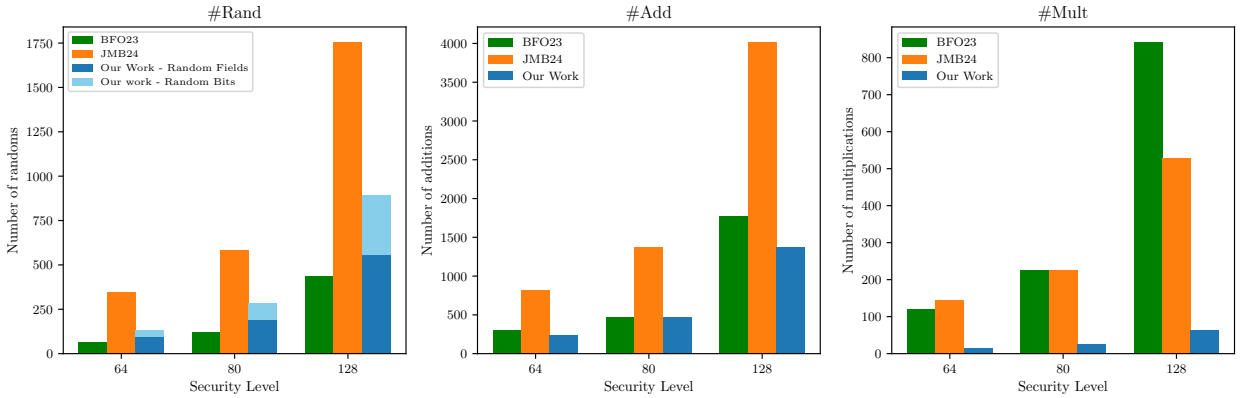
By choosing a small value of $\delta$ (concretely we used $\delta = 0.05$ in our experiments), we can approximate this threshold $\gamma$. We apply this strategy for a range of number of shares $n$. In this way, at each recursive call of UnifMatMult, we use the threshold value $\gamma$ identified for the recursive parameters $n_x$ and $n_y$. These values are generally smaller than the threshold $\gamma$ required for the full UnifMatMult instance, which improves complexity while merely preserving the same level of security. This adaptive choice of $\gamma$ allows us to achieve better security-performance trade-offs. The specific number of iterations $\gamma$ used for the refresh gadget is detailed in Figure 8.




(a) $p = 2^{-10}$




(b) $p = 2^{-15}$




(c) $p = 2^{-20}$

Fig. 9: Complexity comparison of three multiplication gadgets with random probing security of $\varepsilon = 2^{-64}, 2^{-80}, 2^{-128}$.

**Comparison with the state of the art.** Hereafter, we compare our results of security and complexity with the two other state-of-the-art multiplication gadgets for which the random probing security has been previously established[5]:

[5] We omit comparisons with earlier approaches, such as the line of work on random probing expansion, as their trade-offs are expected to be significantly less favorable.

1. The multiplication gadget from [11], referred to as BFO23, is based on the ISW multiplication gadget [18] which follows the same two-step structure as Algorithm 1. For two $n$-sharing $[\![x]\!], [\![y]\!]$, Expand computes for any $i, j \in [n], i \neq j$ the cross-products $x_i \cdot y_j$ and $x_j \cdot y_i$, add them together with a random value $r_{i,j}$ in one share and store the random value in another share of the $n^2$-sharing. If $i = j$, then it computes the cross-product $x_i \cdot y_i$ and store it in a share. The Compress step consists in a tree of additions which outputs an $n$-sharing.
2. The multiplication gadget proposed in [19], referred to as JMB24, adopts the same two-step structure described in Algorithm 1. For the Expand phase, it matches our second variant, MatMultSym, incorporating a refresh of each sub-sharing before every recursive call to ensure stronger security guarantees. For the Comp phase, it retains the original design introduced in [4].

Since both of these works provide analytical bounds on random probing security, we can directly compare our results with theirs. Indeed, a bound on the random probing security of our gadgets can be derived from their cardinal RPC security. In fact, if our multiplication gadget is $(p, \boldsymbol{\mathcal{E}})$ cardinal RPC then it is $(p, \varepsilon)$ random probing secure with

$$\varepsilon = \sum_{\substack{t_1^{\mathtt{in}}, t_2^{\mathtt{in}} \in [0,n] \\ t_1^{\mathtt{in}} = n \text{ or } t_2^{\mathtt{in}} = n}} \boldsymbol{\mathcal{E}}_0(t_1^{\mathtt{in}}, t_2^{\mathtt{in}}).$$

Building on this equality, we compare in Figure 9 the complexity of the different multiplication designs for a leakage rate $p \in \{2^{-10}, 2^{-15}, 2^{-20}\}$ and a target random probing security level $\varepsilon \in \{2^{-64}, 2^{-80}, 2^{-128}\}$. For the randomness complexity of our gadget, we further account for the random bits used in selecting random share indices during the refresh, as well as those used for the random permutations (highlighted in light blue). As previously discussed, this form of randomness is likely to be less costly in practice than the randomness required for implementing random gates.

BF023 [11] is absent from several scenarios, as it fails to achieve the selected security levels for some of the tested leakage rates, regardless of the number of shares $n \in \mathbb{N}$. Similarly, the analytical formula provided in JMB24 [19] is only valid up to $n = 30$ shares, which is insufficient to reach some target security levels when the leakage rate is low. Consequently, some scenarios include both comparisons, others only JMB24, and a few rely exclusively on our own results.

As illustrated in the figure, our proposal generally achieves significantly better complexity than existing state-of-the-art approaches, except in a few scenarios with low leakage rates and low target security levels, where the required amount of randomness is comparable to that of previous works.

# 6 Compiler and Application to AES

## 6.1 Description of the Compiler

Our proposed circuit compiler features the following gadgets:

- *Addition and multiplication by a constant:* $\mathsf{Add}^\gamma$ and $\mathsf{cMult}^\gamma$ from [8] , described in Algorithm 6 and 7;
- *Multiplication:* The UnifMatMult gadget presented in Section 5;
- *Addition of a constant:* The linear gadget $\mathsf{cAdd}^\gamma$ adds a constant $c \in \mathbb{K}$ by first refreshing the inputs, then adding $c$ to the first share. It is formally described in Algorithm 8 and its cardinal RPC security is provided in Lemma 11 (see Appendix C.2);
- *Squaring (characteristic 2 only):* In the case where $\mathbb{K} = \mathbb{F}_{2^k}$ with $k \in \mathbb{N}^*$ (as for the application to AES), we define a squaring gadget $\mathsf{Sq}^\gamma$, which takes as input an $n$-sharing, refreshes it, and squares each share component-wise. It is formalized in Algorithm 9 in Appendix C.2, and its cardinal RPC security can be computed similarly to that of $\mathsf{cMult}^\gamma$.

Moreover, our compiler inserts random permutations and refresh gadgets in targeted regions of the circuit to deal with copies. Specifically, we consider two situations:

1. *Fan-out 2 setting:* When an $n$-sharing serves as input to exactly two subsequent gadgets, we simply apply a permutation before the input of one of the two gadgets. This approach preserves the *UI-cardinal RPC* property, as established in Lemma 3.
2. *Fan-out greater than 2 setting:* A dedicated procedure is required to safely reuse the same sharing across multiple gadgets. Without appropriate safeguards, repeated use of the same sharing increases the probability of leakage from each individual share, thereby significantly weakening the overall random probing security. To address this pitfall, whenever a sharing $[\![x]\!]^{(0)}$ is used more than twice (we cannot technically avoid the use of two copies of a single sharing), we use a refresh gadget to securely duplicate the data. Specifically, the sharing $[\![x]\!]^{(0)}$ is used directly in the first gadget, after which it is refreshed to produce a new independent sharing $[\![x]\!]^{(1)}$. Like in the fan-out 2 setting, a permutation is applied before the first gadget or before the refreshing. If no further duplications are required, the refreshed copy can also be used directly in the second and the third gadgets, with a permutation inserted in one of the branches (as in the fan-out 2 setting). Otherwise, the procedure continues recursively: one copy is used directly, and the other is refreshed again to produce $[\![x]\!]^{(2)}$, with a permutation applied to one of the inputs, and so on.
   Figure 10 illustrates the procedures for a fan-out 3.

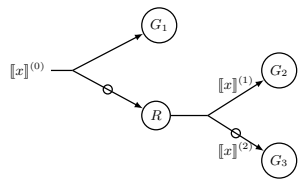



Fig. 10: Illustration of the duplication of a sharing $[\![x]\!]^{(0)}$ across three gadgets $G_1$, $G_2$, and $G_3$. The black circles on the edges indicate the permutations applied to the shares.

### 6.2 Composition Strategy for the Compiled Circuits

We now describe the composition strategy to evaluate the random probing security of compiled circuits. Ideally, this would involve computing the cardinal RPC envelope of the entire compiled circuit. However, this approach is often impractical, particularly in the case of circuits with numerous input and output sharings, each composed of a substantial number of shares. The complexity of such an analysis grows at least as $(n+1)^{\ell+m}$, where $\ell$ and $m$ denote the number of $n$-shared inputs and outputs, respectively.

This is why we adopt a *block-wise approach* for computing the random probing security of larger circuits. We start by partitioning our larger circuit into blocks of gadgets for which the global cardinal RPC envelopes can be computed within a reasonable amount of time. Subsequently, we use [8, Lemma 1] to infer the threshold RPC security of each block from its cardinal RPC security, which in turn enables us to upper bound the overall random probing security of the compiled circuit.

To ensure efficient computation of the block-level cardinal RPC security, we impose constraints on the size of each block. Specifically, given $\ell$ input sharings and $m$ output sharings with $n$ shares, we require that $n^{\ell+m} < 2^{\delta}$, where $\delta$ is a chosen threshold value (we use $\delta = 30$ in the application to AES presented below). In this paper, we do not focus on optimizing the block partitioning strategy to maximize the global random probing security, this is left as a direction for future work. Instead, we rely on manual clustering of the compiled circuits, as exemplified hereafter in the case of AES.

### 6.3 Application to AES

AES is a block cipher, which operates on blocks of 16 bytes. Each byte $b \in \mathbb{F}_{256}$ is a sensible variable of the computation which need to be masked and processed as a sharing $[\![b]\!]$. We describe hereafter a masked AES

implementation using our compiler. We start by describing the circuits of the four building blocks of an AES implementation. Namely, we describe the circuits of AddRoundKey, SubBytes, MixColumns and ShiftRows.

AddRoundKey. This operation consists of adding a 16-byte round key block to the current 16-byte AES state. The masked circuit is constructed by applying 16 addition gadgets, each operating on a shared byte of the state and the corresponding shared byte from the round key. As a result, we obtain 16 envelopes for the AddRoundKey step, each one associated to one shared byte of the state and one shared byte of the round key.

Subbytes. This operation is the only non-linear part of the AES. It consists in applying an s-box to each byte of the state. This s-box has a cricuit representation which is composed of two parts. The *exponentiation part* maps the input byte $b \in \mathbb{F}_{2^8}$ to $b^{254}$ (which is the multiplicative inverse for every $n \neq 0$ and 0 for $b = 0$). This step can be realized through an addition chain to obtain 254, for instance ($1 \to 2 \to 3 \to 12 \to 15 \to 240 \to 252 \to 254$). Figure 11 describes the exponentiation circuit used in our compiler, where the small empty gate are the permutations.

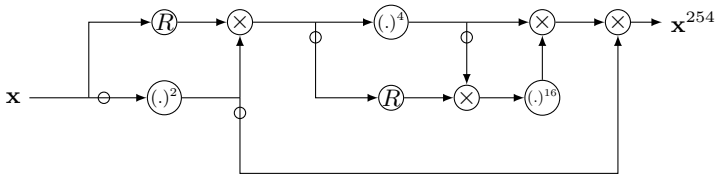



Fig. 11: Exponentiation circuit.

*Remark 3.* Compared to the rules enforced by the compiler, we manually insert two additional refresh gadgets on the first input variable of the first two multiplication gadgets. This choice was motivated by the observation that refreshing this specific input significantly improves the security level of the block and, consequently, provides a better security–complexity trade-off.

The *affine part* of the s-box consists in the application of the function $\mathsf{Affine} : \mathbb{F}_{2^8} \to \mathbb{F}_{2^8}$ defined as:

$$\mathsf{Affine}(b) = (((((((207b)^2 + 22b)^2 + b)^2 + 73b)^2 + 204b)^2 + 168b)^2 + 238b)^2 + 5b + 99 \ .$$

While the above function is used for AES encryption, the inverse function involved for an AES decryption satisfies:

$$\mathsf{Affine}^{-1}(b) = (((((((147b)^2 + 146b)^2 + 190b)^2 + 41b)^2 + 73b)^2 + 139b)^2 + 79b)^2 + 5b + 5 \ .$$

The circuit for the Affine and $\mathsf{Affine}^{-1}$ functions is depicted in Figure 12.

For AES encryption, the Subbytes operation applies the exponentiation part followed by the Affine to each shared byte composing the state. For AES decryption, the inverse Subbytes operation applies the $\mathsf{Affine}^{-1}$ function followed by the exponentiation part to each shared byte composing the state. In both cases, we end up with 16 envelopes for the (inverse) SubBytes operation, each one associated to one shared byte of the state.

*Remark 4.* Since the AddRoundKey and SubBytes follow one another, we can compose their cardinal RPC envelopes to enhance the threshold RPC security results by building bigger blocks. We will use this optimisation to end up with 16 envelopes for the 16 computations: AddRoundKey $\to$ SubBytes.

*ShiftRows.* This operation performs a permutation of the row of the state. It affects only the wiring of the circuit but do not involve any gadgets.

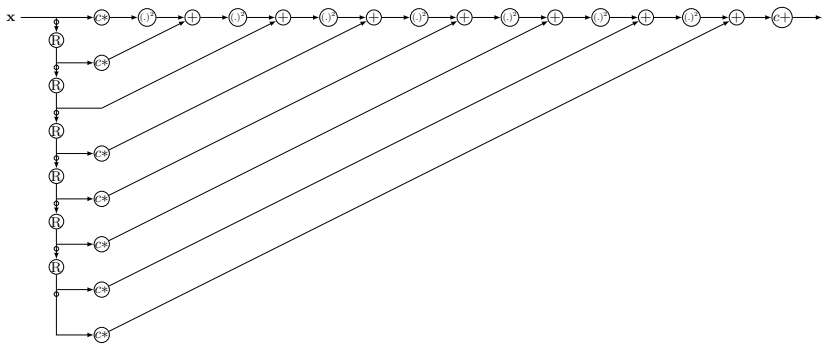


Fig. 12: Circuit for Affine and Affine$^{-1}$.

*MixColumns.* This linear transformation takes in input a state $B = (b_{i,j})_{0 \leq i,j \leq 3}$ and computes an output state $B' = (b'_{i,j})_{0 \leq i,j \leq 3}$ where, for all $j \in [0, 3]$:

$$\begin{aligned} b'_{0,j} &= 02 \cdot b_{0,j} \oplus 03 \cdot b_{1,j} \oplus b_{2,j} \oplus b_{3,j} \\ b'_{1,j} &= b_{0,j} \oplus 02 \cdot b_{1,j} \oplus 03 \cdot b_{2,j} \oplus b_{3,j} \\ b'_{2,j} &= b_{0,j} \oplus b_{1,j} \oplus 02 \cdot b_{2,j} \oplus 03 \cdot b_{3,j} \\ b'_{3,j} &= 03 \cdot b_{0,j} \oplus b_{1,j} \oplus b_{2,j} \oplus 02 \cdot b_{3,j} \end{aligned}$$

Figure 13 gives the circuit for the MixColumns operation applied to one column of the state (out of 4). We were not able to compute the overall envelopes of the MixColumns block as its complexity is too big. However, we can split the MixColumns circuit into three sub-circuits: (i & ii) The blue-framed circuits perform all the operations of MixColumns, except for the final addition that produces the four $n$-sharings. This final step is handled separately in two distinct blocks, which simplifies the computation of this part of the process and (iii) the red-framed circuit corresponds only to 4 additions gadgets in parallel, that we can merge to the next block (namely an AddRoundKey $\rightarrow$ SubBytes block). This way, we can add this part to a bigger block and avoid going through the threshold RPC security.

**RPC security of the overall AES.** We end up with

- 16 blocks of AddRoundKey $\rightarrow$ SubBytes with known cardinal RPC security envelopes $\boldsymbol{\mathcal{E}}^1$.
- $8 \times 16 = 128$ blocks of MixColumns $\rightarrow$ AddRoundKey $\rightarrow$ SubBytes with known cardinal RPC security envelopes $\boldsymbol{\mathcal{E}}^2$.
- 16 blocks of MixColumns $\rightarrow$ AddRoundKey $\rightarrow$ SubBytes $\rightarrow$ AddRoundKey with known cardinal RPC security envelopes $\boldsymbol{\mathcal{E}}^3$.
- $9 \times 8 = 72$ blocks for the first part of MixColumns with known cardinal RPC security envelopes $\boldsymbol{\mathcal{E}}^4$.

From these cardinal RPC envelopes, we derive the threshold RPC security for each block. We note $\varepsilon_t^i$ the $t$-threshold RPC security derived from the envelopes $\boldsymbol{\mathcal{E}}^i$. Finally, we take $\varepsilon = \min_t \max(\varepsilon_t^1, \varepsilon_t^2, \varepsilon_t^3, \varepsilon_t^4)$. The AES implementation is then $(p, 232 \cdot \varepsilon)$ random probing secure.

### 6.4 Results and Comparison

Figure 14 illustrates the security-complexity trade-offs of the resulting masked AES in the cardinal RPC framework and compares them with prior random probing secure AES compilers. Specifically, for leakage rates $p \in \{2^{-16}, 2^{-20}\}$, our construction achieves random probing security levels $\varepsilon \in \{2^{-64}, 2^{-80}, 2^{-128}\}$. We

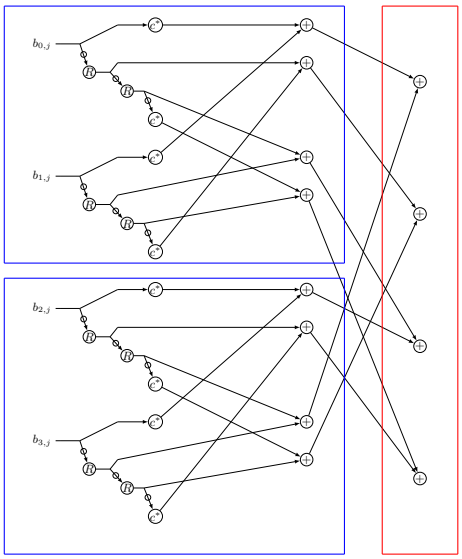
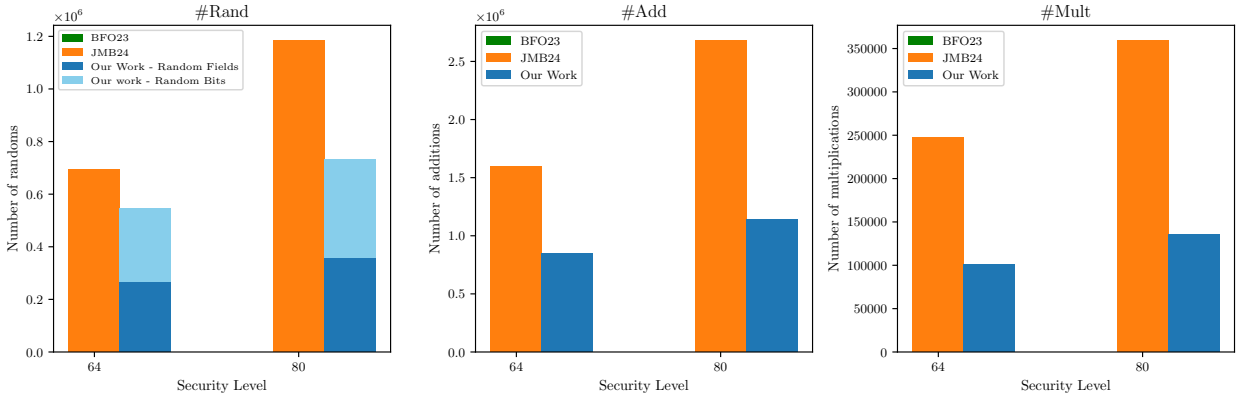



Fig. 13: Circuit for the computation of one column of MixColumns

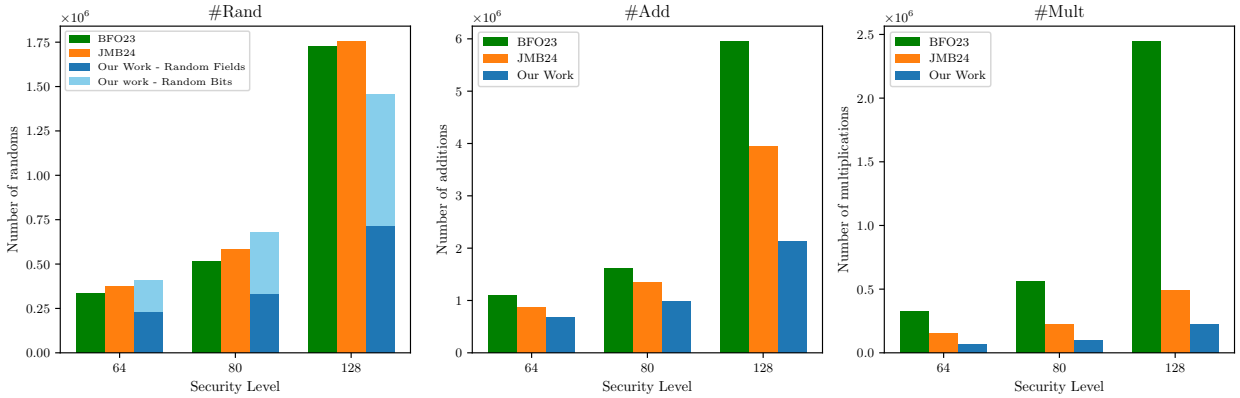
also report the randomness usage (with light blue for the random bits dedicated to the permutations, that can be revealed) and the counts of field additions and multiplications required in each of these settings, and compare with the following compilers that provide concrete random probing security bounds:

- **Expansion Strategy** [6,9,10]: This approach introduces a compiler equipped with addition, copy, refresh, and multiplication gadgets. Its application to AES has been studied in [6,10,5], resulting in complexities several orders of magnitude higher than those obtained by our method. For example, [5, Table 4] reports that achieving 128-bit security at a leakage rate of $2^{-30}$ requires approximately $4.33 \cdot 10^9$ gates. Due to this significant gap, Figure 14 does not include results for the expansion strategy.
- **BFO23** [11]: This compiler also uses addition, copy, refresh, and multiplication gadgets. Notably, it inserts a refresh gadget between each operation to ensure composability. The authors also provide an analytical formula to estimate the random probing security of their circuits. We applied this compiler to our masked AES implementation for comparison. For $p = 2^{-16}$, the achieved security level never exceeds 44 bits (i.e. the failure probability remains above $2^{-44}$) for any number of shares $n \in \mathbb{N}^*$, making meaningful comparisons with our results infeasible. For $p = 2^{-20}$, our compiler achieves lower complexities (or similar for random complexities) for all three target security levels.
- **JMB24** [19]: This work presents a masked AES implementation with random probing security levels expressed as functions of $n$ and $p$. As shown in Figure 14, our compiler consistently outperforms JMB24 in both complexity and randomness usage for each leakage probability and target security level. It also offers a strictly stronger guarantee: our construction ensures a security level $\varepsilon$ in the standard simulation-based random probing security model, whereas JMB24 only upper bounds the mutual information between the leakage and a single byte of the secret key. Such a mutual information bound only limits average leakage and does not yield simulation-based security, whereas simulation-based security (when the distance between the simulated leakage and the actual leakage is statistical) both implies the corresponding mutual information bound and composes well within larger protocols while preserving their security.

Beyond these results, we also compare the combination of our gadgets under the threshold RPC framework with their combination under the block-level cardinal RPC setting. At $p = 2^{-20}$, Figure 14(b) shows our block-level cardinal RPC compiler already exceeds the 64-bit target (with exactly $\varepsilon = 2^{-67}$) with $n = 8$ shares. In the same setting $(p, n)$, combining directly the threshold RPC advantages of our gadgets (copies included) would only yield $\varepsilon = 2^{-61}$. This gap highlights the benefit of (block-level) cardinal RPC.




(a) $p = 2^{-16}$




(b) $p = 2^{-20}$

Fig. 14: Number of randoms, additions and multiplications for three security levels.

**Acknowledgments.** This work was funded by the European Union (ERC, AMAskZONE, 101077506) and Bpifrance (France national quantum strategy, RESQUE project). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

## References


1. Miklós Ajtai. Secure computation with information leaking to an adversary. In Lance Fortnow and Salil P. Vadhan, editors, *43rd Annual ACM Symposium on Theory of Computing*, pages 715–724, San Jose, CA, USA, June 6–8, 2011. ACM Press.
2. Prabhanjan Ananth, Yuval Ishai, and Amit Sahai. Private circuits: A modular approach. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 427–455, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
3. Marcin Andrychowicz, Stefan Dziembowski, and Sebastian Faust. Circuit compilers with $O(1/\log(n))$ leakage rate. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 586–615, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany.
4. Alberto Battistello, Jean-Sébastien Coron, Emmanuel Prouff, and Rina Zeitoun. Horizontal side-channel attacks and countermeasures on the ISW masking scheme. In Benedikt Gierlichs and Axel Y. Poschmann, editors, *Cryptographic Hardware and Embedded Systems – CHES 2016*, volume 9813 of *Lecture Notes in Computer Science*, pages 23–39, Santa Barbara, CA, USA, August 17–19, 2016. Springer Berlin Heidelberg, Germany.
5. Sonia Belaïd, Gaëtan Cassiers, Camille Mutschler, Matthieu Rivain, Thomas Roche, François-Xavier Standaert, and Abdul Rahman Taleb. Sok: A methodology to achieve provable side-channel security in real-world implementations. *IACR Commun. Cryptol.*, 2(1):4, 2025.
6. Sonia Belaïd, Jean-Sébastien Coron, Emmanuel Prouff, Matthieu Rivain, and Abdul Rahman Taleb. Random probing security: Verification, composition, expansion and new constructions. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 339–368, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
7. Sonia Belaïd, Darius Mercadier, Matthieu Rivain, and Abdul Rahman Taleb. IronMask: Versatile verification of masking security. In *2022 IEEE Symposium on Security and Privacy*, pages 142–160, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.
8. Sonia Belaïd, Matthieu Rivain, and Mélissa Rossi. New techniques for random probing security and application to raccoon signature scheme. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 94–123, Cham, 2025. Springer Nature Switzerland.
9. Sonia Belaïd, Matthieu Rivain, and Abdul Rahman Taleb. On the power of expansion: More efficient constructions in the random probing model. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 313–343, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
10. Sonia Belaïd, Matthieu Rivain, Abdul Rahman Taleb, and Damien Vergnaud. Dynamic random probing expansion with quasi linear asymptotic complexity. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 157–188, Singapore, December 6–10, 2021. Springer, Cham, Switzerland.
11. Francesco Berti, Sebastian Faust, and Maximilian Orlt. Provable secure parallel gadgets. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):420–459, 2023.
12. Gaëtan Cassiers, Sebastian Faust, Maximilian Orlt, and François-Xavier Standaert. Towards tight random probing security. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 185–214, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
13. Gaëtan Cassiers and François-Xavier Standaert. Towards globally optimized masking: From low randomness to low noise rate. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2019(2):162–198, 2019.
14. Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part I*, volume 14920 of *Lecture Notes in Computer Science*, pages 409–444, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
15. Alexandre Duc, Stefan Dziembowski, and Sebastian Faust. Unifying leakage models: From probing attacks to noisy leakage. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 423–440, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
16. Stefan Dziembowski, Sebastian Faust, and Karol Zebrowski. Simple refreshing in the noisy leakage model. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part III*, volume

11923 of *Lecture Notes in Computer Science*, pages 315–344, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.

17. Dahmun Goudarzi, Antoine Joux, and Matthieu Rivain. How to securely compute with noisy leakage in quasilinear complexity. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part II*, volume 11273 of *Lecture Notes in Computer Science*, pages 547–574, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland.
18. Yuval Ishai, Amit Sahai, and David Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481, Santa Barbara, CA, USA, August 17–21, 2003. Springer Berlin Heidelberg, Germany.
19. Vahid Jahandideh, Bart Mennink, and Lejla Batina. An algebraic approach for evaluating random probing security with application to AES. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(4):657–689, 2024.
20. Emmanuel Prouff and Matthieu Rivain. Masking against side-channel attacks: A formal security proof. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 142–159, Athens, Greece, May 26–30, 2013. Springer Berlin Heidelberg, Germany.

# A Recall of Random Probing Partitioning Definition

**Definition 10 (Partition ZeroEncoding from [8]).** *For $P = (n_1, \ldots, n_\ell) \in \mathcal{P}(n)$ an integer partition, we define* $\mathsf{PartitionZeroEnc}(P)$ *as a particular type of distribution producing a sharing $[\![z]\!] = [\![z_1]\!] \parallel \ldots \parallel [\![z_\ell]\!]$ where $[\![z_i]\!]$ is a fresh $n_i$-sharing of $0$ for every $i \in [1, \ell]$. In other words,*

$$\mathsf{PartitionZeroEnc}(n_1, \ldots, n_\ell) = \underbrace{z_1, \cdots, z_{n_1}}_{\sum z_i = 0}, \underbrace{z_{n_1+1}, \cdots, z_{n_1+n_2}}_{\sum z_i = 0}, \cdots, \underbrace{z_{n-n_\ell+1}, \cdots, z_n}_{\sum z_i = 0}.$$

**Definition 11 (Random Probing Partitionning from [8]).** *Let $n \in \mathbb{N}$ and $\mathcal{P}(n)$ be the set of partitions of $n$. Let $\mathcal{E}$ be a distribution envelope over $\mathcal{P}(n)$. An $n$-share zero-encoding gadget* $\mathsf{ZeroEnc} : \emptyset \to \mathbb{K}^n$ *is* $(p, \mathcal{E})$-random-probing-partitioned *(RPP) for some $p \in [0, 1]$ if there exists a deterministic algorithm $\phi$ and a PPT simulator* $\mathsf{Sim}$ *such that the outputs of the random experiment 2 satisfy:*

1. *$P \lesssim \mathcal{E}$, and*
2. *$[\![z]\!]$ and $[\![w]\!] + [\![u]\!]$ are identically distributed.*

*Experiment 2: Random Probing Partition*

| | |
|---|---|
| $\mathcal{W} \leftarrow \mathsf{LeakingWires}(\mathsf{ZeroEnc}, p)$ | Drawing the leaking wires. |
| $\rho \leftarrow \$$ | Drawing a random tape. |
| $[\![z]\!] \leftarrow \mathsf{ZeroEnc}(\rho)$ | Drawing $[\![z]\!]$ an encoding of $0$. |
| $leak \leftarrow \mathsf{AssignWires}(\mathsf{ZeroEnc}, \rho, \mathcal{W}, \emptyset)$ | Computing the leaked values. |
| $[\![w]\!] \leftarrow \mathsf{Sim}(\mathcal{W}, leak)$ | The leakages are given to the simulator. It outputs an encoding of $0$ whose randomness is "weakened" due to the leakage. |
| $(P, \sigma) := \phi(\mathcal{W})$ | Computing a partition and a permutation corresponding to the leaking wires. |
| $[\![u]\!] \leftarrow \sigma\big(\mathsf{PartitionZeroEnc}(P)\big)$ | Drawing an encoding of $0$ corresponding to the "remaining" randomness to compensate the randomness that was given in the leakage. |
| *return* $(P, [\![z]\!], [\![w]\!], [\![u]\!])$ | |

# B Proofs of Section 4

*Proof (Proof of Lemma 2).* Let $n, \ell, m \in \mathbb{N}$ and let $G : (\mathbb{K}^n)^\ell \to (\mathbb{K}^n)^m$ be an $n$-share gadget. Let $p \in [0, 1]$.

On the one hand, we suppose that $G$ is *uniformly-$(p, \boldsymbol{\mathcal{E}}^u)$-cardinal random probing composable*. We show that $G$ is $(p, \boldsymbol{\mathcal{E}}^{ui})$-*UI-cardinal random probing composable* and $(p, \boldsymbol{\mathcal{E}}^{uo})$-*UO-cardinal random probing composable* and that

$$\boldsymbol{\mathcal{E}}^u = \boldsymbol{\mathcal{E}}^{ui} = \boldsymbol{\mathcal{E}}^{uo}$$

Let $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$ and $(J'_1, \ldots, J'_m) \in \mathcal{P}([n])^m$ such that

$$(|J_1|, \ldots, |J_m|) = (|J'_1|, \ldots, |J'_m|)$$

and let $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ and $(I'_1, \ldots, I'_\ell) \in \mathcal{P}([n])^\ell$ such that

$$(|I_1|, \ldots, |I_\ell|) = (|I'_1|, \ldots, |I'_\ell|)$$

By the uniformly cardinal RPC property of $G$, we know that $G$ is $(p, \boldsymbol{\mathcal{E}}^g)$-general RPC with

$$\boldsymbol{\mathcal{E}}^g_{J_1, \ldots, J_m}(I_1, \ldots, I_\ell) = \boldsymbol{\mathcal{E}}^g_{J_1, \ldots, J_m}(I'_1, \ldots, I'_\ell) \tag{4}$$

and with

$$\boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell) = \boldsymbol{\mathcal{E}}^g_{J'_1,\ldots,J'_m}(I_1,\ldots,I_\ell) \tag{5}$$

Therefore, by Equation 4, $G$ is $(p,\boldsymbol{\mathcal{E}}^{ui})$-*UI-cardinal random probing composable* such that for all $(t_1^{\mathtt{in}},\ldots,t_\ell^{\mathtt{in}}) \in [0,n]^\ell$ and for all $(t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}) \in [0,n]^m$ we have

$$\begin{aligned}
\boldsymbol{\mathcal{E}}^{ui}_{t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}}(t_1^{\mathtt{in}},\ldots,t_\ell^{\mathtt{in}}) &= \max_{\substack{J'_1,\ldots J'_m \\ |J'_1|=t_1^{\mathtt{out}},\ldots,|J'_m|=t_m^{\mathtt{out}}}} \boldsymbol{\mathcal{E}}^g_{J'_1,\ldots,J'_m}(I_1,\ldots,I_\ell)\prod_{\alpha=1}^{\ell}\binom{n}{t_\alpha^{\mathtt{in}}} \\
&\overset{5}{=} \boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell)\prod_{\alpha=1}^{\ell}\binom{n}{t_\alpha^{\mathtt{in}}} \\
&= \boldsymbol{\mathcal{E}}^u_{t_1^{\mathtt{out}},\ldots t_m^{\mathtt{out}}}(t_1^{\mathtt{in}},\ldots,t_\ell^{\mathtt{in}})
\end{aligned}$$

where $(J_1,\ldots J_m) \in \mathcal{P}([n])^m$ with $|J_1| = t_1^{\mathtt{out}},\ldots,|J_m| = t_m^{\mathtt{out}}$ and $(I_1,\ldots,I_\ell) \in \mathcal{P}([n])^\ell$ with $|I_1| = t_1^{\mathtt{in}},\ldots,|I_\ell| = t_\ell^{\mathtt{in}}$. Furthermore, by Equation 5, $G$ is $(p,\boldsymbol{\mathcal{E}}^{uo})$-*UO-cardinal random probing composable* such that

$$\begin{aligned}
\boldsymbol{\mathcal{E}}^{uo}_{t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}}(t_1^{\mathtt{in}},\ldots,t_\ell^{\mathtt{in}}) &= \sum_{\substack{I'_1,\ldots,I'_\ell \\ |I'_1|=t_1^{\mathtt{in}},\ldots,|I'_\ell|=t_\ell^{\mathtt{in}}}} \boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I'_1,\ldots,I'_\ell) \\
&\overset{4}{=} \boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell)\prod_{\alpha=1}^{\ell}\binom{n}{t_\alpha^{\mathtt{in}}} \\
&= \boldsymbol{\mathcal{E}}^u_{t_1^{\mathtt{out}},\ldots t_m^{\mathtt{out}}}(t_1^{\mathtt{in}},\ldots,t_\ell^{\mathtt{in}})
\end{aligned}$$

where $(J_1,\ldots J_m) \in \mathcal{P}([n])^m$ with $|J_1| = t_1^{\mathtt{out}},\ldots,|J_m| = t_m^{\mathtt{out}}$ and $(I_1,\ldots,I_\ell) \in \mathcal{P}([n])^\ell$ with $|I_1| = t_1^{\mathtt{in}},\ldots,|I_\ell| = t_\ell^{\mathtt{in}}$. Finally we have

$$\boldsymbol{\mathcal{E}}^u = \boldsymbol{\mathcal{E}}^{ui} = \boldsymbol{\mathcal{E}}^{uo}.$$

On the other hand, we suppose that $G$ is $(p,\boldsymbol{\mathcal{E}}^{ui})$-*UI-cardinal random probing composable* and $(p,\boldsymbol{\mathcal{E}}^{uo})$-*UO-cardinal random probing composable* with $\boldsymbol{\mathcal{E}}^u := \boldsymbol{\mathcal{E}}^{ui} = \boldsymbol{\mathcal{E}}^{uo}$. Then we show that $G$ is *uniformly*-$(p,\boldsymbol{\mathcal{E}}^u)$-*cardinal random probing composable*.Let $(J_1,\ldots,J_m) \in \mathcal{P}([n])^m$ and $(J'_1,\ldots,J'_m) \in \mathcal{P}([n])^m$ such that

$$(|J_1|,\ldots,|J_m|) = (|J'_1|,\ldots,|J'_m|)$$

and let $(I_1,\ldots,I_\ell) \in \mathcal{P}([n])^\ell$ and $(I'_1,\ldots,I'_\ell) \in \mathcal{P}([n])^\ell$ such that

$$(|I_1|,\ldots,|I_\ell|) = (|I'_1|,\ldots,|I'_\ell|)$$

By the *UI-cardinal random probing composability* we know that $G$ is $(p,\boldsymbol{\mathcal{E}}^g)$-general RPC with

$$\boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell) \overset{UI}{=} \mathcal{E}^g_{J_1,\ldots,J_m}(I'_1,\ldots,I'_\ell) \tag{6}$$

Moreover, by the *UO-cardinal random probing composability*, we know that

$$\sum_{\substack{I_1*\subseteq[n],\ldots,I_\ell*\subseteq[n] \\ |I_1*|=|I_1|,\ldots,|I_\ell*|=|I_\ell|}} \boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1*,\ldots,I_\ell*) = \sum_{\substack{I_1*\subseteq[n],\ldots,I_\ell*\subseteq[n] \\ |I_1*|=|I_1|,\ldots,|I_\ell*|=|I_\ell|}} \boldsymbol{\mathcal{E}}^g_{J'_1,\ldots,J'_m}(I_1*,\ldots,I_\ell*)$$

In particular, by Equation 6, we obtain

$$c.\boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell) = c.\boldsymbol{\mathcal{E}}^g_{J'_1,\ldots,J'_m}(I'_1,\ldots,I'_\ell)$$

where $c := |\{I_1* \subseteq [n], \ldots, I_\ell* \subseteq [n]$ such that $|I_1*| = |I_1|, \ldots, |I_\ell*| = |I_\ell|\}|$. Therefore, dividing by $c$, we show that $G$ is *uniformly-*$(p, \boldsymbol{\mathcal{E}})$*-cardinal random probing composable* such that for all $(t_1^{\mathtt{in}}, \ldots, t_\ell^{\mathtt{in}}) \in [0, n]^\ell$ and $(t_1^{\mathtt{out}}, \ldots, t_m^{\mathtt{out}}) \in [0, n]^m$ we have

$$\begin{aligned}
\boldsymbol{\mathcal{E}}_{t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}}(t_1^{\mathtt{in}}, \ldots, t_\ell^{\mathtt{in}}) &= \boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I_1, \ldots, I_\ell) \prod_{\alpha=1}^{\ell} \binom{n}{t_\alpha^{\mathtt{in}}} \\
&\overset{UI}{=} \sum_{\substack{I'_1,\ldots,I'_\ell \\ |I'_1|=t_1^{\mathtt{in}},\ldots,|I'_\ell|=t_\ell^{\mathtt{in}}}} \boldsymbol{\mathcal{E}}^g_{J_1,\ldots,J_m}(I'_1, \ldots, I'_\ell) \\
&= \boldsymbol{\mathcal{E}}^{ui}_{t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}}(t_1^{\mathtt{in}}, \ldots, t_\ell^{\mathtt{in}}) \\
&= \boldsymbol{\mathcal{E}}^{u}_{t_1^{\mathtt{out}},\ldots,t_m^{\mathtt{out}}}(t_1^{\mathtt{in}}, \ldots, t_\ell^{\mathtt{in}})
\end{aligned}$$

Finally, $G$ is *uniformly-*$(p, \boldsymbol{\mathcal{E}}^u)$*-cardinal random probing composable*. This concludes the proof. □

*Proof (Proof of Lemma 3).* Let $[\![\mathbf{x}]\!] \in \mathbb{K}^n$ , $J_1 \subseteq [n], J_2 \subseteq [n]$, and $\mathcal{W} = \mathsf{LeakingWires}(G, p)$. We split $\mathcal{W}$ into two parts: the set $\mathcal{W}_1$ containing the wires of $G_1$ and the set $\mathcal{W}_2$ containing the wires in $G_2$. We then build two simulators $(\mathsf{Sim}_1, \mathsf{Sim}_2)$ from the simulators $(\mathsf{Sim}_1^{G_1}, \mathsf{Sim}_2^{G_1})$ and $(\mathsf{Sim}_1^{G_2}, \mathsf{Sim}_2^{G_2})$ of the cardinal RPC property of $G_1$ and $G_2$ (see Definition 7).

$$\mathsf{Sim}_1(G, \mathcal{W}, J_1, J_2) := \mathsf{Sim}_1^{G_1}(G_1, \mathcal{W}_1, J_1) \cup \mathsf{Sim}_1^{G_2}(G_2, \mathcal{W}_2, J_2).$$

Let $I_1 \leftarrow \mathsf{Sim}_1^{G_1}(G_1, \mathcal{W}_1, J_1)$ and $I_2 \leftarrow \mathsf{Sim}_1^{G_2}(G_2, \mathcal{W}_2, J_2)$ denote the resulting sets. Therefore, the resulting input set for $\mathsf{Sim}_1(G, \mathcal{W}, J_1, J_2)$ is $I = I_1 \cup I_2$. We then define the second simulator as follows:

$$\mathsf{Sim}_2([\![\mathbf{x}]\!]) := (out_1^1 || out_1^2, out_2^1 || out_2^2)$$

where $(out_1^1, out_2^1) \leftarrow \mathsf{Sim}_2^{G_1}([\![\mathbf{x}]\!]|_I)$ and $(out_1^2, out_2^2) \leftarrow \mathsf{Sim}_2^{G_2}([\![\mathbf{x}]\!]|_I)$.

Since $out_1^1$ is identically distributed to the internal leakage of $\mathcal{W}_1$ in $G_1$ (because $I_1 \subseteq I$) and $out_1^2$ is identically distributed to the internal leakage of $\mathcal{W}_2$ in $G_2$ (because $I_2 \subseteq I$), it follows that $out_1 := (out_1^1 || out_1^2)$ is identically distributed to the internal leakage $\mathcal{W}$ in $G$. Similarly, $out_2 := (out_2^1 || out_2^2)$ is identically distributed to the output leakage of $G$. This ensures that the second condition of the cardinal RPC property is satisfied.

Regarding the first condition, we have for all $t^{\mathtt{in}} \in [0, n]$ and for all $(t_1^{\mathtt{out}}, t_2^{\mathtt{out}}) \in [0, n]^2$:

$$\begin{aligned}
&\mathbb{P}(|I| = t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}) \\
&= \mathbb{P}(|I_1 \cup I_2| = t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}) \\
&= \sum_{i_1=0}^{t^{\mathtt{in}}} \sum_{i_2=t^{\mathtt{in}}-i_1}^{t^{\mathtt{in}}} \mathbb{P}(|I_1 \cup I_2| = t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}, |I_1| = i_1, |I_2| = i_2) \\
&\qquad \cdot \mathbb{P}(|I_1| = i_1, |I_2| = i_2 \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}})
\end{aligned}$$

The two events $|I_1| = i_1$ and $|I_2| = i_2$ are independent with their respective conditions on the output cardinals, therefore we have:

$$\begin{aligned}
&\mathbb{P}(|I| = t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}) \\
&\leq \sum_{i_1=0}^{t^{\mathtt{in}}} \sum_{i_2=t^{\mathtt{in}}-i_1}^{t^{\mathtt{in}}} \mathbb{P}(|I_1 \cup I_2| = t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}, |I_1| = i_1, |I_2| = i_2) \\
&\qquad \cdot \mathcal{E}^1_{t_1^{\mathtt{out}}}(i_1) \cdot \mathcal{E}^{ui2}_{t_2^{\mathtt{out}}}(i_2)
\end{aligned}$$

Since $G_2$ is $UI$ cardinal RPC, each set of input shares required for the simulation of $G_2$ is equiprobable for equal cardinals. Therefore, we can compute the probability of the cardinality of $|I_1 \cup I_2|$ only from the cardinalities $|I_1|$ and $|I_2|$. Specifically $|I_1 \cap I_2| = |I_1| + |I_2| - |I_1 \cup I_2|$ follows a hypergeometric law. We thus have

$$\mathbb{P}(|I_1 \cap I_2| = i_1 + i_2 - t^{\mathtt{in}} \mid |I_1| = i_1, |I_2| = i_2, |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}) = \frac{\binom{i_1}{i_1+i_2-t^{\mathtt{in}}} \cdot \binom{n-i_1}{t^{\mathtt{in}}-i_1}}{\binom{n}{i_2}}$$

and we obtain

$$\begin{aligned}
&\mathbb{P}(|I| = t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}) \\
&\leq \sum_{i_1=0}^{t^{\mathtt{in}}} \sum_{i_2=t^{\mathtt{in}}-i_1}^{t^{\mathtt{in}}} \mathbb{P}(|I_1 \cap I_2| = i_1 + i_2 - t^{\mathtt{in}} \mid |J_1| = t_1^{\mathtt{out}}, |J_2| = t_2^{\mathtt{out}}, |I_1| = i_1, |I_2| = i_2) \\
&\qquad \cdot \mathcal{E}^1_{t_1^{\mathtt{out}}}(i_1) \cdot \mathcal{E}^{ui2}_{t_2^{\mathtt{out}}}(i_2) \\
&\leq \sum_{i_1=0}^{t^{\mathtt{in}}} \sum_{i_2=t^{i}-i_1}^{t^{\mathtt{in}}} \frac{\binom{i_1}{i_1+i_2-t^{\mathtt{in}}} \cdot \binom{n-i_1}{t^{\mathtt{in}}-i_1}}{\binom{n}{i_2}} \cdot \mathcal{E}^1_{t_1^{\mathtt{out}}}(i_1) \cdot \mathcal{E}^{ui2}_{t_2^{\mathtt{out}}}(i_2)
\end{aligned}$$

which concludes the proof. □

*Proof (Proof of Lemma 5).* We apply [12, Theorem 1] to give us the general RPC envelopes of the parallel composition of both refresh gadget $\mathsf{RPRefresh}$, we note this parallel composition as gadget $G_1$. We note $G_2$ the addition gadget which consists simply in adding the shares of each secret together (i.e. the last loop **for** in Algorithm 6).We compute the general RPC envelopes of $G_2$. Let $(I'_1, I'_2) \subseteq [n]^2$ and $J \subseteq [n]$. Since the outputs shares are not masked by randoms in $G_2$, we need at least the $|J|$ input shares of each secrets leaked by $J$. Then we can complete with the leakage on the input shares. This gives us the general RPC envelopes of $G_2$

$$\boldsymbol{\mathcal{E}}_J(I'_1, I'_2) = \begin{cases} 0 & \text{if } J \not\subset I'_1 \text{ or } J \not\subset I'_2 \\ p^{|I'_1|-|J|} \cdot (1-p)^{n-|I'_1|} \cdot p^{|I'_2|-|J|} \cdot (1-p)^{n-|I'_2|} & \text{otherwise} \end{cases}$$

Then we apply [12, Theorem 2] to give us the general RPC envelopes of the sequential composition of $G_1$ and $G_2$, which gives us the general RPC envelopes of $\mathsf{Add}^\gamma$. □

*Proof (Proof of Lemma 6).* We note $G_2$ the addition gadget which consist simply to multiply the shares by the constant $c$ (i.e. tha last loop **for** in Algorithm 7). We compute the general RPC envelopes of $G_2$ .Let $I_1 \subseteq [n]$ and $J \subseteq [n]$. Since the output shares are not masked by randoms in $G_2$, we need at least the $|J|$ input shares leaked by $J$. Then we can complete with the leakage on the input shares. This gives us the general RPC envelopes of $G_2$

$$\boldsymbol{\mathcal{E}}_J(I_1) = \begin{cases} 0 & \text{if } J \not\subset I_1 \\ p^{|I_1|-|J|}.(1-p)^{n-|I_1|} & \text{otherwise} \end{cases}$$

Then the overall gadget is a sequential composition of the $\mathsf{RPRefresh}$ gadget and the gadget $G_2$. By [12, Theorem 2], we obtain the general RPC envelopes of $\mathsf{cMult}^\gamma$. □

*Proof (Proof of Lemma 7).* First, we establish the general RPC envelopes of the concatenation of permutations $\sigma$. Specifically, by the definition of a uniform random permutation, for all $(I_1, \ldots, I_\ell) \subseteq [n]^\ell$ and $(J_1, \ldots, J_\ell) \subseteq [n]^\ell$, we have

$$\boldsymbol{\mathcal{E}}^\sigma_{J_1,\ldots,J_\ell}(I_1, \ldots, I_\ell) = \frac{1}{\prod_{i=1}^\ell \binom{n}{|I_i|}} \cdot \mathbf{1}_{(|J_1|=|I_1|)\wedge\cdots\wedge(|J_\ell|=|I_\ell|)}.$$

Next, we apply the upper bound for the general RPC envelopes of the sequential composition of two gadgets from [12, Theorem 2]. This gives the following expression for all $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ and $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$:

$$\begin{aligned}\boldsymbol{\mathcal{E}}^{G\circ\sigma}_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell) &= \sum_{I_1^*,\ldots,I_\ell^*\subseteq[n]} \boldsymbol{\mathcal{E}}^{\sigma}_{I_1^*,\ldots,I_\ell^*}(I_1,\ldots,I_\ell)\cdot\boldsymbol{\mathcal{E}}^{G}_{J_1,\ldots,J_m}(I_1^*,\ldots,I_\ell^*)\\ &= \sum_{\substack{I_1^*,\ldots,I_\ell^*\subseteq[n]\\ |I_1^*|=|I_1|,\ldots|I_\ell^*|=|I_\ell|}} \frac{1}{\prod_{i=1}^{\ell}\binom{n}{|I_i|}}\cdot\boldsymbol{\mathcal{E}}^{G}_{J_1,\ldots,J_m}(I_1^*,\ldots,I_\ell^*)\end{aligned}$$

which proves Equation 1. Finally, the expression for $\boldsymbol{\mathcal{E}}^{ui}$ is directly derived from Definition 8. □

*Proof (Proof of Lemma 8).* First, we establish the general RPC envelopes of the concatenation of permutations $\sigma$. Specifically, by the definition of a uniform random permutation, for all $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ and $(J_1, \ldots, J_\ell) \in \mathcal{P}([n])^\ell$, we have

$$\boldsymbol{\mathcal{E}}^{\sigma}_{J_1,\ldots,J_\ell}(I_1,\ldots,I_\ell) = \frac{1}{\prod_{i=1}^{\ell}\binom{n}{|I_i|}}\cdot\mathbf{1}_{(|J_1|=|I_1|)\wedge\cdots\wedge(|J_\ell|=|I_\ell|)}.$$

Next, we apply the upper bound for the general RPC envelopes of the sequential composition of two gadgets from [12, Theorem 2]. This gives the following expression for all $(I_1, \ldots, I_\ell) \in \mathcal{P}([n])^\ell$ and $(J_1, \ldots, J_m) \in \mathcal{P}([n])^m$:

$$\begin{aligned}\boldsymbol{\mathcal{E}}^{\sigma\circ G}_{J_1,\ldots,J_m}(I_1,\ldots,I_\ell) &= \sum_{J_1^*,\ldots,J_m^*\subseteq[n]} \boldsymbol{\mathcal{E}}^{G}_{J_1^*,\ldots,J_m^*}(I_1,\ldots,I_\ell)\cdot\boldsymbol{\mathcal{E}}^{\sigma}_{J_1,\ldots,J_m}(J_1^*,\ldots,J_m^*)\\ &= \sum_{\substack{J_1^*,\ldots,J_m^*\subseteq[n]\\ |J_1^*|=|J_1|,\ldots|J_m^*|=|J_m|}} \boldsymbol{\mathcal{E}}^{G}_{J_1^*,\ldots,J_m^*}(I_1,\ldots,I_\ell)\cdot\frac{1}{\prod_{i=1}^{m}\binom{n}{|J_i|}}\end{aligned}$$

which proves Equation 2. Finally, the expression for $\boldsymbol{\mathcal{E}}^{uo}$ is directly derived from Definition 9. □

*Proof (Proof of Lemma 4).* Let $I, J \subseteq [n]$ denote the sets of required input shares and leaked output shares, respectively. We decompose the set of required shares $I$ into two subsets. The first subset, denoted by $K$, corresponds to the input shares required for the simulation of the output shares $J$ and the leakage of the gadget $\mathsf{RPZeroEnc}^{n,\gamma}_{+}$. The second subset is $I \backslash K$, representing the shares that leak directly at the input. From the RPP (Random Probing Partition) property of the gadget $\mathsf{RPZeroEnc}^{n,\gamma}_{+}$, proven in [8], its output shares can be seen, by the random probing attacker, as divided into independent uniformly random 0-sharings. From the uniform properties of $\mathsf{RPZeroEnc}^{n,\gamma}_{+}$, these 0-sharings are fully determined by a partition $P = (c_1, \ldots, c_\ell)$ of $n$, where each $c_i$ represents the size of a 0-sharing. Given such a partition $P$, we consider tuples $(k_1, \ldots, k_\ell)$ such that $0 \leq k_i \leq c_i$ and:

$$\sum_{i=1}^{\ell} k_i = |J|.$$

These tuples represent the number of shares in each 0-sharing at the output of $\mathsf{RPZeroEnc}^{n,\gamma}_{+}$ that matches the leaked output shares in $J$. Now, in order for some input shares to be required for simulation, they must be associated with fully leaked 0-sharings. That is, for each $i$ such that $k_i = c_i$, all the output shares in that 0-sharing are included in $J$, and the corresponding input shares must be included in $K$. To ensure that exactly $|K|$ such input shares are required, we impose the condition:

$$\sum_{i=1}^{\ell} k_i \cdot \mathbf{1}_{(k_i=c_i)} = |K|.$$

The probability of such a configuration occurring is determined by the envelopes $\boldsymbol{\mathcal{E}}^{\gamma}$ of $\mathsf{RPZeroEnc}_{+}^{n,\gamma}$ and the probability of selecting the tuple $(k_1, \ldots, k_\ell)$ given $P$, $J$, and $K$. This is given by:

$$\frac{\prod_{i=1}^{\ell} \binom{c_i}{k_i}}{\binom{|J|}{|K|} \cdot \binom{n}{|J|}}$$

Finally, the remaining required shares in $I \backslash K$ must leak independently at the input, each with probability $p$. The probability that these shares leak, while the input shares not in $I$ must remain un-leaked, is given by:

$$p^{|I|-|K|} \cdot (1-p)^{n-|I|},$$

which concludes the proof. □

# C Gadgets

## C.1 Linear gadgets from [8] are UO cardinal RPC

**Algorithm 4** $\mathsf{RPZeroEnc}^{\gamma}$ from [8]

**Output :** $[\![z]\!]$ such that $\sum_{i=1}^{n} z_i = 0$
  $[\![z]\!] = (0, \ldots, 0)$
  **for** $i = 1$ **to** $\gamma$ **do**
    $i_1, i_2 \xleftarrow{\$} [1, n]$
    $r \xleftarrow{\$} \mathbb{K}$
    $z_{i_1} \leftarrow z_{i_1} + r$
    $z_{i_2} \leftarrow z_{i_2} - r$
  **end for**
  **return** $[\![z]\!]$

**Algorithm 5** $\mathsf{RPRefresh}^{\gamma}$ from [8]

**Input :** $[\![a]\!] = (a_1, \ldots, a_n) \in \mathbb{K}^n$
**Output :** $[\![s]\!] = (s_1, \ldots s_n) \in \mathbb{K}^n$ such that $\sum_{i=1}^{n} s_i = \sum_{i=1}^{n} a_i$
  $[\![z]\!] \leftarrow \mathsf{RPZeroEnc}^{\gamma}$
  **for** $i = 1$ **to** $n$ **do**
    $s_i \leftarrow a_i + z_i$
  **end for**
  **return** $[\![s]\!]$

**Algorithm 6** $\mathsf{Add}^{\gamma}$

**Input :** $[\![a]\!], [\![b]\!]$ two $n$-sharing.
**Output :** $[\![c]\!]$ a $n$-sharing of $a + b$.
  $[\![a']\!] \leftarrow \mathsf{RPRefresh}^{\gamma}([\![a]\!])$
  $[\![b']\!] \leftarrow \mathsf{RPRefresh}^{\gamma}([\![b]\!])$
  **for** $i = 1$ **to** $n$ **do**
    $c_i \leftarrow a'_i + b'_i$
  **end for**
  **return** $[\![c]\!]$

**Algorithm 7** $\mathsf{cMult}^{\gamma}$

**Input :** $[\![a]\!]$ a $n$-sharing, $c$ a constant.
**Output :** $[\![b]\!]$ a $n$-sharing of $c.a$.
  $[\![a']\!] \leftarrow \mathsf{RPRefresh}^{\gamma}([\![a]\!])$
  **for** $i = 1$ **to** $n$ **do**
    $b_i \leftarrow c.a'_i$
  **end for**
  **return** $[\![b]\!]$

*Proof (Proof of Corollary 2).* Let $J, J' \subseteq [n]$ such that $|J| = |J'|$. Let $(t_1^i, t_2^i) \in [0, n]^2$, we have

$$\sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_J^g(I_1, I_2) = \sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \sum_{I'_1 \subseteq [n]} \sum_{I'_2 \subseteq [n]} \boldsymbol{\mathcal{E}}_{I'_1}^{g_1}(I_1).\boldsymbol{\mathcal{E}}_{I'_2}^{g_1}(I_2).\boldsymbol{\mathcal{E}}_J(I'_1, I'_2)$$

These sums are finite and we can rearrange them

$$\sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_J^g(I_1, I_2) = \sum_{\substack{I'_1 \subseteq [n] \\ J \subseteq I'_1}} \sum_{\substack{I'_2 \subseteq [n] \\ J \subseteq I'_2}} \boldsymbol{\mathcal{E}}_J(I'_1, I'_2)\Big( \sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_{I'_1}^{g_1}(I_1).\boldsymbol{\mathcal{E}}_{I'_2}^{g_1}(I_2)\Big)$$

Now, by the UO cardinal RPC security of $\mathsf{RPRefresh}^{\gamma}$, we know that for $(I_1^*, I_2^*) \subseteq [n]^2$ such that $|I'_1| = |I_1^*|$ and $|I'_2| = |I_2^*|$, we have

$$\sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_{I'_1}^{g_1}(I_1).\boldsymbol{\mathcal{E}}_{I'_2}^{g_1}(I_2) = \sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_{I_1^*}^{g_1}(I_1).\boldsymbol{\mathcal{E}}_{I_2^*}^{g_1}(I_2)$$

Furthermore, if $J' \subseteq I_1^*$ and $J' \subseteq I_2^*$ we have also

$$\boldsymbol{\mathcal{E}}_J(I_1', I_2') = \boldsymbol{\mathcal{E}}_{J'}(I_1^*, I_2^*)$$

Finally, because $|J| = |J'|$, for any cardinality $\ell \in [0, n]$, we have

$$\#\{I' \subseteq [n] | |I'| = \ell \text{ and } J \subseteq I'\} = \#\{I* \subseteq [n] | |I*| = \ell \text{ and } J' \subseteq I*\}$$

Therefore we have

$$\sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_J^g(I_1, I_2) = \sum_{\substack{I_1^* \subseteq [n] \\ J' \subseteq I_1^*}} \sum_{\substack{I_2^* \subseteq [n] \\ J' \subseteq I_2^*}} \boldsymbol{\mathcal{E}}_{J'}(I_1^*, I_2^*)( \sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_{I_1^*}^{g_1}(I_1).\boldsymbol{\mathcal{E}}_{I_2^*}^{g_1}(I_2))$$

Eventually, we obtain the results

$$\sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_J^g(I_1, I_2) = \sum_{\substack{I_1 \subseteq [n] \\ |I_1| = t_1^i}} \sum_{\substack{I_2 \subseteq [n] \\ |I_2| = t_2^i}} \boldsymbol{\mathcal{E}}_{J'}^g(I_1, I_2)$$

Hence, $\mathsf{Add}^\gamma$ is UO cardinal RPC. This conclude the proof. □

*Proof (Proof of Corollary 3).* Let $J, J' \subseteq [n]$ such that $|J| = |J'|$. Let $t_1^i \in [0, n]$, we have

$$\sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_J^g(I) = \sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \sum_{I_1 \subseteq [n]} \boldsymbol{\mathcal{E}}_{I_1}^{g_1}(I).\boldsymbol{\mathcal{E}}_J(I_1)$$

These sums are finite and we can rearrange them

$$\sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_J^g(I) = \sum_{I_1 \subseteq [n]} \boldsymbol{\mathcal{E}}_J(I_1).( \sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_{I_1}^{g_1}(I))$$

Now by the UO cardinal RPC of $\mathsf{RPRefresh}^\gamma$, we know that for $I_1' \subseteq [n]$ such that $|I_1| = |I_1'|$, we have

$$\sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_{I_1}^{g_1}(I) = \sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_{I_1'}^{g_1}(I)$$

Furthermore, if $J \subseteq I_1$ and $J' \subseteq I_1'$, we have also

$$\boldsymbol{\mathcal{E}}_J(I_1) = \boldsymbol{\mathcal{E}}_{J'}(I_1')$$

Finally, because $|J| = |J'|$, for any cardinality $\ell \in [0, n]$, we have

$$\#\{I_1 \subseteq [n] | |I_1| = \ell \text{ and } J \subseteq I_1\} = \#\{I_1' \subseteq [n] | |I_1'| = \ell \text{ and } J' \subseteq I_1'\}$$

Therefore we have

$$\sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_J^g(I) = \sum_{I_1' \subseteq [n]} \boldsymbol{\mathcal{E}}_{J'}(I_1').( \sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_{I_1'}^{g_1}(I))$$

Eventually, we obtain the results

$$\sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_J^g(I) = \sum_{\substack{I \subseteq [n] \\ |I| = t_1^i}} \boldsymbol{\mathcal{E}}_{J'}^g(I)$$

Hence, $\mathsf{cMult}^\gamma$ is UO cardinal RPC. This concludes the proof. □

### C.2 Constant Addition Gadget

**Algorithm 8** $\mathsf{cAdd}^{\gamma}$

**Input :** A $n$-sharing $[\![x]\!] = (x_1, \ldots, x_n)$, a constant $c \in \mathbb{K}$
**Output :** A $n$-sharing $[\![z]\!]$ where $z = x + c$.

$[\![y]\!] \leftarrow \mathsf{RPRefresh}^{\gamma}([\![x]\!])$
$z_1 \leftarrow y_1 + c$
**return** $(z_1, y_2, \ldots, y_n)$

**Algorithm 9** $\mathsf{Sq}^{\gamma}$

**Input :** A $n$-sharing $[\![x]\!]$ of $x$
**Output :** A $n$-sharing $[\![z]\!]$ of $z = x^2$

$[\![y]\!] \leftarrow \mathsf{RPRefresh}^{\gamma}([\![x]\!])$
**for** $i \in [1, n]$ **do**
  $z_i \leftarrow y_i^2$
**end for**
**return** $[\![z]\!]$

The following lemma provides the cardinal RPC security of the $\mathsf{cAdd}^{\gamma}$ gadget.

**Lemma 11.** *Let $n \in \mathbb{N}$. Let $\mathsf{Refresh} : \mathbb{K}^n \to \mathbb{K}^n$ be an $n$-share refresh gadget satisfying $(p, (\boldsymbol{\mathcal{E}}_t^R)_{t\in[0,n]})$-*cardinal RPC *for some collection of envelopes $(\boldsymbol{\mathcal{E}}_t^R)_{t\in[0,n]}$ and some probability $p \in [0, 1]$. Then, $\mathsf{cAdd}^{\gamma}$ is $(p, (\boldsymbol{\mathcal{E}}_t^{cadd})_{t\in[0,n]})$ where for all $t^{\boldsymbol{in}} \in [0, n]$ and all $t^{\boldsymbol{out}} \in [0, n]$ we have*

$$\boldsymbol{\mathcal{E}}_{t^{out}}^{cadd}(t^{in}) = \begin{cases} \boldsymbol{\mathcal{E}}_{t^{out}}^{R}(t^{in}) & \text{if } t^{in} = n \\ \max(p.\boldsymbol{\mathcal{E}}_{t^{out}+1}^{R}(t^{in}) + (1-p).\boldsymbol{\mathcal{E}}_{t^{out}}^{R}(t^{in}), \boldsymbol{\mathcal{E}}_{t^{out}}^{R}(t^{in})) & \text{otherwise} \end{cases}$$

## D Knuth-Yates Shuffling

**Algorithm 10** Knuth-Yates shuffling

**Input :** A $n$-sharing $(x_1, \ldots, x_n)$
**Output :** A uniform permutation $(\sigma(x_1), \ldots, \sigma(x_n))$ of the $n$-sharing of $x$, with $\sigma \xleftarrow{\$} \mathcal{S}_n$

**for** $i \leftarrow n$ down to 2 **do**
  $j \xleftarrow{\$} \{1, \ldots, i\}$
  $t \leftarrow x_i$
  $x_i \leftarrow x_j$
  $x_j \leftarrow t$
**end for**
**return** $(x_1, \ldots, x_n)$

## E 2-Stage Butterfly Network

We now consider a two-stage butterfly network, built on the same gadget. As shown in Figure 15, this circuit takes 4 masked inputs and produces 4 masked outputs, each with $n$ shares. We perform the same tests than on the one-stage butterfly network, but this time without the general RPC notions which does not scale.

The conclusions align with those drawn for the one-stage butterfly network, though the differences between variants (v1 and v3 versus v2 and v4, which include additional refreshes) are more pronounced. Overall, the results further highlight the benefits of using the composition method grounded in the uniformly cardinal RPC framework.

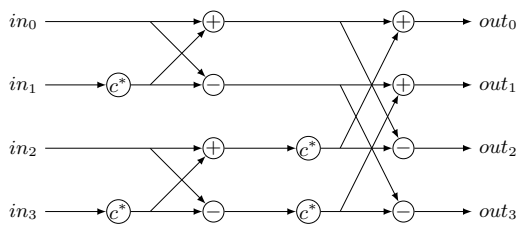
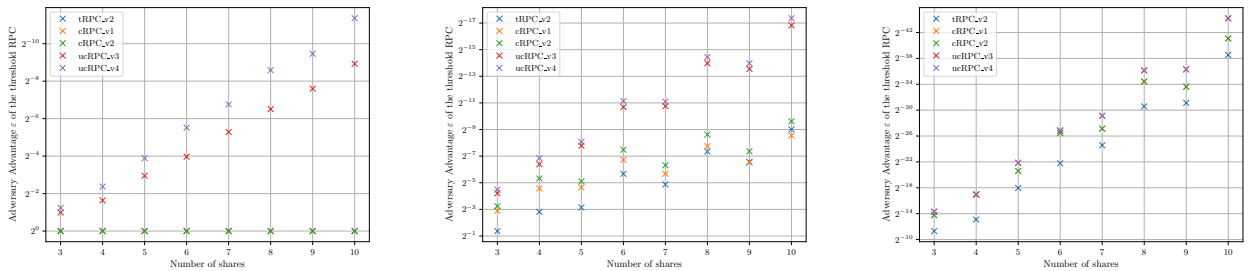



Fig. 15: 2-stages butterfly network.




(a) $p = 2^{-3}$ (b) $p = 2^{-5}$ (c) $p = 2^{-10}$

Fig. 16: Threshold RPC security of 2-stage butterfly network with respect to the number of shares $n$ and the leakage rates $p$ for different composition approaches.

## F Proofs of Section 5

**Lemma 12.** *Let $n_x, n_y \in \mathbb{N}$. For any $n \in \mathbb{N}$, let $\mathsf{G}_{\mathsf{ref}} : \mathbb{K}^n \to \mathbb{K}^n$ be an $n$-share refresh gadget satisfying $(p, (\mathcal{E}_t^{\mathsf{G}_{\mathsf{ref}}(n)})_{t\in[0,n]})$-cardinal RPC for some collection of envelopes $(\mathcal{E}_t^{\mathsf{G}_{\mathsf{ref}}(n)})_{t\in[0,n]}$ and some probability $p \in [0,1]$. Then, gadget $\mathsf{MatMultRef} : \mathbb{K}^{n_x} \times \mathbb{K}^{n_y} \to \mathbb{K}^{n_x.n_y}$ is $(p, (\mathcal{E}_t^{MMR(n_x,n_y)})_{t\in[0,n_x.n_y]})$-cardinal RPC with an envelope that can be computed by recurrence.*

*Proof (Proof of Lemma 12).* Using the result [8, Lemma 4], we can compute the envelope of $\mathsf{MatMultRef}(n_x, n_y)$ by recurrence. In fact we compose the cardinal RPC envelopes $\boldsymbol{\mathcal{E}}^{\mathsf{G}_{\mathsf{ref}}}$ of the refresh gadget $\mathsf{G}_{\mathsf{ref}}$ and the cardinal RPC envelopes $\boldsymbol{\mathcal{E}}^{\mathsf{MMR}}$ of $\mathsf{MatMultRef}(m_1, m_2)$, where $m_1 \in \{\lfloor \frac{n_x}{2} \rfloor, \lceil \frac{n_x}{2} \rceil\}$ and $m_2 \in \{\lfloor \frac{n_y}{2} \rfloor, \lceil \frac{n_y}{2} \rceil\}$. We terminate with the base cases $\mathsf{MatMultRef}(m_1, m_2)$, where $m_1 \in \{1, 2\}$ and $m_2 \in \{1, 2\}$ for which we can assess the cardinal RPC envelopes manually. Throughout the proof, we have

$$0 \le j_1 \le \left\lfloor \frac{n_x}{2} \right\rfloor . \left\lfloor \frac{n_y}{2} \right\rfloor \qquad 0 \le j_2 \le \left\lceil \frac{n_x}{2} \right\rceil . \left\lfloor \frac{n_y}{2} \right\rfloor$$
$$0 \le j_3 \le \left\lfloor \frac{n_x}{2} \right\rfloor . \left\lceil \frac{n_y}{2} \right\rceil \qquad 0 \le j_4 \le \left\lceil \frac{n_x}{2} \right\rceil . \left\lceil \frac{n_y}{2} \right\rceil$$

We develop the formulas hereafter. For all $t^{\mathtt{out}} \in [0, n_x.n_y]$ we have:

$$\forall \;\; 0 \le t^{\mathtt{in,x}} < \left\lfloor \frac{n_x}{2} \right\rfloor, 0 \le t^{\mathtt{in,y}} < \left\lfloor \frac{n_y}{2} \right\rfloor, \quad \mathcal{E}_{t^{\mathtt{out}}}^{\mathsf{MMR}(n_x,n_y)}(t^{\mathtt{in,x}}, t^{\mathtt{in,y}}) =$$
$$\max_{\substack{j_1,j_2,j_3,j_4 \\ j_1+j_2+j_3+j_4=t^{\mathtt{out}}}} \sum_{i^{x_1}=0}^{t^{\mathtt{in,x}}} \sum_{i_1^{x_1}=0}^{i^{x_1}} \sum_{i_1^{x_2}=0}^{t^{\mathtt{in,x}}-i^{x_1}} \sum_{i^{y_1}=0}^{t^{\mathtt{in,y}}} \sum_{i_1^{y_1}=0}^{i^{y_1}} \sum_{i_1^{y_2}=0}^{t^{\mathtt{in,y}}-i^{y_1}} \mathcal{E}_{j_1}^{\mathsf{MMR}(\lfloor \frac{n_x}{2} \rfloor, \lfloor \frac{n_y}{2} \rfloor)}(i_1^{x_1}, i_1^{y_1})$$
$$\cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1} - i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1} - i_1^{x_1}, i_1^{y_2}) \cdot \mathcal{E}_{j_4}^{G_3}(t^{\mathtt{in,x}} - i^{x_1} - i_1^{x_2}, t^{\mathtt{in,y}} - i^{y_1} - i_1^{y_2})$$

$$\forall \quad \left\lfloor \frac{n_x}{2} \right\rfloor \leq t^{\mathtt{in,x}} \leq n_x, 0 \leq t^{\mathtt{in,y}} < \left\lfloor \frac{n_y}{2} \right\rfloor, \quad \mathcal{E}_{t^{\mathtt{out}}}^{\mathsf{MMR}(n_x,n_y)}(t^{\mathtt{in,x}}, t^{\mathtt{in,y}}) =$$

$$\max_{\substack{j_1,j_2,j_3,j_4 \\ j_1+j_2+j_3+j_4=t^{\mathtt{out}}}} \sum_{\substack{i^{x_1}=t^{\mathtt{in,x}}-\lfloor \frac{n_x}{2} \rfloor \\ t^{\mathtt{in,x}}-\min(i^{x_1},\lfloor \frac{n_x}{2} \rfloor)<\lceil \frac{n_x}{2} \rceil}}^{n_x} \sum_{i_1^{x_1}=\max(0,i^{x_1}-\lfloor \frac{n_x}{2} \rfloor)}^{\min(i^{x_1},\lfloor \frac{n_x}{2} \rfloor)} \sum_{i_1^{x_2}=0}^{t^{\mathtt{in,x}}-\min(i^{x_1},\lfloor \frac{n_x}{2} \rfloor)}$$

$$\sum_{i^{y_1}=0}^{t^{\mathtt{in,y}}} \sum_{i_1^{y_1}=0}^{i^{y_1}} \sum_{i_1^{y_2}=0}^{t^{\mathtt{in,y}}-i^{y_1}} \mathcal{E}_{j_1}^{\mathsf{MMR}(\lfloor \frac{n_x}{2} \rfloor,\lfloor \frac{n_y}{2} \rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1}-i_1^{y_1}) \cdot$$

$$\mathcal{E}_{j_3}^{G_2}(i^{x_1}-i_1^{x_1}, i_1^{y_2}) \cdot \mathcal{E}_{j_4}^{G_3}(t^{\mathtt{in,x}}-\min(i^{x_1}, \left\lfloor \frac{n_x}{2} \right\rfloor)-i_1^{x_2}, t^{\mathtt{in,y}}-i^{y_1}-i_1^{y_2})$$

$$+ \sum_{\substack{i^{x_1}=t^{\mathtt{in,x}}-\lfloor \frac{n_x}{2} \rfloor \\ t^{\mathtt{in,x}}-\min(i^{x_1},\lfloor \frac{n_x}{2} \rfloor)\geq\lceil \frac{n_x}{2} \rceil}}^{n_x} \sum_{i_1^{x_1}=\max(0,i^{x_1}-\lfloor \frac{n_x}{2} \rfloor)}^{\min(i^{x_1},\lfloor \frac{n_x}{2} \rfloor)} \sum_{i^{x_2}=\lceil \frac{n_x}{2} \rceil}^{n_x} \sum_{i_1^{x_2}=i^{x_2}-\lceil \frac{n_x}{2} \rceil}^{\lceil \frac{n_x}{2} \rceil}$$

$$\sum_{i^{y_1}=0}^{t^{\mathtt{in,y}}} \sum_{i_1^{y_1}=0}^{i^{y_1}} \sum_{i_1^{y_2}=0}^{t^{\mathtt{in,y}}-i^{y_1}} \mathcal{E}_{j_1}^{\mathsf{MMR}(\lfloor \frac{n_x}{2} \rfloor,\lfloor \frac{n_y}{2} \rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1}-i_1^{y_1}) \cdot$$

$$\mathcal{E}_{j_3}^{G_2}(i^{x_1}-i_1^{x_1}, i_1^{y_2}) \cdot \mathcal{E}_{j_4}^{G_3}(i^{x_2}-i_1^{x_2}, t^{\mathtt{in,y}}-i^{y_1}-i_1^{y_2})$$

$$\forall \quad 0 \leq t^{\mathtt{in,x}} < \left\lfloor \frac{n_x}{2} \right\rfloor, \left\lfloor \frac{n_y}{2} \right\rfloor \leq t^{\mathtt{in,y}} \leq n_y, \quad \mathcal{E}_{t^{\mathtt{out}}}^{\mathsf{MMR}(n_x,n_y)}(t^{\mathtt{in,x}}, t^{\mathtt{in,y}}) =$$

$$\max_{\substack{j_1,j_2,j_3,j_4 \\ j_1+j_2+j_3+j_4=t^{\mathtt{out}}}} \sum_{i^{x_1}=0}^{t^{\mathtt{in,x}}} \sum_{i_1^{x_1}=0}^{i^{x_1}} \sum_{i_1^{x_2}=0}^{t^{\mathtt{in,x}}-i^{x_1}}$$

$$\sum_{\substack{i^{y_1}=t^{\mathtt{in,y}}-\lfloor \frac{n_y}{2} \rfloor \\ t^{\mathtt{in,y}}-\min(i^{y_1},\lfloor \frac{n_y}{2} \rfloor)<\lceil \frac{n_y}{2} \rceil}}^{n_y} \sum_{i_1^{y_1}=\max(0,i^{y_1}-\lfloor \frac{n_y}{2} \rfloor)}^{\min(i^{y_1},\lfloor \frac{n_y}{2} \rfloor)} \sum_{i_1^{y_2}=0}^{t^{\mathtt{in,y}}-\min(i^{y_1},\lfloor \frac{n_y}{2} \rfloor)}$$

$$\mathcal{E}_{j_1}^{\mathsf{MMR}(\lfloor \frac{n_x}{2} \rfloor,\lfloor \frac{n_y}{2} \rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1}-i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1}-i_1^{x_1}, i_1^{y_2})$$

$$\cdot \mathcal{E}_{j_4}^{G_3}(t^{\mathtt{in,x}}-i^{x_1}-i_1^{x_2}, t^{\mathtt{in,y}}-\min(i^{y_1}, \left\lfloor \frac{n_y}{2} \right\rfloor)-i_1^{y_2})$$

$$+ \sum_{i^{x_1}=0}^{t^{\mathtt{in,x}}} \sum_{i_1^{x_1}=0}^{i^{x_1}} \sum_{i_1^{x_2}=0}^{t^{\mathtt{in,x}}-i^{x_1}}$$

$$\sum_{\substack{i^{y_1}=t^{\mathtt{in,y}}-\lfloor \frac{n_y}{2} \rfloor \\ t^{\mathtt{in,y}}-\min(i^{y_1},\lfloor \frac{n_y}{2} \rfloor)\geq\lceil \frac{n_y}{2} \rceil}}^{n_y} \sum_{i_1^{y_1}=\max(0,i^{y_1}-\lfloor \frac{n_y}{2} \rfloor)}^{\min(i^{y_1},\lfloor \frac{n_y}{2} \rfloor)} \sum_{i^{y_2}=\lceil \frac{n_y}{2} \rceil}^{n_y} \sum_{i_1^{y_2}=i^{y_2}-\lceil \frac{n_y}{2} \rceil}^{\lceil \frac{n_y}{2} \rceil}$$

$$\mathcal{E}_{j_1}^{\mathsf{MMR}(\lfloor \frac{n_x}{2} \rfloor,\lfloor \frac{n_y}{2} \rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1}-i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1}-i_1^{x_1}, i_1^{y_2})$$

$$\cdot \mathcal{E}_{j_4}^{G_3}(t^{\mathtt{in,x}}-i^{x_1}-i_1^{x_2}, i^{y_2}-i_1^{y_2})$$

$$\forall \quad \left\lfloor \frac{n_x}{2} \right\rfloor \le t^{\texttt{in,x}} \le n_x, \left\lfloor \frac{n_y}{2} \right\rfloor \le t^{\texttt{in,y}} \le n_y, \quad \mathcal{E}_{t^{\texttt{out}}}^{\mathsf{MMR}(n_x,n_y)}(t^{\texttt{in,x}}, t^{\texttt{in,y}}) =$$

$$\max_{\substack{j_1,j_2,j_3,j_4 \\ j_1+j_2+j_3+j_4=t^{\texttt{out}}}} \sum_{\substack{i^{x_1}=t^{\texttt{in,x}}-\left\lfloor \frac{n_x}{2} \right\rfloor \\ t^{\texttt{in,x}}-\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)<\left\lceil \frac{n_x}{2} \right\rceil}}^{n_x} \sum_{i_1^{x_1}=\max(0,i^{x_1}-\left\lfloor \frac{n_x}{2} \right\rfloor)}^{\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)} \sum_{i_1^{x_2}=0}^{t^{\texttt{in,x}}-\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)}$$

$$\sum_{\substack{i^{y_1}=t^{\texttt{in,y}}-\left\lfloor \frac{n_y}{2} \right\rfloor \\ t^{\texttt{in,y}}-\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)<\left\lceil \frac{n_y}{2} \right\rceil}}^{n_y} \sum_{i_1^{y_1}=\max(0,i^{y_1}-\left\lfloor \frac{n_y}{2} \right\rfloor)}^{\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)} \sum_{i_1^{y_2}=0}^{t^{\texttt{in,y}}-\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)}$$

$$\mathcal{E}_{j_1}^{\mathsf{MMR}(\left\lfloor \frac{n_x}{2} \right\rfloor,\left\lfloor \frac{n_y}{2} \right\rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1} - i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1} - i_1^{x_1}, i_1^{y_2})$$

$$\cdot \mathcal{E}_{j_4}^{G_3}(t^{\texttt{in,x}} - \min(i^{x_1}, \left\lfloor \frac{n_x}{2} \right\rfloor) - i_1^{x_2}, t^{\texttt{in,y}} - \min(i^{y_1}, \left\lfloor \frac{n_y}{2} \right\rfloor) - i_1^{y_2})$$

$$+ \sum_{\substack{i^{x_1}=t^{\texttt{in,x}}-\left\lfloor \frac{n_x}{2} \right\rfloor \\ t^{\texttt{in,x}}-\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)<\left\lceil \frac{n_x}{2} \right\rceil}}^{n_x} \sum_{i_1^{x_1}=\max(0,i^{x_1}-\left\lfloor \frac{n_x}{2} \right\rfloor)}^{\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)} \sum_{i_1^{x_2}=0}^{t^{\texttt{in,x}}-\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)}$$

$$\sum_{\substack{i^{y_1}=t^{\texttt{in,y}}-\left\lfloor \frac{n_y}{2} \right\rfloor \\ t^{\texttt{in,y}}-\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)\ge\left\lceil \frac{n_y}{2} \right\rceil}}^{n_y} \sum_{i_1^{y_1}=\max(0,i^{y_1}-\left\lfloor \frac{n_y}{2} \right\rfloor)}^{\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)} \sum_{i^{y_2}=\left\lceil \frac{n_y}{2} \right\rceil}^{n_y} \sum_{i_1^{y_2}=i^{y_2}-\left\lceil \frac{n_y}{2} \right\rceil}^{\left\lceil \frac{n_y}{2} \right\rceil}$$

$$\mathcal{E}_{j_1}^{\mathsf{MMR}(\left\lfloor \frac{n_x}{2} \right\rfloor,\left\lfloor \frac{n_y}{2} \right\rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1} - i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1} - i_1^{x_1}, i_1^{y_2})$$

$$\cdot \mathcal{E}_{j_4}^{G_3}(t^{\texttt{in,x}} - \min(i^{x_1}, \left\lfloor \frac{n_x}{2} \right\rfloor) - i_1^{x_2}, i^{y_2} - i_1^{y_2})$$

$$+ \sum_{\substack{i^{x_1}=t^{\texttt{in,x}}-\left\lfloor \frac{n_x}{2} \right\rfloor \\ t^{\texttt{in,x}}-\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)\ge\left\lceil \frac{n_x}{2} \right\rceil}}^{n_x} \sum_{i_1^{x_1}=\max(0,i^{x_1}-\left\lfloor \frac{n_x}{2} \right\rfloor)}^{\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)} \sum_{i^{x_2}=\left\lceil \frac{n_x}{2} \right\rceil}^{n_x} \sum_{i_1^{x_2}=i^{x_2}-\left\lceil \frac{n_x}{2} \right\rceil}^{\left\lceil \frac{n_x}{2} \right\rceil}$$

$$\sum_{\substack{i^{y_1}=t^{\texttt{in,y}}-\left\lfloor \frac{n_y}{2} \right\rfloor \\ t^{\texttt{in,y}}-\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)<\left\lceil \frac{n_y}{2} \right\rceil}}^{n_y} \sum_{i_1^{y_1}=\max(0,i^{y_1}-\left\lfloor \frac{n_y}{2} \right\rfloor)}^{\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)} \sum_{i_1^{y_2}=0}^{t^{\texttt{in,y}}-\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)}$$

$$\mathcal{E}_{j_1}^{\mathsf{MMR}(\left\lfloor \frac{n_x}{2} \right\rfloor,\left\lfloor \frac{n_y}{2} \right\rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1} - i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1} - i_1^{x_1}, i_1^{y_2})$$

$$\cdot \mathcal{E}_{j_4}^{G_3}(i^{x_2} - i_1^{x_2}, t^{\texttt{in,y}} - \min(i^{y_1}, \left\lfloor \frac{n_y}{2} \right\rfloor) - i_1^{y_2})$$

$$+ \sum_{\substack{i^{x_1}=t^{\texttt{in,x}}-\left\lfloor \frac{n_x}{2} \right\rfloor \\ t^{\texttt{in,x}}-\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)\ge\left\lceil \frac{n_x}{2} \right\rceil}}^{n_x} \sum_{i_1^{x_1}=\max(0,i^{x_1}-\left\lfloor \frac{n_x}{2} \right\rfloor)}^{\min(i^{x_1},\left\lfloor \frac{n_x}{2} \right\rfloor)} \sum_{i^{x_2}=\left\lceil \frac{n_x}{2} \right\rceil}^{n_x} \sum_{i_1^{x_2}=i^{x_2}-\left\lceil \frac{n_x}{2} \right\rceil}^{\left\lceil \frac{n_x}{2} \right\rceil}$$

$$\sum_{\substack{i^{y_1}=t^{\texttt{in,y}}-\left\lfloor \frac{n_y}{2} \right\rfloor \\ t^{\texttt{in,y}}-\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)\ge\left\lceil \frac{n_y}{2} \right\rceil}}^{n_y} \sum_{i_1^{y_1}=\max(0,i^{y_1}-\left\lfloor \frac{n_y}{2} \right\rfloor)}^{\min(i^{y_1},\left\lfloor \frac{n_y}{2} \right\rfloor)} \sum_{i^{y_2}=\left\lceil \frac{n_y}{2} \right\rceil}^{n_y} \sum_{i_1^{y_2}=i^{y_2}-\left\lceil \frac{n_y}{2} \right\rceil}^{\left\lceil \frac{n_y}{2} \right\rceil}$$

$$\mathcal{E}_{j_1}^{\mathsf{MMR}(\left\lfloor \frac{n_x}{2} \right\rfloor,\left\lfloor \frac{n_y}{2} \right\rfloor)}(i_1^{x_1}, i_1^{y_1}) \cdot \mathcal{E}_{j_2}^{G_1}(i_1^{x_2}, i^{y_1} - i_1^{y_1}) \cdot \mathcal{E}_{j_3}^{G_2}(i^{x_1} - i_1^{x_1}, i_1^{y_2})$$

$$\cdot \mathcal{E}_{j_4}^{G_3}(i^{x_2} - i_1^{x_2}, i^{y_2} - i_1^{y_2})$$

where the envelopes $\mathcal{E}^{G_1}$, $\mathcal{E}^{G_2}$ and $\mathcal{E}^{G_3}$ are on the form:

$$\mathcal{E}^{G_1}_{j_2}(i_1^{x_2}, i_2^{y_1}) = \sum_{\ell=0}^{\lfloor \frac{n_y}{2} \rfloor} \mathcal{E}^{\mathsf{MMR}(\lceil \frac{n_x}{2} \rceil, \lfloor \frac{n_y}{2} \rfloor)}_{j_2}(i_1^{x_2}, \ell).\mathcal{E}^{\mathsf{G_{ref}}(\lfloor \frac{n_y}{2} \rfloor)}_{\ell}(i_2^{y_1})$$

$$\mathcal{E}^{G_2}_{j_3}(i_2^{x_1}, i_1^{y_2}) = \sum_{\ell=0}^{\lfloor \frac{n_x}{2} \rfloor} \mathcal{E}^{\mathsf{MMR}(\lfloor \frac{n_x}{2} \rfloor, \lceil \frac{n_y}{2} \rceil)}_{j_3}(\ell, i_1^{y_2}).\mathcal{E}^{\mathsf{G_{ref}}(\lfloor \frac{n_x}{2} \rfloor)}_{\ell}(i_2^{x_1})$$

$$\mathcal{E}^{G_3}_{j_4}(i_2^{x_2}, i_2^{y_2}) = \sum_{\ell_1=0}^{\lceil \frac{n_x}{2} \rceil} \sum_{\ell_2=0}^{\lceil \frac{n_y}{2} \rceil} \mathcal{E}^{\mathsf{MMR}(\lceil \frac{n_x}{2} \rceil, \lceil \frac{n_y}{2} \rceil)}_{j_4}(\ell_1, \ell_2).\mathcal{E}^{\mathsf{G_{ref}}(\lceil \frac{n_x}{2} \rceil)}_{\ell_1}(i_2^{x_2}).\mathcal{E}^{\mathsf{G_{ref}}(\lceil \frac{n_y}{2} \rceil)}_{\ell_2}(i_2^{y_2})$$

for any $i_2^{x_1} \in [0, \lfloor \frac{n_x}{2} \rfloor]$ , $i_2^{y_1} \in [0, \lfloor \frac{n_y}{2} \rfloor]$, $(i_1^{x_2}, i_2^{x_2}) \in [0, \lceil \frac{n_x}{2} \rceil]$, $(i_1^{y_2}, i_2^{y_2}) \in [0, \lceil \frac{n_y}{2} \rceil]$. *Initialisation.* We compute manually the cardinal RPC envelopes of $\mathsf{MatMultRef}(n_x, n_y)$ for $n_x \in \{1, 2\}, n_y \in \{1, 2\}$. We start with $n_x = 1$ and $n_y = 1$.

For $t^{\mathtt{out}} = 0$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 |
|---|---|---|
| 0 | $(1-p)^2$ | $p.(1-p)$ |
| 1 | $p.(1-p)$ | $p^2$ |

For $t^{\mathtt{out}} = 1$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |

For $n_x = 1$ and $n_y = 2$, we have :

For $t^{\mathtt{out}} = 0$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | $(1-p)^3$ | $2.p.(1-p)^2$ | $p^2.(1-p)$ |
| 1 | $p.(1-p)^2$ | $2.p^2.(1-p)$ | $p^3$ |

For $t^{\mathtt{out}} = 1$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | $1-p$ | $p$ |

For $t^{\mathtt{out}} = 2$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |

For $n_x = 2$ and $n_y = 1$, the array is the transpose of the above array. Eventually, for $n_x = 2$ and $n_y = 2$, if we note $v = (1-p)^3$ and $w = 1 - v$, we have :

For $t^{\mathtt{out}} = 0$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | $v^4$ | $2.w.v^3$ | $v^2.w^2$ |
| 1 | $2.w.v^3$ | $4.w^2.v^2$ | $2.w^3.v$ |
| 2 | $w^2.v^2$ | $2.w^3.v$ | $w^4$ |

For $t^{\mathtt{out}} = 1$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | $v^2$ | $w.v$ |
| 2 | 0 | $w.v$ | $w^2$ |

For $t^{\mathtt{out}} = 2$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | $v$ |
| 2 | 0 | $v$ | 1 |

For $t^{\mathtt{out}} = 3$ or $t^{\mathtt{out}} = 4$ :

| $t^{\mathtt{in,x}} \backslash t^{\mathtt{in,y}}$ | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 |
| 2 | 0 | 0 | 1 |

□

MatMultSym is formally described in Algorithm 11.

**Algorithm 11** MatMultSym

**Input:** $n_x$-sharing $[\![x]\!]$ and $n_y$-sharing $[\![y]\!]$
**Output:** $(n_x.n_y)$-sharing $\mathbf{M} = (M_{i,j})_{\substack{1\leq i\leq n_x\\1\leq j\leq n_y}}$ such that $\sum_{\substack{1\leq i\leq n_x\\1\leq j\leq n_y}} M_{i,j} = x \cdot y$

**if** $n_x = 1$ and $n_y = 1$ **then**
  $\mathbf{M} \leftarrow (x_1 \cdot y_1)$
**else if** $n_x = 1$ and $n_y = 2$ **then**
  $\mathbf{M} \leftarrow \begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 \end{pmatrix}$
**else if** $n_x = 2$ and $n_y = 1$ **then**
  $\mathbf{M} \leftarrow \begin{pmatrix} x_1 \cdot y_1 \\ x_2 \cdot y_1 \end{pmatrix}$
**else if** $n_x = 2$ and $n_y = 2$ **then**
  $\mathbf{M} \leftarrow \begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 \\ x_2 \cdot y_1 & x_2 \cdot y_2 \end{pmatrix}$
**else**
  $\mathbf{x}^{(1)} \leftarrow (x_1, \ldots, x_{\lfloor \frac{n_x}{2} \rfloor}), \quad \mathbf{x}^{(2)} \leftarrow (x_{\lceil \frac{n_x}{2} \rceil}, \ldots, x_{n_x})$
  $\mathbf{y}^{(1)} \leftarrow (y_1, \ldots, y_{\lfloor \frac{n_y}{2} \rfloor}), \quad \mathbf{y}^{(2)} \leftarrow (y_{\lceil \frac{n_y}{2} \rceil}, \ldots, y_{n_y})$
  $\mathbf{x}_1^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}^{(1)}), \quad \mathbf{x}_1^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}^{(2)})$
  $\mathbf{y}_1^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}^{(1)}), \quad \mathbf{y}_1^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}^{(2)})$
  $\mathbf{M}^{(1,1)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_1^{(1)}, \mathbf{y}_1^{(1)})$
  $\mathbf{x}_2^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}^{(1)}), \quad \mathbf{y}_2^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}^{(1)})$
  $\mathbf{M}^{(1,2)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_2^{(1)}, \mathbf{y}_1^{(2)})$
  $\mathbf{M}^{(2,1)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_1^{(2)}, \mathbf{y}_2^{(1)})$
  $\mathbf{x}_2^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}^{(2)}), \quad \mathbf{y}_2^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}^{(2)})$
  $\mathbf{M}^{(2,2)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_2^{(2)}, \mathbf{y}_2^{(2)})$
  $\mathbf{M} = \begin{pmatrix} \mathbf{M}^{(1,1)} & \mathbf{M}^{(1,2)} \\ \mathbf{M}^{(2,1)} & \mathbf{M}^{(2,2)} \end{pmatrix}$
**end if**
**return M**

**Lemma 13.** *Let $n_x, n_y \in \mathbb{N}$. For any $n \in \mathbb{N}$, let $\mathsf{G}_{\mathsf{ref}} : \mathbb{K}^n \to \mathbb{K}^n$ be an $n$-share refresh gadget satisfying $(p, (\mathcal{E}_t^{\mathsf{G}_{\mathsf{ref}}(n)})_{t\in[0,n]})$-cardinal RPC for some collection of envelopes $(\mathcal{E}_t^{\mathsf{G}_{\mathsf{ref}}(n)})_{t\in[0,n]}$ and some probability $p \in [0,1]$. Then, gadget $\mathsf{MatMultSym} : \mathbb{K}^{n_x} \times \mathbb{K}^{n_y} \to \mathbb{K}^{n_x.n_y}$ is $(p, (\mathcal{E}_t^{\mathsf{MMS}(n_x,n_y)})_{t\in[0,n_x\cdot n_y]})$-cardinal RPC with an envelope that can be computed by recurrence.*

*Proof (Proof of Lemma 13).* The proof is similar to the proof of Lemma 12. The envelopes of the base case with $n_x \in \{1,2\}$ and $n_y \in \{1,2\}$ are exactly the same. As for the induction step, the only differences is that we replace $\boldsymbol{\mathcal{E}}_{t_1^{\mathsf{out}}}^{\mathsf{MMR}}, \boldsymbol{\mathcal{E}}_{t_2^{\mathsf{out}}}^{G_1}, \boldsymbol{\mathcal{E}}_{t_3^{\mathsf{out}}}^{G_2}, \boldsymbol{\mathcal{E}}_{t_4^{\mathsf{out}}}^{G_3}$ by $\boldsymbol{\mathcal{E}}_{t^{\mathsf{out}}}^{G,(m_1,m_2)}$ where $m_1 \in \{\lfloor \frac{n_x}{2} \rfloor, \lceil \frac{n_x}{2} \rceil\}$ and $m_2 \in \{\lfloor \frac{n_y}{2} \rfloor, \lceil \frac{n_y}{2} \rceil\}$ (according to the considered case) where for all $t_1^{\mathtt{in}} \in [0, m_1], t_2^{\mathtt{in}} \in [0, m_2]$ we have

$$\mathcal{E}_{t^{\mathsf{out}}}^{G,(m_1,m_2)}(t_1^{\mathtt{in}}, t_2^{\mathtt{in}}) = \sum_{\ell_1=0}^{m_1} \sum_{\ell_2=0}^{m_2} \mathcal{E}_{t^{\mathsf{out}}}^{\mathsf{MMR}(m_1,m_2)}(\ell_1, \ell_2).\mathcal{E}_{\ell_1}^{\mathsf{G}_{\mathsf{ref}}(m_1)}(t_1^{\mathtt{in}}).\mathcal{E}_{\ell_2}^{\mathsf{G}_{\mathsf{ref}}(m_2)}(t_2^{\mathtt{in}})$$

□

UnifMatMult is formally described in Algorithm 12.

**Algorithm 12** UnifMatMult

**Input:** $n_x$-sharing $[\![x]\!]$ and $n_y$-sharing $[\![y]\!]$
**Output:** $(n_x.n_y)$-sharing $\mathbf{M} = (M_{i,j})_{\substack{1\le i\le n_x\\ 1\le j\le n_y}}$ such that $\sum_{\substack{1\le i\le n_x\\ 1\le j\le n_y}} M_{i,j} = x \cdot y$

**if** $n_x = 1$ and $n_y = 1$ **then**
  $\mathbf{M} \leftarrow (x_1 \cdot y_1)$
**else if** $n_x = 1$ and $n_y = 2$ **then**
  $\mathbf{M} \leftarrow \begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 \end{pmatrix}$
**else if** $n_x = 2$ and $n_y = 1$ **then**
  $\mathbf{M} \leftarrow \begin{pmatrix} x_1 \cdot y_1 \\ x_2 \cdot y_1 \end{pmatrix}$
**else if** $n_x = 2$ and $n_y = 2$ **then**
  $\mathbf{M} \leftarrow \begin{pmatrix} x_1 \cdot y_1 & x_1 \cdot y_2 \\ x_2 \cdot y_1 & x_2 \cdot y_2 \end{pmatrix}$
**else**
  $\mathbf{x}^{(1)} \leftarrow (x_1, \ldots, x_{\lfloor \frac{n_x}{2} \rfloor}), \quad \mathbf{x}^{(2)} \leftarrow (x_{\lceil \frac{n_x}{2} \rceil}, \ldots, x_{n_x})$
  $\mathbf{y}^{(1)} \leftarrow (y_1, \ldots, y_{\lfloor \frac{n_y}{2} \rfloor}), \quad \mathbf{y}^{(2)} \leftarrow (y_{\lceil \frac{n_y}{2} \rceil}, \ldots, y_{n_y})$
  $\sigma_1, \sigma_2, \sigma_3, \sigma_4 \xleftarrow{\$} \mathcal{S}_*$
  $\mathbf{x}_1^{(1)} \leftarrow \sigma_1(\mathbf{x}^{(1)}), \mathbf{x}_1^{(2)} \leftarrow \sigma_2(\mathbf{x}^{(2)})$
  $\mathbf{y}_1^{(1)} \leftarrow \sigma_3(\mathbf{y}^{(1)}), \mathbf{y}_1^{(2)} \leftarrow \sigma_4(\mathbf{y}^{(2)})$
  $\mathbf{x}_1^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}_1^{(1)}), \quad \mathbf{x}_1^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}_1^{(2)})$
  $\mathbf{y}_1^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}_1^{(1)}), \quad \mathbf{y}_1^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}_1^{(2)})$
  $\mathbf{M}^{(1,1)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_1^{(1)}, \mathbf{y}_1^{(1)})$
  $\mathbf{x}_2^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}^{(1)}), \quad \mathbf{y}_2^{(1)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}^{(1)})$
  $\mathbf{M}^{(1,2)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_2^{(1)}, \mathbf{y}_1^{(2)})$
  $\mathbf{M}^{(2,1)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_1^{(2)}, \mathbf{y}_2^{(1)})$
  $\mathbf{x}_2^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{x}^{(2)}), \quad \mathbf{y}_2^{(2)} \leftarrow \mathsf{RPRefresh}(\mathbf{y}^{(2)})$
  $\mathbf{M}^{(2,2)} \leftarrow \mathsf{UnifMatMult}(\mathbf{x}_2^{(2)}, \mathbf{y}_2^{(2)})$
  $\mathbf{M} = \begin{pmatrix} \mathbf{M}^{(1,1)} & \mathbf{M}^{(1,2)} \\ \mathbf{M}^{(2,1)} & \mathbf{M}^{(2,2)} \end{pmatrix}$
**end if**
**return M**

**Lemma 14.** *Let $n_x, n_y \in \mathbb{N}$. For any $n \in \mathbb{N}$, let $\mathsf{G}_{\mathsf{ref}} : \mathbb{K}^n \to \mathbb{K}^n$ be an $n$-share refresh gadget satisfying $(p, (\mathcal{E}_t^{\mathsf{G}_{\mathsf{ref}}(n)})_{t\in[0,n]})$-cardinal RPC for some collection of envelopes $(\mathcal{E}_t^{\mathsf{G}_{\mathsf{ref}}(n)})_{t\in[0,n]}$ and some probability $p \in [0,1]$. Then, gadget $\mathsf{UnifMatMult} : \mathbb{K}^{n_x} \times \mathbb{K}^{n_y} \to \mathbb{K}^{n_x \cdot n_y}$ is $(p, (\mathcal{E}_t^{\mathsf{UMM}(n_x,n_y)})_{t\in[0,n_x\cdot n_y]})$-cardinal RPC with an envelope that can be computed by recurrence.*

*Proof (Proof of Lemma 14).* Using the result [8, Lemma 4], we can compute the envelope of $\mathsf{UnifMatMult}(n_x, n_y)$ by recurrence. In fact we compose the cardinal RPC envelopes $\boldsymbol{\mathcal{E}}^{\mathsf{G}_{\mathsf{ref}}}$ of the refresh gadget $\mathsf{G}_{\mathsf{ref}}$ and the cardinal RPC envelopes $\boldsymbol{\mathcal{E}}^{UMM}$ of $\mathsf{UnifMatMult}(m_1, m_2)$, where $m_1 \in \{\lfloor \frac{n_x}{2} \rfloor, \lceil \frac{n_x}{2} \rceil\}$ and $m_2 \in \{\lfloor \frac{n_y}{2} \rfloor, \lceil \frac{n_y}{2} \rceil\}$. We terminate with the base cases $\mathsf{UnifMatMult}(m_1, m_2)$, where $m_1 \in \{1, 2\}$ and $m_2 \in \{1, 2\}$ for which we can assess the cardinal RPC envelopes manually. Throughout the proof, we have

$$0 \le j_1 \le \left\lfloor \frac{n_x}{2} \right\rfloor . \left\lfloor \frac{n_y}{2} \right\rfloor \qquad 0 \le j_2 \le \left\lceil \frac{n_x}{2} \right\rceil . \left\lfloor \frac{n_y}{2} \right\rfloor$$
$$0 \le j_3 \le \left\lfloor \frac{n_x}{2} \right\rfloor . \left\lceil \frac{n_y}{2} \right\rceil \qquad 0 \le j_4 \le \left\lceil \frac{n_x}{2} \right\rceil . \left\lceil \frac{n_y}{2} \right\rceil$$

We develop the formulas hereafter. For all $t^{\mathtt{out}} \in [0, n_x.n_y]$ we have:

$$
\begin{aligned}
\mathcal{E}_{t^{\mathtt{out}}}^{\mathsf{UMM}(n_x,n_y)}(t^{\mathtt{in,x}},t^{\mathtt{in,y}}) = & \sum_{\substack{j_1,j_2,j_3,j_4\\ j_1+j_2+j_3+j_4=t^{\mathtt{out}}}} \frac{1}{\#\{(j_1,j_2,j_3,j_4) s.t. j_1+j_2+j_3+j_4=t^{\mathtt{out}}\}} \\
& \sum_{i^{x_1}=\max(0,t^{\mathtt{in,x}}-\lfloor\frac{n_x}{2}\rfloor)}^{\min(t^{\mathtt{in,x}},\lfloor\frac{n_x}{2}\rfloor)} \sum_{i_1^{x_1}=0}^{i^{x_1}} \sum_{i_2^{x_1}=i^{x_1}-i_1^{x_1}} \sum_{i_1^{x_2}=0}^{t^{\mathtt{in,x}}-i^{x_1}} \sum_{i_2^{x_2}=t^{\mathtt{in,x}}-i^{x_1}-i_1^{x_2}} \\
& \sum_{i^{y_1}=\max(0,t^{\mathtt{in,y}}-\lfloor\frac{n_y}{2}\rfloor)}^{\min(t^{\mathtt{in,y}},\lfloor\frac{n_y}{2}\rfloor)} \sum_{i_1^{y_1}=0}^{i^{y_1}} \sum_{i_2^{y_1}=i^{y_1}-i_1^{y_1}} \sum_{i_1^{y_2}=0}^{t^{\mathtt{in,y}}-i^{y_1}} \sum_{i_2^{y_2}=t^{\mathtt{in,y}}-i^{y_1}-i_1^{y_2}} \\
& \frac{\binom{i_1^{x_1}}{i_1^{x_1}+i_2^{x_1}-i^{x_1}}.\binom{\lfloor\frac{n_x}{2}\rfloor-i_1^{x_1}}{i^{x_1}-i_1^{x_1}}}{\binom{\lfloor\frac{n_x}{2}\rfloor}{i_2^{x_1}}}.\frac{\binom{i_1^{x_2}}{i_1^{x_2}+i_2^{x_2}-t^{\mathtt{in,x}}+i^{x_1}}.\binom{\lfloor\frac{n_x}{2}\rfloor-i_1^{x_2}}{t^{\mathtt{in,x}}-i^{x_1}-i_1^{x_2}}}{\binom{\lfloor\frac{n_x}{2}\rfloor}{i_2^{x_2}}} \\
& \frac{\binom{i_1^{y_1}}{i_1^{y_1}+i_2^{y_1}-i^{y_1}}.\binom{\lfloor\frac{n_y}{2}\rfloor-i_1^{y_1}}{i^{y_1}-i_1^{y_1}}}{\binom{\lfloor\frac{n_y}{2}\rfloor}{i_2^{y_1}}}.\frac{\binom{i_1^{y_2}}{i_1^{y_2}+i_2^{y_2}-t^{\mathtt{in,y}}+i^{y_1}}.\binom{\lfloor\frac{n_y}{2}\rfloor-i_1^{y_2}}{t^{\mathtt{in,y}}-i^{y_1}-i_1^{y_2}}}{\binom{\lfloor\frac{n_y}{2}\rfloor}{i_2^{y_2}}} \\
& \boldsymbol{\mathcal{E}}_{j_1}^1(i_1^{x_1},i_1^{y_1}).\boldsymbol{\mathcal{E}}_{j_2}^2(i_1^{x_2},i_2^{y_1}).\boldsymbol{\mathcal{E}}_{j_3}^3(i_2^{x_1},i_1^{y_2})\boldsymbol{\mathcal{E}}_{j_4}^4(i_2^{x_2},i_2^{y_2})
\end{aligned}
$$

where the envelopes $\boldsymbol{\mathcal{E}}^1$, $\boldsymbol{\mathcal{E}}^2$, $\boldsymbol{\mathcal{E}}^3$ and $\boldsymbol{\mathcal{E}}^4$ are on the form :

$$
\boldsymbol{\mathcal{E}}_{j_1}^1(i_1^{x_1},i_1^{y_1}) = \sum_{\ell_1=0}^{\lfloor\frac{n_x}{2}\rfloor}\sum_{\ell_2=0}^{\lfloor\frac{n_y}{2}\rfloor} \mathcal{E}_{j_1}^{\mathsf{UMM}(\lfloor\frac{n_x}{2}\rfloor,\lfloor\frac{n_y}{2}\rfloor)}(\ell_1,\ell_2).\mathcal{E}_{\ell_1}^{\mathsf{G_{ref}}(\lfloor\frac{n_x}{2}\rfloor)}(i_1^{x_1}).\mathcal{E}_{\ell_2}^{\mathsf{G_{ref}}(\lceil\frac{n_y}{2}\rceil)}(i_1^{y_1})
$$

$$
\boldsymbol{\mathcal{E}}_{j_2}^2(i_1^{x_2},i_2^{y_1}) = \sum_{\ell_1=0}^{\lceil\frac{n_x}{2}\rceil}\sum_{\ell_2=0}^{\lfloor\frac{n_y}{2}\rfloor} \mathcal{E}_{j_2}^{\mathsf{UMM}(\lceil\frac{n_x}{2}\rceil,\lfloor\frac{n_y}{2}\rfloor)}(\ell_1,\ell_2).\mathcal{E}_{\ell_1}^{\mathsf{G_{ref}}(\lceil\frac{n_x}{2}\rceil)}(i_1^{x_2}).\mathcal{E}_{\ell_2}^{\mathsf{G_{ref}}(\lfloor\frac{n_y}{2}\rfloor)}(i_2^{y_1})
$$

$$
\boldsymbol{\mathcal{E}}_{j_3}^3(i_2^{x_1},i_1^{y_2}) = \sum_{\ell_1=0}^{\lfloor\frac{n_x}{2}\rfloor}\sum_{\ell_2=0}^{\lceil\frac{n_y}{2}\rceil} \mathcal{E}_{j_3}^{\mathsf{UMM}(\lfloor\frac{n_x}{2}\rfloor,\lceil\frac{n_y}{2}\rceil)}(\ell_1,\ell_2).\mathcal{E}_{\ell_1}^{\mathsf{G_{ref}}(\lceil\frac{n_x}{2}\rceil)}(i_2^{x_1}).\mathcal{E}_{\ell_2}^{\mathsf{G_{ref}}(\lfloor\frac{n_y}{2}\rfloor)}(i_1^{y_2})
$$

$$
\boldsymbol{\mathcal{E}}_{j_4}^4(i_2^{x_2},i_2^{y_2}) = \sum_{\ell_1=0}^{\lceil\frac{n_x}{2}\rceil}\sum_{\ell_2=0}^{\lceil\frac{n_y}{2}\rceil} \mathcal{E}_{j_4}^{\mathsf{UMM}(\lceil\frac{n_x}{2}\rceil,\lceil\frac{n_y}{2}\rceil)}(\ell_1,\ell_2).\mathcal{E}_{\ell_1}^{\mathsf{G_{ref}}(\lceil\frac{n_x}{2}\rceil)}(i_2^{x_2}).\mathcal{E}_{\ell_2}^{\mathsf{G_{ref}}(\lfloor\frac{n_y}{2}\rfloor)}(i_2^{y_2})
$$

*Initialisation.* The cardinal RPC envelopes for $(n_x,n_y) \in \{(1,1),(1,2),(2,1),(2,2)\}$ are the same than in proof of Lemma 12.

□